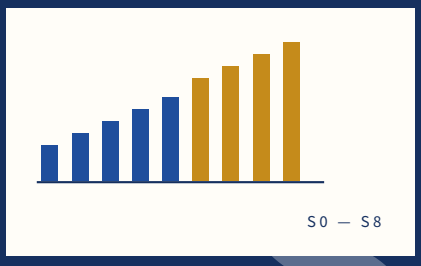
DSWS

从零到一

电商问数智能体 · 设计图集与开发手册

从契约、构建、召回到生成、安全、失败通道与前端收口



S0—S8

Contents

目录

baseline eea5e75 · 2026-10-29

01 第一部分 · 先读这一页 4

契约是事实源，图是论证，阶段是施工顺序，证据是发言资格。

1. 这本书是什么、不是什么 4
2. 一分钟看懂 DSWS 5
3. 三个根因：为什么要重建 7
4. S0–S8 地图 11
5. 证据等级：已验证 / 推断 / 未验证 12
6. 怎么读这本书 13
7. 本书的约定 13

02 第二部分 · S0–S4：从骨架到生成 15

S1 的第一件事，就是逼你回答：你说的「客单价」到底是哪一个。

0. 这一部分的地图 17
1. S0 · 先让管道跑通 19
2. S1 · 契约是唯一事实源 26
3. S2 · 构建元知识 33
4. S3 · 检索 46
5. S4 · 生成 SQL 53
6. 走到这里，系统能做什么、还不能做什么 62
7. 复习题 64

03 第三部分 · S5–S8：从能跑到敢用 66

拒绝要可判定，错误要有名字，类型要保真，部署边界要说清。

0. 这一部分的地图 68
1. S5 · 安全边界 70
2. S6 · 失败通道 83
3. S7 · 前端收口 96
4. S8 · 生产化（边界与下一步） 109
5. 走到这里，系统能做什么、还不能做什么 118
6. 复习题 120

04 第四部分 · 设计图集：三十二个角度 122

同一套系统，从三十二个角度看，会得到三十二个不同的问题。

0. 怎么用这套图 122
1. 三十二个角度索引 123

05 第五部分 · 部署、测试与故障排查 126

先跑起来，再证明没坏，最后才轮到「坏了先看哪一眼」。

1. 两条启动路径：本地开发 vs Docker 126
2. Docker Compose 拓扑 127
3. 环境变量与凭据 132
4. 启动顺序与失败定位 134
5. 隔离：旧系统 / rebuild 135
6. 测试金字塔与阶段门禁 137
7. 失败注入矩阵 141
8. 故障排查表 143
9. 本机边界与未验证清单 146
10. 上线前检查清单 146
11. 排障小结 148

06 附录 149

这份附录只做三件事：索引、速查、对账。

A. 32 张图索引 149
B. 12 个步骤 152
C. 16 个错误码 154
D. 5 表 24 列 6 指标 155
E. 命令速查 157
F. 术语表 159
G. 文件地图 160
H. 证据台账 161

01

第一部分 ·

先读这一页

先看一个旧系统的数字。`Decimal("10")` 从 MySQL 出来，经过 `default=str`，到了浏览器变成 `"10"`。表格还是那张表格，列也对齐，复制到表格软件里却再也不能求和。没有报错，也没有红框。

这本书要处理的就是这类失败：让它有名字、有位置、有证据。

> 没有报错，也没有红框。

1. 这本书是什么、不是什么

这本书是 DSWS 重建项目的使用说明、设计论证和证据台账。S0–S8 每个阶段都写清了输入、动作、输出和硬验收命令，读者可以从空目录一路走到能跑的问数流水线；32 张设计图从架构、运行时、数据、安全、部署、测试六个角度解释同一套系统，每张图只回答一个具体问题。每个结论都带证据标记：✓ 是本机跑过或文件明确存在，! 是从代码结构推导、尚未实跑，? 是环境不具备、没有执行。读者据此判断一句话是验收依据、设计假设，还是待办。

它明确**不是**下面几样东西：

1. **不是旧系统的说明书。** 旧系统 `shopkeeper-agent` 已冻结为 `legacy`，只作为反例库和负向夹具。书里提旧系统只有一个用途：说清哪一种静默失败必须被新系统挡住。兼容旧系统不在范围内。
2. **不是 API 参考。** 接口、步骤、错误码、SQL 策略的唯一事实源是 `contracts/` 下的文件；本书只解释这些文件为什么这样写，不复制一份可能过期的接口表。
3. **不是生产运维手册。** 当前实现是单机教学规模：单 MySQL、单 ES、单 Qdrant、单后端 worker。多租户、权限体系、容量规划、高可用都不在 S0–S7 的范围内，S8 也只给出方向。
4. **不是「782 个测试全部通过」的证明。** README 声称后端 782 passed / 31 skipped，但本次设计工作没有跑全量测试，因此书中一律写作「README 声称 782，本机未跑」，并标 !。类似地，前端 40 passed、14 条 SSE 夹具也来自 README 与契约文件，而不是本轮的实跑输出。

读者对象有三类：第一次接触这个项目、准备按阶段重建的工程师；需要评审「设计是否成立」的架构师或评审者；线上出问题后需要快速定位的运维人员。三类人不需要从头读到尾，§6 给出三条路线。

全书的总原则一句话：契约是事实源，图是论证，阶段是施工顺序，证据是发言资格。缺一条，读者就没法复核书里的任何结论。

2. 一分钟看懂 DSWS

DSWS 是一个电商问数智能体：用户用自然语言问「上个月华东的销售额是多少」，系统把它变成一条只读 SQL，在数仓上执行，再把结果按列类型渲染出来。它不做对话，也不是通用 BI 平台，边界只有四个词：单轮、单库、只读、可拒答。

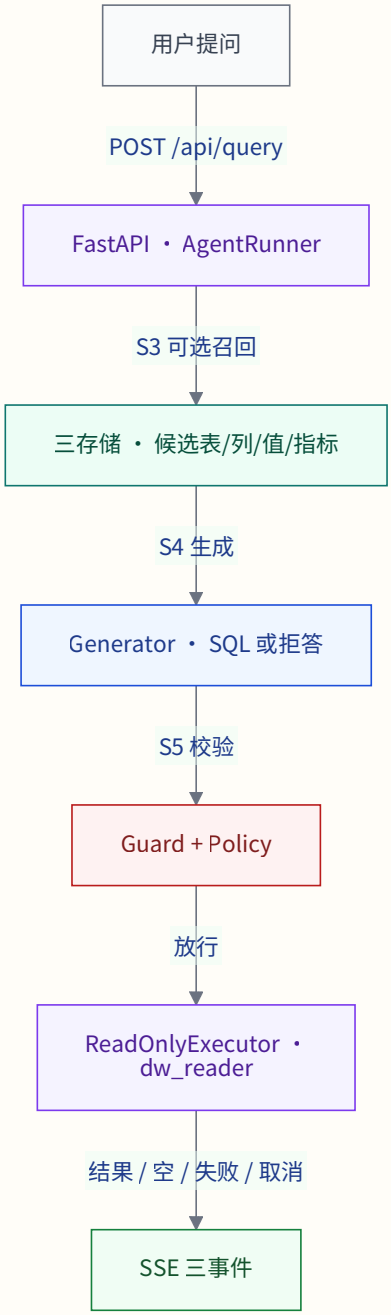


图 00-front

把这套系统压缩成五句话：

1. **用户问数。** 前端只做一件事：把一个自然语言问题、一个取消按钮和一条 SSE 流，组织成一次可观察的运行。它不猜 SQL，也不替后端补数据。
2. **召回。** S3 把问题拆成关键词，去 Qdrant 找列与指标、去 ES 找枚举取值、去 Meta 取详情，输出的是「候选表、候选列、候选取值、候选指标」，不是 SQL。召回为空是一个可记录的 trace，不是一次成功。

3. **生成**。 S4 把候选压进 7 个提示词槽位，交给确定性生成器或真实 LLM，产出「一条 SQL」或「一次拒答」两种形状之一；输出形状先过 JSON Schema，再过 AST 静态校验。
4. **校验**。 S5 在连库之前检查单语句、只读、表白名单、列白名单、常量投影、`ORDER BY` / `LIMIT`，并把注释去掉、把 `LIMIT` 补上。最终执行的是策略重新序列化后的那条 SQL，而不是模型原文。
5. **执行与回答**。 只读账号 `dw_reader` 执行，叠加 `max_execution_time`；SSE 一定以 `result` 或 `error` 收尾。0 行是 `EMPTY_RESULT`（类别 `empty`），用户取消是 `CANCELLED`（类别 `cancelled`），两者都不许伪装成「查询成功」。

五个环节里，生成 SQL 只占一环。剩下的部分大都在处理同一件事：失败怎么被看见、被命名。

3. 三个根因：为什么要重建

旧系统不是「架构不漂亮」才被替换，而是有三类静默失败：界面显示成功，语义已经错了；用户没有任何线索发现。重建的每一层，基本都在回答这三个根因中的一个。

一顿『成功』的晚餐

生活类比 · 界面说上齐了，桌上却没有一道能吃的菜。

生活类比



共同点：三盘都上了，但没有一盘能吃。

新系统：让失败有名字、有位置、有通道。

生活类比：餐厅三幕 · 事实来源：00-front.md §3.1–3.4 · contracts/errors.yaml · docs/S7-HANDOFF.md

图 3-0 · 三种静默失败：界面成功，语义已错

3.1 根因一：伪 SQL 被当成数据

生成失败时，旧系统把一段错误说明塞进「一行一列」的结果里，列名像错误文案，值是一段文本。这个结构在协议上完全合法：有 `columns`、有 `rows`，前端于是按表格渲染。用户看到的是「查询成功」加一句话，而不是一次失败。

这类错误最危险：形状合法，语义完全错。下游复制、导出、再计算，拿到的都是这段文本。

> 形状合法，语义完全错。

新系统的三道止损：

- S5 在连库前拒绝非 `SELECT`、多语句、非白名单表列的 SQL，伪 SQL 形态拿不到执行许可；
- S6 规定失败必须走 `type:"error"`，错误码来自 `contracts/errors.yaml`，不允许把错误文案塞进结果行；
- S7 在展示层用 `looksLikePseudoError()` 做保守识别，命中则渲染 `PseudoErrorCard`，并且**刻意不编造错误码**——错误码只有契约一份，前端硬编一个等于又抄了一份契约。

对应阶段：S5 → S6 → S7。对应图：图 24 · 错误码分类与 UI 映射、图 31 · 失败注入矩阵、图 13 · S7 SSE 事件序列。证据：`docs/S7-HANDOFF.md` §2.4 与 §4.2 的 F5、`fixtures/sse/pseudo_row.json`、`frontend/src/lib/result.ts`。

3.2 根因二：召回为空也算 success

旧系统把空数组 `data: []` 当成一次正常结果：表格消失，页面看起来「跑完了」，但用户既不知道是没有符合条件的数据，还是召回根本没找到东西。审计把这条记为 FP-8：**空数组是合法语义，不是失败，但也不是成功的数据**。

新系统把「空」拆成两个可观察的事实：

- 召回为空时，S3 的 trace 记录候选数；生成层拿不到足够上下文时选择拒答，而不是编造一张表；
- 执行返回 0 行时，S6 发 `EMPTY_RESULT`，类别是 `empty`：界面给虚线空态卡，执行步骤保持绿色，**不标红**；错误卡只在真正失败时出现。

所以新系统不允许用同一种界面表达三种不同的事实：没有数据、召回失败、系统失败。加一张空态卡解决不了这个问题。

对应阶段：S3（可观察的召回）→ S6（分类）→ S7（渲染）。对应图：图 09 · S3 召回流水线时序、图 24 · 错误码分类与 UI 映射、图 13 · S7 SSE 事件序列。证据：`contracts/errors.yaml` 中

`EMPTY_RESULT` 的注释、`docs/S7-HANDOFF.md` §1 的 FP-8 引用、`fixtures/sse/empty_rows.json`。

3.3 根因三：错误被吞掉

「吞掉」有四种常见形态，旧系统至少中了三种：

1. **类型被静默改写**。SSE 序列化传了 `default=str`，MySQL 的 `Decimal("10")` 悄悄变成字符串 `"10"`。界面看起来正常，复制出去的数字已经不可计算。
2. **错误没有归属**。红框出现了，但用户不知道哪一步失败；前端把「progress / result / 其余」全当 error，后端新增一个事件类型就会制造一次莫名其妙的失败。
3. **HTTP 错误被压成一句话**。400/422/5xx/网络不可达都显示「无法连接问数接口」，排障只能靠猜。
4. **没有依赖时编造结果**。没有数据库就返回一份假数据，而不是诚实失败。

新系统的对策是「让失败有名字、有位置、有通道」：

- S6 用 16 个错误码、3 个类别（`failure` / `empty` / `cancelled`）和 `retryable` 描述每一种失败；`SQL_REJECTED` 永远不可重试，避免拿被拒的 SQL 去问模型「改一下」；
- S7 的 `_sse()` 不传 `default=str`，未知类型直接变成 `INTERNAL_ERROR`；HTTP 400/422/404/5xx/网络错误分级给文案；失败步骤名来自 `/api/steps`；
- 没有 MySQL 时，`/api/query` 以一条诚实的 `INTERNAL_ERROR` 收尾，而不是编一份假数据。

对应阶段：S4（输出形状）→ S5（执行错误）→ S6（错误通道）→ S7（界面映射）。对应图：图 24 · 错误码分类与 UI 映射、图 32 · 证据台账、图 13 · S7 SSE 事件序列。证据：`docs/S7-HANDOFF.md` §2.1 与 §2.4、`backend/app/main.py` 的 `_sse()`、`contracts/errors.yaml` 文件头注释。

3.4 根因与阶段的对应关系

根因	旧系统的可见症状	新系统主要由谁承担	图号
伪 SQL 被当成数据	一行一列的错误文案被渲染成成功表格	S5 拒绝 → S6 错误事件 → S7 展示层止损	13、24、31
召回为空也算 success	`data: []` 后表格消失，无任何说明	S3 trace → S6 `EMPTY_RESULT` → S7 空态卡	09、13、24

根因	旧系统的可见症状	新系统主要由谁承担	图号
错误被吞掉	类型被改写、错误无步骤、HTTP 一句话、假数据	S4 形状 → S5 执行 → S6 分类 → S7 映射	13、24、32

4. S0–S8 地图

S0–S7 是已经实现的八个阶段，S8 是后续的生产化方向。每个阶段引入一种新的不确定性，也留一个可停点：到这里为止，系统仍然可演示、可验收、可回退。图号与 `WBS.md` 的 32 张图一一对应。

阶段	新增能力	新增的不确定性	可停点	对应图号
S0 骨架	12 步 SSE 契约、5 个 HTTP 端点、前端协议守卫	前后端能否对同一份事件契约保持一致	后端能起服务，前端能解析/回放事件	01、02、04、05、07、13、29、30
S1 Schema 契约	5 表 / 24 列 / 6 指标；DDL 与真实值校验；旧系统负向夹具	契约是否与真实数据一致（粒度、值域、口径）	`make stage1` 全绿；旧夹具必须红	14、15、30
S2 元知识	Meta / ES / Qdrant 三份可检索知识 + manifest + 幂等	同一份契约翻成三种存储后是否仍然等价	`make stage2`；`--verify` 对账；第二次构建 `NOOP: 0 writes`	06、08、16、17、19、30
S3 检索	20 题黄金集；ES 1 次 `_msearch`、Qdrant 2 次批量、Meta 5 条 SQL	召回质量：哈希向量不是语义模型，候选可能不足	`make stage3`；只读验收，不做 `--fresh`	09、16、19、30
S4 生成 SQL	7 槽位提示词、确定性生成器、JSON Schema 输出校验、AST 静态校验	模型输出形状与语义；真实 LLM 尚未验证	`make stage4`，完全离线也能跑	10、15、21、30、31
S5 安全边界	六道 SQL 检查、只读执行器、36 条注入语料、11 条零误杀	策略覆盖度（新语法、新函数）与真实账号权限	`make stage5`；接 MySQL 用 `stage5-live`	11、20、21、22、23、30、31
S6 失败通道	16 个错误码 / 3 类失败、校正闭环最多 1 次、取消	失败分类是否穷尽；校正是否引入新风险	`make stage6`；浏览器用 `stage6-ui`	07、12、13、24、30、31
S7 前端收口	契约生成物、14 条 SSE 夹具、13 条 F 缺陷闭环	真实召回与真实模型下，界面语义是否仍成立	`make stage7`；浏览器用 `stage7-ui`	04、13、24、30、31、32
S8 性能与生产化	真召回接主链路、真库、真 LLM、并发/超时/缓存/可观测	容量、并发、真实数据分布、多租户	尚未实现；本书只保留设计与边界	05、25、26、27、28、29、32

这张表有两处容易读错。阶段不是功能清单，是不确定性清单：每加一层能力，就多一个可能出错的地方，所以每层都要有硬验收。可停点也不等于可上线：S3 通过只说明召回流水线成立，不说明前端能真

实问数；S7 通过只说明协议、契约和界面收口成立，真实召回、真实数据库与真实 LLM 仍属于 S8。

5. 证据等级： / /

证据标记在这本书里是要兑现的：标 得能指到命令或行号，标 得写清推导链，标 得说明缺什么。

图标对照：圆形勾号 = 已验证；圆形感叹号 = 推断；圆形问号 = 未验证。

标记	含义	读者可以怎么用
已验证	本机命令跑过、退出码为 0，或文件明确存在且可逐行引用	可以作为验收依据；仍要注意环境差异
推断	从代码、契约或配置推导出来，但没有实跑	可以作为设计假设，不能当作已验收
未验证	环境不具备或本轮没有执行	只能作为待办或风险，不能作为结论

读者常把 当 用：照着做失败时，会误以为是自己的环境错了；评审时会把「代码里应该有」当成「系统确实做过」。把推断写成已验证，风险就转给了读者。反过来，把 写成 会浪费已有的证据，让读者重跑一遍。

本机的三条硬边界，必须在读任何结论前知道：

1. **Docker socket 无权限。** 本设计工作树没有跑 `docker compose up`，也没有跑 S2 的 `--fresh` 写三存储。S2 的计数来自 `--dry-run` 与 `plan.counts()`；写入总数 241 是按代码公式推导（44 行 Meta + 76 ES + 98 + 22 Qdrant + 1 manifest），标 ，不是实测。
2. **`dw_reader` 密码不在仓库里。** 只读账号密码由 `REBUILD_DW_RO_PASSWORD` 提供，没有默认值。没有这个变量，`make stage5-live` 不会跑；只读账号的 GRANT 与执行器行为来自代码、脚本和假连接测试，标 。
3. **没有 LLM 端点与 Key。** 生成默认走确定性生成器，`correct_sql` 在未配置 LLM 时显示「未触发」。`HttpLlmGenerator` / `LlmCorrector` 的接口、提示词与校验都在，但没有对着真实端点跑过，标 。

另外两条本轮边界：全量后端测试没有跑，README 声称 782 passed / 31 skipped（）；前端浏览器验收本轮没有跑，`stage0-ui` / `stage6-ui` / `stage7-ui` 的历史结论来自文档（），本轮状态为 。

6. 怎么读这本书

路线一：新手照着做。 先读本章 §1–§2，建立「问数五环节」的心智模型；然后读 `10-s0-s4.md` 与 `20-s5-s8.md`，按阶段执行 `make stage0` 到 `make stage7`。每跑完一个阶段，回到 `30-atlas.md` 找对应图号，确认自己刚才跑的东西在整条链路里的位置。遇到需要三存储的阶段（S2/S3/S5-live），先读 `40-ops.md` 的启动顺序，不要在没有 `source scripts/export-docker-env.sh` 的情况下直接跑 CLI。S8 只读设计，不照着做。

路线二：评审看设计。 先读本章 §3–§4，知道三个根因与九个阶段；再读 `30-atlas.md` 的架构角度（图 01–06）与数据/安全角度（图 14–24），核对「契约是否真的贯穿构建时与查询时」「安全是否真的在连库之前」；最后读图集 §7「图集没有回答的问题」，把 ⓘ 项列入评审待办。评审时看每张图的「证据与边界」有没有指到文件或命令，图漂不漂亮不作数。

路线三：运维查故障。 直接读 `40-ops.md`，再按症状查图：图 25 · Docker Compose 拓扑 看服务依赖，图 26 · 端口与网络暴露面 看为什么 8080 通而 8100 不通，图 27 · 启动顺序与健康检查 看 `init` 卡住会停在哪，图 24 · 错误码分类与 UI 映射 看用户看到的那句话对应哪个码，图 31 · 失败注入矩阵 看这个码能不能被稳定复现，图 32 · 证据台账 看这条结论到底验没验过。命令层面的排障步骤以 `docs/SETUP-FROM-SCRATCH.md` §6 为准。

三条路线共用一条纪律：先看证据标记，再看结论。一个 ⓘ 的结论不值得你花半小时复现；一个 ✓ 的结论，也仍要确认自己的端口、目录和依赖版本与书中一致。

7. 本书的约定

命令。 所有 `make` 命令都在源码仓库根执行；设计工作树的命令会先写 `cd design`。代码块里不写提示符，直接给可复制的命令。需要环境变量的命令会显式写出变量名，例如 `REBUILD_DW_RO_PASSWORD=... make stage5-live`，不暗示仓库里有默认密码。

路径。 源码路径相对仓库根，例如 `backend/app/main.py`、`contracts/steps.yaml`、`fixtures/dw.sql`。设计产物路径相对 `design/`，例如 `diagrams/02-runtime/08-s2-build-sequence.svg`、`inventory/facts.md`。章节里的图片路径相对章节文件本身（`design/book/chapters/`），所以统一写成 `../../`

`diagrams/<角度>/<编号>-<短名>.svg`，解析后落在 `design/diagrams/**`。

端口。 后端 `8100`，Vite `5273`，前端容器 `8080 → 80`，MySQL `3306`，Elasticsearch `9200`，Qdrant `6333`。临时验收端口：S0 用 `8101/5274`，S6 用 `8102/5275`，S7 用 `8103/5276`，刻意避开旧系统的 `8000/5173`。除 `8080` 外，其余端口默认只绑 `127.0.0.1`。

计数。 契约 5 表 / 24 列 / 6 指标 / 9 条列一指标关系；12 个步骤；16 个错误码、3 个类别；3 种 SSE 事件；5 个 HTTP 端点。S2 三存储：Meta 5/24/6/9（合计 44 行）、ES 76 条取值文档、Qdrant 98 条列文本 + 22 条指标文本、manifest 1 条；写入总数 241 是公式推导（!）。代码规模：后端 60 个 Python 文件、45 个测试文件，前端 2 个测试文件。测试数一律写「README 声称 782 passed / 31 skipped，本机未跑」，不写成已验证。

图号。 图号 `01 – 32` 在全图集内唯一，不按目录重排。正文首次提到某图时给「图 NN · 标题」并链接；完整说明集中在第四部分 `30-atlas.md`。图题在图下方，表题在表上方；表号用「表 2-1」这种「章-序」格式，只在需要交叉引用时编号。

证据标记。 ✓ / ! / ? 只出现在证据句或证据表里，不用来修饰产品能力。一个段落如果同时包含事实与推断，会分开写，不把两种语气混在一句话里。

术语。 「构建时」指 S2 一次性 CLI 任务，退出后不参与查询；「查询时」指 `/api/query` 触发的 FastAPI + AgentRunner 链路。「召回」专指 S3 的候选生成，不指最终 SQL。「伪 SQL」指形状像结果、内容是错误文案的载荷。「空结果」专指执行成功但 0 行，不等于召回为空。「失败通道」专指 S6 的错误码、类别、校正与取消机制。术语首次出现时给英文标识符，之后用中文；英文标识符保留原文，不改写。

7.1 如果只记三件事

- 契约是事实源。`contracts/` 里的表结构、步骤、错误码改一个字，代码和前端都得跟着对账。
- 证据分三级。✓ 是本机跑过或文件明确存在，! 是推导，? 是没跑；别把 ! 说成 ✓。
- 失败要有名字。`contracts/errors.yaml` 里的错误码决定用户看到什么，也决定能不能重试。

02

第二部分 ·

S0–S4：从骨架到生成

旧系统的「客单价」能同时算出 2427.47、448.81、5.41。2427.47 是订单金额除以订单数，448.81 是金额除以件数，5.41 是件数除以订单数；三个都叫「客单价」，三个都能在 SQL 里自圆其说。

S1 的第一件事，就是逼你回答：你说的「客单价」到底是哪一个。

口径错误通常不会报错：它只会安静地给出另一个能自洽的数字。换一个分母、换一个聚合对象，报表就换一个数，每个数看起来都对。

不过，S1 之前还有一条更基础的管道。前端不知道后端内部有几个步骤，后端发了一个前端不认识的事件，前端忽略它，页面停在半路——这是 S0 要先解决的麻烦。先把管道跑通，再谈口径。

这一部分要回答的问题可以压成一句：一条问数流水线，怎样从「前后端能对上话」走到「能生成一条可校验的 SQL，或者给出一次有名字的拒答」？两个限定词值得停一下：可校验，意味着产物要能被机器检查；有名字，意味着拒绝也要有理由，不能只留一个空白的界面。

每一个阶段都配四件东西：一份契约，说明谁和谁对齐了什么；一个可停点，说明停在这里还能演示什么；一条验收命令，说明凭什么说它成立；一份证据，说明这条结论是跑过、推导，还是没跑。四件事凑齐，一个阶段才算完成；缺一件，它就只是一个功能模块，进不了评审。

这一部分讲「一条问数流水线怎么从零长出来」。组织方式是五种不确定性，不是功能清单：S0 先让前后端对同一种事件语法达成一致；S1 把「表结构、口径、值域以谁为准」钉死；S2 把契约翻译成三种可检索的存储；S3 证明检索能在 20 个真实问题上站得住；S4 把上下文变成一条可被机器校验的 SQL 或一次有名字的拒答。每一步都留下一个可停点：停在这里，系统仍然能演示、能验收、能回退。

读这一部分时请始终带着五个问题：这个阶段消灭了哪个不确定性？实体是谁？输入是什么？输出写到哪？查询时它还在不在场？后面的每一节都按这五个问题展开，最后用「怎么证明它真的成立」收口。命令以 `Makefile` 与 `docs/SETUP-FROM-SCRATCH.md` 为准；计数以

`design/inventory/facts.md` 为准；证据标记遵守 `design/STYLEGUIDE.md` 的 ✓ / ! / ? 纪律。

> **读这一部分之前**
> 你应当已经读过第一部分，知道 DSWS 是一个单轮、单库、只读、可拒答的电商问数智能体；会写 Python，能看懂 SELECT、JOIN、GROUP BY；知道 HTTP 请求与 JSON 响应长什么样。SSE、Pydantic、AST、Qdrant、Elasticsearch 会在本章第一次出现时解释，不需要提前学过。
>
> **读完这一部分，你应当能**
> - 说出 S0–S4 各自消灭了哪个不确定性，以及停在该阶段时系统还能做什么；
> - 在给定故障现象时，判断该查契约、查存储、查召回还是查生成层；
> - 跑通 S0、S1、S4 的离线门禁，读懂 S2、S3 的记录输出，并说清它们为什么仍是推断或未验证；
> - 解释为什么写入总数 241 只能算公式推导，以及 S3 为什么不进生产主链路。

本章首次出现的几个词，先给一句话：SSE（Server-Sent Events）是服务器单向推送的事件流，浏览器按行读取；Pydantic 是 FastAPI 用来校验请求体的库，字段为空或超长会在进入处理函数之前被拒绝；AST（抽象语法树）是把 SQL 解析成树之后的结构，策略与校验都在这棵树上做；Qdrant 是向量库，按向量相似度找候选；Elasticsearch（下文简称 ES）是基于倒排索引的搜索引擎，按词找取值。

> **动手之前**
> 先按 `docs/SETUP-FROM-SCRATCH.md` 装好工具链；S0、S1、S4 不依赖外部服务，S2、S3 需要 Docker。把这一章当实验手册用时，每节按「命令 → 期望输出 → 失败先查哪里」读：验收小节给命令与输出，常见故障小节给排查顺序。没有 Docker 时，S2 用 `--dry-run` 验证计划，S3 只能读记录输出；不要把「计划通过」当成「存储通过」。

0. 这一部分的地图

把 S0–S4 想成盖房子：S0 先定电线和插座的标准，S1 画承重墙和房间用途，S2 把材料按三个仓库分门别类放好，S3 练习「用户说一个词，能不能找到对的材料」，S4 才真正开始砌墙。标准没定就砌墙，后面每改一次协议都要拆一次前端；材料没编号就开工，工人只能靠记忆找砖。

这个类比有用，但它有一个局限：盖房子可以返工，口径错了不会有人报警。房子塌了看得见；一条把件数当订单数的 SQL 会安静地返回一个数。所以这套流水线每个阶段都要配一份可执行的验收，类比只负责帮你记住顺序，不负责证明顺序是对的。

把五个阶段各压成一句：

- **S0 统一语言。** 前后端对有哪些步骤、事件长什么样、流怎么结束达成一致。
- **S1 统一语义。** 表、列、指标、值域以一份契约为准，口径写成可校验的断言。
- **S2 统一物化。** 把契约翻译成查询时真正能用的三种存储，并让三份表示在计数、维度与样例内容上可对账（阶段表里的「等价」即此意）。
- **S3 把召回变成可回归的验收。** 用固定问题集检查检索能否召回对的表、列、值与指标、会不会夹带噪声；它不证明开放问法的召回质量。
- **S4 收口产物。** 把上下文变成一条可静态校验的 SQL，或者一次带原因的拒答。

阶段	新增能力	新增不确定性	可停点	前置
S0 骨架	12 步步骤目录、3 种 SSE 事件、5 个 HTTP 端点、前端协议守卫	前后端是否真的按同一份事件契约说话	后端能起服务，前端能解析事件并以 `result` / `error` 收尾	`make install`
S1 契约	5 表 / 24 列 / 6 指标；粒度、度量、值域、别名；旧系统负向夹具	契约是否与 DDL、真实数据、口径一致	`make stage1` 全绿，且旧夹具必须红	S0
S2 元知识	Meta / ES / Qdrant 三份可检索元知识 + manifest + 幂等	同一份契约翻成三种存储后是否仍然等价	`make stage2`；`--verify` 对账；第二次 `NOOP: 0 writes`	S1 + 三存储
S3 检索	8 个确定性节点、批量召回、20 题黄金集、稳定 YAML	召回质量：候选够不够、噪声多不多	`make stage3`；只读验收，不重建	S2
S4 生成	7 槽位提示词、确定性 / LLM 生成器、输出 schema、AST 静态校验	产物形状与语义；真实 LLM 尚不可控	`make stage4`，完全离线可跑	S1（S3 可选）

S3 那一行的「召回质量」是它要处理的问题，不是它已经证明的结论；§4 会把结论收窄成「固定问题集上可回归」。

表里有个容易忽略的细节：S2 与 S3 需要三存储，S0、S1、S4 不需要。这是刻意的。能离线判定的东西不绑 Docker；必须碰真实存储的，前置条件写在 `Makefile` 注释里，不藏在某个人脑子里。

表里的「可停点」值得单独解释。停点并不意味着做到一半；它意味着停在这里仍然能演示、能验收、能回退。S0 的停点是前端能把一条流解析到 `result` 或 `error`；S1 的停点是契约与真实数据对账通过；S2 的停点是三份计数与 manifest 对得上；S3 的停点是 20 题黄金集全过；S4 的停点是基线全过、拒答按预期出现。停点之前的阶段都能独立运行，所以排障时先问「最后一条绿的验收是哪条」，比问「代码改到哪了」更快定位。

本章的证据标记分三级：已验证（本机跑过，或文件、测试记录明确存在）、推断（从代码或文档推导，未实跑）、未验证（环境不具备或未执行）。标记沿用基线，已验证不等于本轮复现，表后会注明覆盖范围；图例见 `design/STYLEGUIDE.md`。

上面那张阶段表只覆盖 S0–S4。S5 讲安全执行，S6 讲失败通道与错误码，S7 讲前端收口，S8 讲生产化边界；本部分只在必要处提它们的名字，不展开。

如果只有一节课，主线是 S0 → S1 → S4；S2 与 S3 可以课后自学，课内只保留它们留下的两条边界：三存储的分工，以及召回质量还没有被证明。

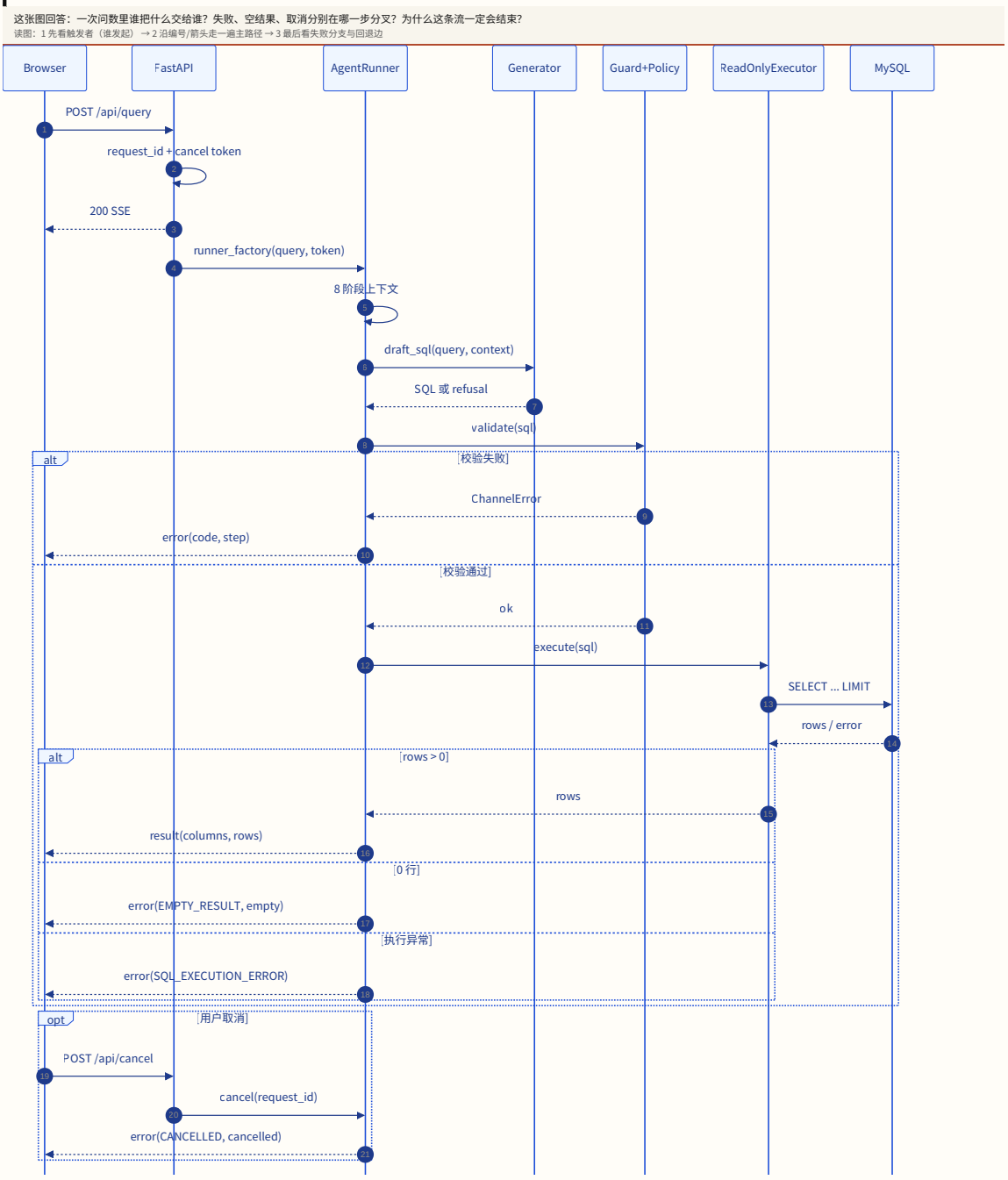


图 07 · 端到端请求生命周期

图 07 把五个阶段放回一次真实请求里看：S0 定义这条请求的语法，S1 提供上下文的事实，S2 与 S3 负责找材料，S4 负责写 SQL。图中 S3 是虚线可选路径，后面不少边界都从这条虚线来。

1. S0 · 先让管道跑通

S0 只保证一件事：前后端对同一件事的说法一致。SQL 写得对不对，不归它管。旧系统把 12 个步骤节点硬编码在前端，后端新增一个事件类型，前端就可能把它当成失败；条件步骤 `correct_sql` 永远灰着，用户看不出它是「没触发」还是「卡住了」。S0 的做法很朴素：把步

骤目录和事件语法抽成契约文件，两端都只认这份契约。后面流水线再复杂，「谁在什么时候说了什么」至少能对上。

1.1 问题

没有 S0 时，最常见的故障是沉默，不是报错：后端发了一个前端不认识的事件，前端忽略它，页面停在半路；或者后端少发了一个步骤的终态，进度条永远差一格。更隐蔽的是「条件步骤」——`correct_sql` 在设计上可能不执行，如果它没有显式的 `skipped`，前端只能显示一个永远 pending 的灰块。S0 要求每一个步骤都有结论，每一条流都必须以 `result` 或 `error` 收尾。缺了这两条，前端手上就没有任何能判断的东西。

`correct_sql` 是校验失败后按错误码重写 SQL 的那一步；它是步骤目录里唯一可能不执行的步骤，所以前端不能把它画成永远 pending。它的触发条件与重试上限在 §5.4 讲。

> **关键问题**：前端并不知道后端内部有几个步骤、哪一步会失败。它凭什么判断现在该转圈、该打勾，还是该显示未触发？S0 的答案是把这份名单交给契约，让两端读同一份文件。

把这条原则说得更一般一点：协议里凡是前端需要知道的东西，都不能只存在于后端的代码里。步骤目录、事件类型、错误码都属于这一类；S0 只处理前两样，错误码要到 S6 才补齐。

1.2 实体与边界

S0 的实体只有四个：**步骤**（`id`、`label`、`conditional`）、**事件**（`progress` / `result` / `error`）、**请求**（`query`，长度 1–500）、**请求标识**（`request_id`，用于取消与排障）。步骤的身份由 `contracts/steps.yaml` 定义，事件的形状由 `contracts/sse.schema.json` 定义，两者都不允许在代码里另抄一份。

把 SSE 再说得具体一点：它是一条长连接，服务器不断往同一个响应里写 data: 这样的行，浏览器按行解析。它没有请求—响应那种一次问、一次答的结构，所以「流什么时候算结束」必须由协议显式规定。S0 规定的结束方式只有两种：`result` 或 `error`。

S0 只管 HTTP 表面与事件语法。SQL 语义、三存储、安全策略都不在它的边界内。当前仓库的 `backend/app/main.py` 已经是 S6/S7 的真实流水线，但 S0 的验收仍然成立——`backend/tests/test_s0_contract.py` 只检查「目录下发了 12 步」「每条事件都过 schema」「流一定收尾」，并不假设流里一定是假数据。历史上的

S0 是一条永远成功的假流；现在的流可能以 `error` 收尾，契约测试接受这一点：失败也得有形状。

1.3 输入

输入	内容	计数	证据
`contracts/steps.yaml`	12 个步骤，`correct_sql` 标为 `conditional: true`	12 / 1	文件本身
`contracts/sse.schema.json`	`progress` / `result` / `error` 三种事件	3	JSON Schema
`QueryRequest`	`query` 字段，1–500 字，空白串拒绝	—	`backend/app/main.py:QueryRequest`
`backend/tests/test_s0_contract.py`	契约测试：目录、生命周期、边界、CORS	19 条	测试文件

先把 12 步全表列出来，后面引用某一步时不再重复解释。id 是契约里的键，前端显示的是 label；只有 `correct_sql` 标了条件执行。

步骤 id	前端显示	条件执行
extract_keywords	抽取关键词	否
recall_column	召回字段信息	否
recall_value	召回字段取值	否
recall_metric	召回指标信息	否
merge_retrieved_info	合并召回信息	否
filter_table	过滤表信息	否
filter_metric	过滤指标信息	否
add_extra_context	添加额外上下文	否
generate_sql	生成SQL	否
validate_sql	校验SQL	否
correct_sql	校正SQL	是
run_sql	执行SQL	否

这张表是前端读到的完整目录；本章只走到生成与静态校验，最后一步执行 SQL 属于 S5/S6。目录是渲染顺序，不是执行次数：校正后重新校验，同一个步骤 id 的结论覆盖前一次，不会多出一格。

步骤目录的顺序有实际用途：它同时是前端步骤条的渲染顺序、SSE `progress.step` 的合法值域，以及 `correct_sql` 能否跳过的依据。12 个步骤里只有 `correct_sql` 是条件步骤；这个「恰好 1 个」被测试硬断言，防止未来有人顺手把别的步骤也标成条件而不改前端。

1.4 动作

先安装依赖，再跑门禁：

```
make install
make stage0
```

`make install` 做两件事：`cd backend && uv sync --frozen` 与 `cd frontend && npm ci`。前者还原后端虚拟环境，后者按锁文件安装前端依赖；首次需要网络，装完之后 S0/S1 可以离线跑。

`make stage0` 依次做三件事：跑 `tests/test_s0_contract.py`、跑前端 `npm run test`、跑前端生产构建。真实的步骤在 `Makefile:stage0`，不是本文重写的。可选的浏览器冒烟是 `make stage0-ui`，它会自起 8101 / 5274 两个临时端口，跑完清理；没有 Chrome / Edge 时会失败，这属于环境前置，不是代码回归。

代码路径只需要记四个：

- `backend/app/main.py:create_app()`：应用工厂，HTTP 与 SSE 管道都在这里装配。
- `backend/app/main.py:get_steps()`：读取 `steps.yaml` 并下发目录；前端不再硬编码节点。
- `backend/app/main.py:_sse()`：把事件编码成 `data: <json>\n\n`，刻意不传 `default=str`。
- `backend/app/main.py:_stream()`：把同步编排器放进工作线程，事件经队列回到协程，流结束时一定收尾。

读这段代码只需要抓住三件事：主事件循环、把事件从工作线程送回主循环的函数、队列的终止信号。下面这段解释了流为什么不会卡死：

```
def worker() -> None:
    try:
        for event in runner.events():
            loop.call_soon_threadsafe(queue.put_nowait, event)
    except Exception as exc:
```

```
            loop.call_soon_threadsafe(queue.put_nowait, error.to_event())
        finally:
            loop.call_soon_threadsafe(queue.put_nowait, None)
```

`loop.call_soon_threadsafe` 负责把事件送回主循环，`queue.put_nowait` 负责入队，`None` 负责告诉消费者流结束了。示意代码省略了 `error` 的构造：完整实现先用 `build_error()` 把异常转成 `INTERNAL_ERROR`，再调用 `error.to_event()`，见 `backend/app/main.py:_stream()`。

同步生成器里要跑数据库，直接塞进异步响应会阻塞事件循环；所以它跑在工作线程，`None` 是队列的终止信号。任何异常都会被构造成一条 `INTERNAL_ERROR` 事件，而不是让连接悄悄断掉。

这里有个容易忽略的因果：数据库驱动是同步的，同步调用会把事件循环占住；事件循环一停，SSE 就写不出去，前端看到的是卡住，而不会收到一条错误。把生成器放进工作线程、用队列把事件送回协程，等于把慢限制在一个线程里，让主循环继续有能力写出事件、处理取消。`None` 是队列的句号；没有这个句号，消费者永远等不到流结束。

1.5 输出

S0 的输出是一组两边共同引用的协议产物：

1. `GET /api/steps` 返回 `version` 与 12 个步骤，前端据此渲染步骤条。
2. `POST /api/query` 返回 `text/event-stream`，每条事件都能被 `contracts/sse.schema.json` 验证。
3. `GET /api/health` 返回 `{"status":"ok","stage":"S7"}`（阶段号随实现推进，形状不变）。
4. 前端得到可解析的事件类型、可显示的步骤标签、以及一个明确的终态。

查询时这些产物还在不在场？在。`steps.yaml` 在应用导入时被读入 `STEPS`，`sse.schema.json` 是前端解析与测试的共同依据；它们不是一次性构建产物，而是运行时契约。区别只在于：S0 当时只有假流，现在同一条管道里跑的是真流水线。

常见误解：S0 过了，SQL 就对了

S0 通过只证明协议对上了：步骤目录一致、事件形状合法、流能收尾。它不检查 SQL 的语义，也不检查数据库能不能连。把 S0 当端到端验收，会在 S4 或 S5 第一次遇到真正的语义错误。判断一个阶段验

了什么，只看它的命令碰不碰存储、碰不碰模型：`make stage0` 两样都不碰。

1.6 验收

```
make stage0
```

期望输出（`docs/SETUP-FROM-SCRATCH.md` §2.3 记录的历史结果）：

```
19 passed
18 passed
S0 PASS
```

三段输出要分开理解：`19 passed` 是后端契约测试，`18 passed` 是当时的前端协议单测，`S0 PASS` 是 Makefile 在所有子命令退出码为 0 后打印的结论。§3 里写的「前端 40（S7 扩写后）」是后续阶段扩写后的累计数量，不是 S0 当天的数量；不要把两个口径混在一起。

失败含义也要能读出来：如果后端测试红，先看是目录数量、事件 schema 还是流收尾；如果前端测试红，先看 SSE 解析器与类型守卫；如果 `npm run build` 红，通常是类型错误或依赖没装，而不是协议本身。`make stage0` 不碰数据库，所以它红了基本可以排除 MySQL / ES / Qdrant。

本设计工作树在 2026-10-29 实跑了后端部分：

```
19 passed, 1 warning in 2.95s
```

前端 `node_modules` 未安装，因此 `npm run test` 与构建在本轮标 ⑦，不冒充已验证。

例题：`skipped` 为什么不能当成成功

题目：`correct_sql` 在没配置 LLM 时不会执行。后端给它发一条 `progress`，状态是 `skipped`。前端如果把它当成成功，用户会看到什么？如果当成失败，又会看到什么？

解答：当成成功，步骤条会给校正这一步打勾，用户以为系统尝试过校正并且成功了；实际上下一步执行用的还是原来那条 SQL。当成失败，步骤条会标红，错误卡会指向一个根本没有失败的步骤，用户会去排查一个不存在的故障。两种误读都来自同一个错误：把「条件未触发」塞进了「成功 / 失败」的二值里。

`skipped` 的语义是「这次不需要执行」，所以它既不打勾也不标红；它的耗时字段必须是空值，前端显示「未触发」，步骤条继续往下走。S0 在契约里给状态留了四个合法值，就是为了让这第四种情况有地方待，而不是被迫挤进前两种。

四个值是 running、success、error、skipped；只有 success 与 error 允许带耗时，running 与 skipped 的耗时必须是空值。这里的 error 是步骤状态；整条流的终止事件也叫 error，同名但不同层：一个是某一步的结论，一个是流的收尾。

结论：一个状态机如果只有成功与失败，条件步骤就一定会被误读。多出来的那个状态承载的是「这次没轮到它」这个事实，不能省。

1.7 常见故障

故障	先查什么	怎么修
`make stage0` 在 `npm ci` 失败	Node / npm 版本、网络、`frontend/package-lock.json` 是否完整	装当前 LTS；确认能访问 npm registry；不要手改 lock 文件
后端 19 条里目录断言失败	`contracts/steps.yaml` 是否被改动、`GET /api/steps` 是否仍读同一文件	恢复 12 步与唯一的 `conditional: true`；不要在 `main.py` 里另写列表
流没有以 `result` / `error` 收尾	`AgentRunner.events()` 的兜底、`_stream()` 的 `finally`	保留兜底异常转 `INTERNAL_ERROR`；不要吞掉异常后直接 `return`
条件步骤没有状态	`_skip_correct()` 是否在每条提前返回路径上被调用	任何提前结束都要显式发 `skipped`，不能留 pending
501 字没有返回 422	Pydantic `Field(max_length=500)` 是否被绕过	保持 `QueryRequest` 的长度约束；空白串单独拒绝
前端 CORS 报错	`create_app()` 的 `allow_origins` 是否只含 5273	开发源就是 `127.0.0.1:5273` / `localhost:5273`；不要用 `*` 掩盖问题
`make stage0-ui` 起不来	机器上有没有 Chrome / Edge、8101 / 5274 是否被占	装浏览器或用 Edge；先释放临时端口，再重跑

表里的三个环境词先解释：422 是请求体校验失败；CORS 是浏览器的跨域限制；5273 是前端开发端口。

1.8 证据

结论	状态	证据
步骤目录 12 步、1 个条件步骤	✓	`contracts/steps.yaml`；`tests/test_s0_contract.py:test_steps_catalog`

结论	状态	证据
SSE 只有 3 种事件，且 `error` 必带 `class` / `step`	✓	`contracts/sse.schema.json`；schema 负例测试
后端 S0 契约测试 19 条通过	✓	2026-10-29 本工作树 `uv run pytest -q tests/test_s0_contract.py`
前端协议单测 18 条、构建通过	!	`docs/SETUP-FROM-SCRATCH.md` §2.3 历史记录；本轮未装 `node_modules`，未重跑
浏览器冒烟	?	本轮未执行；需要 Chrome / Edge

1.9 检查清单

- ☐ `contracts/steps.yaml` 仍是 12 步，且只有 `correct_sql` 标 `conditional`。
- ☐ `GET /api/steps` 的 `version` 与 `steps` 逐字来自 YAML，没有第二份硬编码。
- ☐ 每条 SSE 事件都能被 `contracts/sse.schema.json` 验证。
- ☐ 每条流都以 `result` 或 `error` 收尾；条件步骤有 `skipped` 或终态。
- ☐ 查询长度边界是 500 通过、501 返回 422；空白串返回 422。
- ☐ 能说清楚「S0 验证的是协议，不是 SQL，也不是数据库」。

2. S1 · 契约是唯一事实源

S1 要消灭的是「语义不确定性」：DDL（建表语句）能告诉你 `fact_order` 有 `order_amount`，却不会告诉你它是整单金额还是单价；能告诉你 `dim_region` 有 `region_name`，却不会告诉你它是省还是大区、是不是唯一键。旧系统把这些解释写在 YAML 和提示词里，没有一行代码强制，于是 AOV 同时存在 2427.47、448.81、5.41 三种口径，三种都能「自圆其说」。S1 的做法是把这些解释变成一份可执行校验的契约，再用旧系统当负向夹具，证明检查器真的会红。

2.1 问题

如果没有 S1，后面每个阶段都会各自解释一遍数据：S2 可能把 `order_amount` 当单价建索引，S3 可能把 `region_name` 当省召回，S4 可能生成一条语法正确但口径错误的 SQL。这些错误不会在编译期暴露，也不会在「查询成功」时暴露；它们只会让用户拿到一个看起来合理、实际错误的数字。S1 要做的是让粒度、度量、值域从注释变成断言，多写一个 YAML 只是手段：`dim_region` 一行一个 `region_id`，`region_name` 是省的上级分组，不是唯一键；

`fact_order` 一行一单，`order_quantity` 是件数，`order_amount` 是整单金额。

把三种口径并排写出来：2427.47 是金额除以订单数，448.81 是金额除以件数，5.41 是件数除以订单数。三条 SQL 都能执行，三张结果表都长得像正确答案。S1 不替业务选一个数；它把「你指的是哪一个」变成契约里一条能检查的公式，选择权还在业务，检查权归机器。

把三个数的算式写出来：279159.5 ÷ 115 ≈ 2427.47；279159.5 ÷ 622 ≈ 448.81；622 ÷ 115 ≈ 5.41。三个结果来自同一份数据，只是分母不同。5.41 是四舍五入后的值，所以 448.81 × 5.41 不会精确回到 2427.47；要复核就用 622 ÷ 115，不要用乘法反推。

2.2 实体与边界

S1 的实体是一套四层结构：**数据集**（`dw`，5 表、115 行、2025 Q1）→ **表**（`role`、`grain`、主键、外键、表级度量）→ **列**（类型、角色、可空、别名、值域）→ **指标**（标签、公式、依赖列、聚合方式、期望值、拒收值）。`grain` 说的是「一行代表什么」：`dim_region` 一行一个 `region_id`，`fact_order` 一行一单；写 GROUP BY 时按什么分组，用的就是这一层的粒度。此外还有 **Issue**：检查器发现的每一条问题都有代码、路径、说明，代码集合是发布门禁的一部分。

S1 的边界：它只负责「契约 ↔ DDL ↔ 数据 ↔ 旧夹具」的一致性，不负责建库、不负责写三存储、不负责生成 SQL。它可以在没有 MySQL / Docker / LLM 的机器上跑，因为它只依赖 Python 标准库 `sqlite3` 和 PyYAML。查询时检查器不在场，但契约文件在场：S2 用它建知识，S3 用它做过滤与字段保真，S4 用它裁剪上下文。

> **设计原则：检查器必须能红。** 一个永远返回 PASS 的检查器，比没有检查器更危险：它会让团队误以为契约已经被守住。S1 用旧系统当负向夹具，要求同一套检查在旧配置上报出固定的问题集合；如果它突然变绿，说明检查器漏检，旧系统并没有变好。这条原则后面还会以别的形式出现：S5 要求注入语料全部被拒，S6 要求失败场景逐项命中。

2.3 输入

输入	内容	计数	证据
`contracts/schema.contract.yaml`	5 表、24 列、6 指标、9 条列—指标关系	5 / 24 / 6 / 9	YAML 解析
`fixtures/dw.sql`	物理 DDL + 真实数据，装进内存 SQLite	115 行事实	`schema_contract.py:load_ddl()`

输入	内容	计数	证据
`fixtures/legacy/`	旧系统元数据与功能清单，负向夹具	必须检出 8 个代码	`EXPECTED_LEGACY_CODES`
`docs/SCHEMA-CONTRACT.md`	人读版契约，由契约生成	必须新鲜	`--check-doc`

负向夹具是一份故意写错的旧配置，用来要求检查器报错；它必须红，绿了说明检查器漏检。

`dim_region.region_name` 是理解「值域」的好例子：它的 `value_domain.kind` 是 `enum`，值域写了 `[华南，华东，西南，华北，华中]`，别名是 `[地区，区域，大区]`。这告诉后面的阶段两件事：它可枚举、可用于精确匹配；它描述的是大区，不是省，也不是城市。`fact_order.order_amount` 则是另一种：`kind: range`，范围 30.0–9499.0，角色是 `measure`，描述明确写「整单成交金额（已含件数），不是单价」。这两个例子会在 S2 里再次出现，用来解释「为什么一个进了 ES，一个没进」。

`enum` 与 `range` 是值域的两种形状：前者是有限个可枚举的值，适合精确匹配；后者是一段连续区间，适合比较大小。这个区分会在 S2 决定要不要把值写进 ES 时再次出现，也会在 S4 决定某个取值是否合法时用到。

2.4 动作

```
make stage1
```

它内部执行三步：`python -m app.schema_contract --check` 做契约、负向夹具、文档新鲜度三合一；`--expect-legacy-red` 单独断言旧夹具按预期变红；最后跑 `tests/test_s0_contract.py` 与 `tests/test_s1_schema_contract.py`。如果要重新生成人读版契约，用：

```
cd backend && uv run python -m app.schema_contract --write-doc
```

代码路径集中在 `backend/app/schema_contract.py`：

- `load_contract()` / `load_ddl()`：分别读语义契约与物理 DDL。
- `check_contract()`：把九组检查串起来，返回空列表才算绿。
- `_check_table_structure()` / `_check_grain_and_measures()`：检查列集合、主键、外键、粒度与度量单位。

- `_check_value_domains()`：检查枚举、范围、ID 正则、开放值域的允许名单。
- `_check_metrics()`：检查公式、聚合方式、期望值，防止「把 GMV 公式改成 SUM(件数) 也能自洽」。
- `check_legacy()`：对旧系统夹具跑同一套检查，要求它红。
- `main()`：`--check` / `--check-legacy` / `--expect-legacy-red` / `--write-doc` / `--check-doc` 五个入口。

契约里的公式是能跑的表达式：`check_contract()` 把 `SUM(fact_order.order_amount)` 与真实 SQLite 聚合值对账，GMV 必须是 279159.5，销量必须是 622。口径写成这样，改错了就会红。

2.5 输出

S1 的输出有三层：**检查结论**（PASS / FAIL 与问题列表）、**人读版契约**（`docs/SCHEMA-CONTRACT.md`）、**可被下游加载的语义对象**（S2/S3/S4 的 `Catalog` 与上下文都从这份 YAML 派生）。查询时 S1 的命令不在场，但它的产物在场：没有这份契约，S2 不知道哪些列该同步值域，S4 不知道哪些表列可以出现在 SQL 里。

这张图回答：每张表一行代表什么？主键是什么？事实表如何关联维度？哪些列是真实存在、哪些是常见误问（city、price）？

读图：1 先看实体与 grain（一行代表什么）→ 2 再看主键/外键与关系 → 3 最后看约束、值域与口径

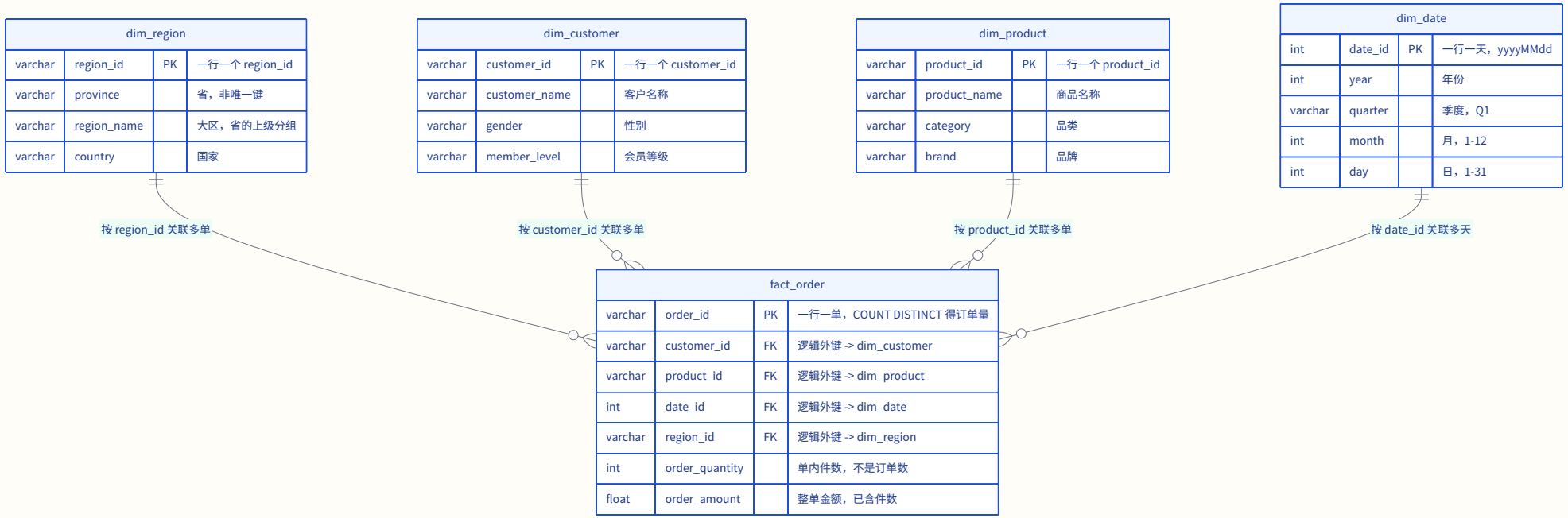


图 14 · 数仓 ER 图

图 14 给出 5 张表的连接关系；图 15 则把「表 / 列 / 指标 / 值域」抽成契约模型。两张图要一起看：ER 图回答「数据怎么连」，契约模型回答「每一条边和每一个列代表什么」。

这张图回答：契约里到底有哪些对象？每个对象带哪些字段？ValueDomain 的 kind 有哪几种？ColumnMetric 为什么是派生表？
读图：1 先看实体与 grain（一行代表什么）→ 2 再看主键/外键与关系 → 3 最后看约束、值域与口径

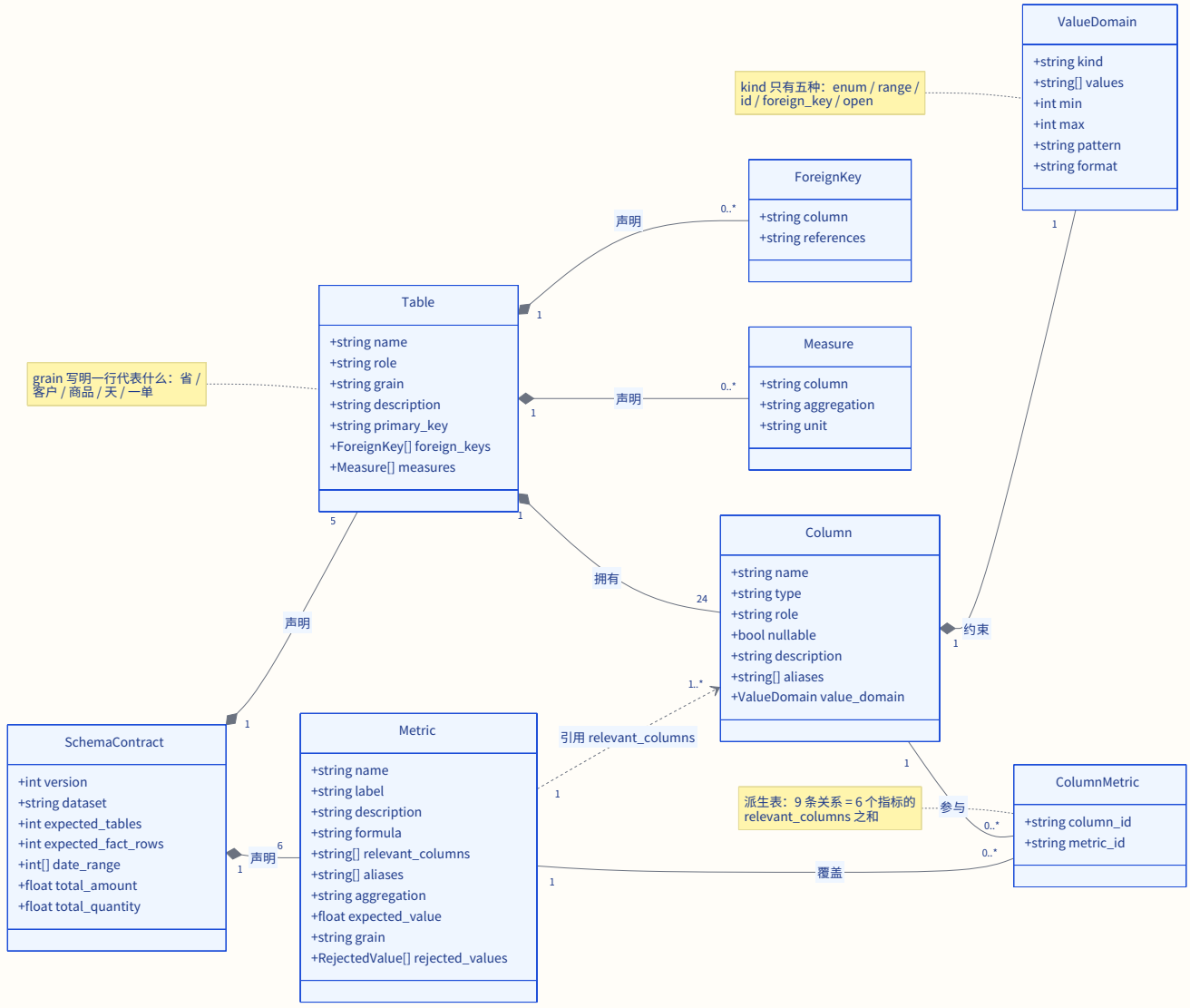


图 15 · 契约模型

2.6 验收

```
make stage1
```

期望输出：

```
PASS: 5 张表 / 24 列 / 115 行 / GMV=279159.5 / 销量=622
PASS: 旧系统 14 条问题 / 8 个代码（预期红）
PASS: docs/SCHEMA-CONTRACT.md 与契约一致
S1 PASS
```

前三行分别代表三种「绿」：契约与真实数据对上了；负向夹具按预期红了（检查器没有退化）；人读文档没有过期。任何一行红都意味着不同的修法：第一行红先查契约与 `dw.sql` 的差异；第二行如果打印「GREEN」，说明检查器漏检了旧系统的缺陷，比第一行红更危险；第三行红只需 `--write-doc` 重新生成并 review diff，但如果 diff 里出现语义变化，要回到契约本身确认。

第二行 PASS 背后的明细，是一份逐条列出的清单，每条都带代码、路径与原因。节选：

```
RED: 旧系统夹具检出 14 条问题
  [AOV_COLUMN_MISMATCH] legacy.metric.AOV: 描述是金额平均，relevant_columns 却是
['fact_order.order_quantity']
  [AOV_THREE_WAY] legacy.metric.AOV: 描述(金额平均) / 依赖列(件数) / 别名(平均单价) 三方互相矛盾；同一词可算出
2427.47、448.81、5.41 三种结果
```

第一行是总数；后两行分别说明错在哪条契约的哪个字段，以及同一词为什么会算出三种结果。

本设计工作树在 2026-10-29 实跑了 `--check`，三行全部通过；S0 + S1 的后端测试合计 59 条通过。前端不参与 S1。

补充：为什么契约检查用内存 SQLite

契约检查要在没有 Docker 的机器上跑，还要能反复跑而不留下状态。`fixtures/dw.sql` 本来就是建表加插入的脚本，装进内存 SQLite 之后，检查器可以在同一个进程里建库、灌数、聚合、丢弃。代价是 SQLite 与 MySQL 在类型与函数上有差异，所以这一步只验契约与数据是否一致，不验 MySQL 上的执行计划。真正的 MySQL 行为要到 S5 的执行器才出现。

2.7 常见故障

故障	先查什么	怎么修
表 / 列数不对	`fixtures/dw.sql` 的 DDL 与契约的 `tables` 是否同步	改契约或改 DDL，二者只能有一个事实源；不要在两处各改一半
GMV 或销量对不上	契约公式、`dw.sql` 数据、SQLite 聚合是否被改	用 `--check` 的问题列表定位；不要用改 `expected_value` 的方式「修绿」
旧夹具显示 GREEN	`EXPECTED_LEGACY_CODES` 是否被删、`check_legacy()` 是否被短路	恢复 8 个代码；负向夹具变绿是检查器退化，不是好消息
文档过期	`docs/SCHEMA-CONTRACT.md` 是否手改过	`--write-doc` 重新生成，再 review；生成物不要手改
	新列是否只写了类型、没写 kind	

故障	先查什么	怎么修
`value_domain` 缺失		主键 / 外键 / 枚举 / 范围 / 开放文本必须显式声明；开放值域只允许 `customer_name` / `product_name`
别名冲突	列别名与指标别名是否共用了一套空间	改别名，不要让「销售额」同时指向列和指标；`find_alias_collisions()` 会报
指标公式被改坏	`EXPECTED_METRIC_FORMULAS` 与契约是否一致	公式是口径，不是实现细节；改公式必须同时改期望值与文档

2.8 证据

结论	状态	证据
契约是 5 表 / 24 列 / 6 指标 / 9 条列—指标关系	✓	`contracts/schema.contract.yaml`；`--dry-run` 的 `column_metrics=9`
DDL 与真实数据是 5 表 / 24 列 / 115 行，GMV=279159.5、销量=622	✓	2026-10-29 本工作树 `--check` 输出
旧系统夹具检出 14 条问题 / 8 个代码	✓	同上；`EXPECTED_LEGACY_CODES` 逐项列出
文档新鲜度	✓	`--check` 第三段 PASS
S0 + S1 后端 59 条测试	✓	2026-10-29 本工作树 `pytest -q tests/test_s0_contract.py tests/test_s1_schema_contract.py`
旧系统三种 AOV 的历史来源	!	`fixtures/legacy/meta_config.yaml` 与 `schema_contract.py` 文件头注释；旧系统源码不在本仓库

2.9 检查清单

- ☐ 能说出 `dim_region` 的粒度是「一行一个 `region_id`」，`region_name` 不是唯一键。
- ☐ 能说出 `order_quantity` 是件数、`order_amount` 是整单金额，二者都不是订单数。
- ☐ 6 个指标的公式、聚合方式、期望值都有契约依据，不是口口相传。
- ☐ 新增列时同时写 `role`、`value_domain`、`aliases`，并且 `--check` 绿。
- ☐ 旧夹具仍然是红的，且 8 个代码一个不少。
- ☐ 改完契约后重新生成 `docs/SCHEMA-CONTRACT.md`，并 review diff。

3. S2 · 构建元知识

S2 要消灭的是「可检索性不确定性」：契约是给人读的，检索需要的是机器能查的结构。Qdrant 的 payload（随点存储的元数据）也能存

`grain`，ES 也能从字段里抽出「华北」。三种存储的分工来自失败面隔离：每类查询都有更直接的实现形态，一类存储坏掉时，另外两类仍能独立重建（查询链路是否完整是另一回事）。只用 ES 也能做 kNN（按向量找最近邻），但取值文档与语义候选的失败会混在一个引擎里；只用 Qdrant 也能用 payload 过滤，但精确对账与跨表事实查询不是它的强项。代价是运维复杂度，收益是每类查询有独立的失败面与重建路径。S2 把同一份契约翻译成三种存储：Meta 存事实、ES 存取值、Qdrant 存语义文本。翻译完之后还必须回答一个问题：三份东西真的等价吗？这就是 manifest、`--verify` 和 `--assert-idempotent` 存在的原因。

3.1 问题

如果只把契约文件丢给生成器，SQL 里出现「华北」时，系统没有任何东西能告诉它这是 `dim_region.region_name` 的一个合法取值；出现「销售额」时，也没有东西能告诉它这对应 GMV 的公式。S2 的目标是把「表是什么、列是什么、值有哪些、指标怎么算」变成查询时可用的三种视图，建索引只是结果。三种视图各有更顺手的查询，所以要同时存在：Meta 给你精确事实，ES 给你取值命中，Qdrant 给你文本相近的候选。

换个说法：契约回答世界上有什么，三种存储回答查询时怎么最快找到它。Meta 是档案柜，一条记录一条事实；ES 是字典，按词查到取值属于哪一列；Qdrant 是推荐员，按意思把相近的列和指标排个序。三个类比都只覆盖一个侧面：档案柜不排序，字典不懂同义，推荐员不保证精确。三种视图同时存在，是因为没有一种能在同等成本下把三类查询都做得同样直接。「按意思」是设计目标，不是当前实现：默认向量是字符片段，真正按语义理解检索要换 embedding（向量模型），这一点 §4.6 会说明。

3.2 实体与边界

S2 的实体是 `BuildPlan` 里的六类对象：`TablePlan`（表）、`ColumnPlan`（列）、`MetricPlan`（指标）、`ValueDoc`（取值文档）、`VectorPoint`（向量点），以及构建完成后的一份 `manifest`（这次构建的凭据）。每类对象都有确定性 ID：表用表名，列用 `table.column`，取值用 `table.column.value`，向量点用 `uuid5(kind:object_id:text)`。确定性 ID 是幂等的基础——同一对象、同一文本，第二次构建必须得到同一个 ID。

把 `uuid5(kind:object_id:text)` 读成「把这段文本哈希成一个固定 ID」：同样的 kind、对象 ID 与文本，永远得到同一个 ID，所以重复构建不会产生第二条。代价是文本改一个字，ID 就变，旧点可能留成孤儿；`--fresh` 重建会清掉。增量构建若只增不减，计数会变，`--`

`verify` 能发现；若增删成对、计数不变，孤儿点它检不到。幂等在这里的意思是：同样的输入重复执行，结果不变，而且不产生第二份写入。

S2 最容易记错的边界是：它是一次性 CLI，不是常驻进程。`python -m app.build_meta_knowledge --fresh` 跑完就退出，查询时它不在场；查询时在场的是它写下的 Meta 表、ES 索引、Qdrant collection。manifest 也不是查询依据，它只在 `--verify`、`--assert-idempotent` 和运维排障时被读。另一个边界是隔离：所有目标都带 `_rebuild` 后缀，`--fresh` 只删 rebuild 自己的库 / 索引 / collection，不碰旧系统。

把一次性再强调一次，因为它是排障时最容易被忘掉的前提：查询时的 `/api/query` 不会调用 S2 的任何代码，S2 也不监听端口。如果查询拿到的还是旧知识，先查 manifest 与构建时间，而不是去翻 FastAPI 的日志；S2 的日志只在构建那一次存在。manifest 是这次构建的身份证：它记录配置哈希、目标、计数与状态，`--verify` 拿它和三个存储对账。

3.3 输入

输入	内容	证据
`contracts/schema.contract.yaml`	语义契约：表、列、指标、值域	`build/pipeline.py:load_plan()`
`fixtures/dw.sql`	物理类型与真实值，装进内存 SQLite	`schema_contract.py:load_ddl()`
环境变量	`REBUILD_*` 目标名、凭据、`REBUILD_VECTOR_SIZE=256`	`backend/app/build/config.py`
Embedding	默认 `hash-ngram-v1`，256 维，不依赖模型下载	`build/embedding.py`

S2 不读旧系统的 `conf/meta_config.yaml`。旧配置只是 S1 的负向夹具；`fixtures/dw.sql` 也不需要手动导入 MySQL，S2 把它装进内存 SQLite 计算真实值、枚举、范围与样例。这个设计让 `--dry-run` 可以在没有三存储的机器上给出完整计数。

3.4 动作

```
make docker-env
docker compose up -d mysql elasticsearch qdrant
source scripts/export-docker-env.sh
make stage2
```

`make docker-env` 生成随机密码；`docker compose up -d` 只起三个存储，不要起 `backend` / `frontend`；`source scripts/export-docker-env.sh` 把 `.env` 里的值导出成宿主 CLI 能读到的 `REBUILD_*`。Compose 解析变量的顺序是「当前 shell 已 export 的值优先，再 `.env`」，所以不 `source` 时容器和 CLI 可能拿到不同密码，症状只有一句 `Access denied`。

`make stage2` 内部依次执行：

```
cd backend
uv run python -m app.build_meta_knowledge --dry-run
uv run python -m app.build_meta_knowledge --fresh
uv run python -m app.build_meta_knowledge --verify
uv run python -m app.build_meta_knowledge --assert-idempotent
uv run pytest -q
```

代码路径：

- `backend/app/build_meta_knowledge.py:main()`：CLI 入口，四个开关互斥。
- `backend/app/build/pipeline.py:build()`：先 `load_plan()`，再判断 noop / 修复 / `--fresh` 全量。
- `backend/app/build/plan.py:build_plan()`：纯内存生成计划，不碰网络。
- `backend/app/build/plan.py:_should_sync()`：决定哪些列的值进 ES。
- `backend/app/build/plan.py:compute_config_hash()`：把契约、目标名、embedding、值摘要压成一个哈希。
- `backend/app/build/pipeline.py:verify()` / `assert_idempotent()`：对账与幂等断言。

`--fresh` 的顺序是刻意的：先删 rebuild 的 Meta 库并建 schema，再删 ES 索引与两个 Qdrant collection，然后重建；Meta 是事实表，先写；ES / Qdrant 是可重建索引，后写；最后写 manifest。任何一步失败，都会尝试写一份 `status: failed` 的 manifest，避免「半套数据看起来像成功」。

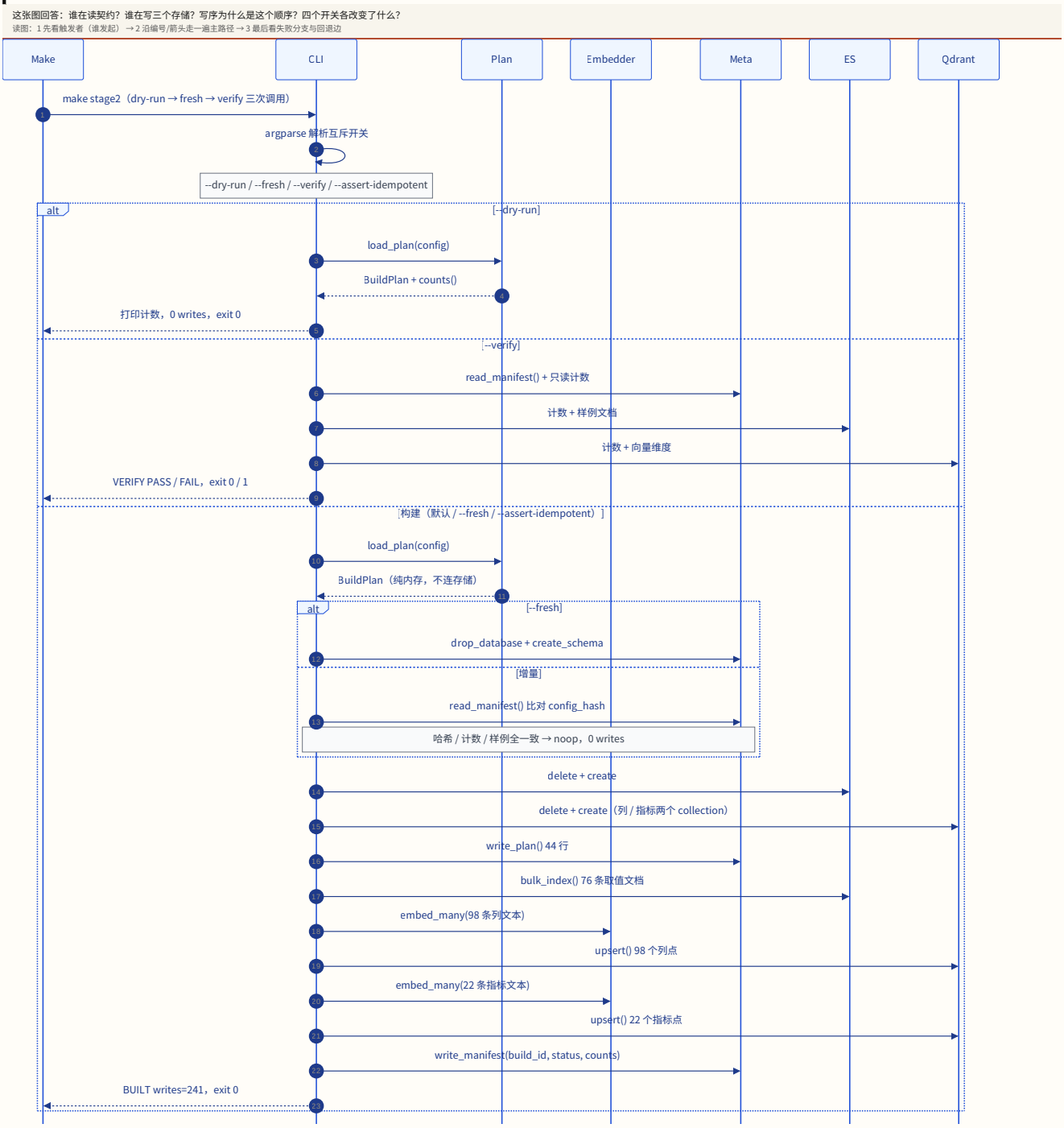


图 08 · S2 构建流水线时序

图 08 把这条顺序画成时序：谁读契约、谁写三个存储、manifest 什么时候落盘。看图时只需确认一件事：查询时这条时序已经不在了，留下的是三个存储。

读这张时序图时，建议盯住两个时刻：`--fresh` 删除旧目标的那一刻，以及 manifest 落盘的那一刻。前者决定重建是否从干净状态开始，后者决定这次构建有没有可对账的凭据。重建问题几乎都出在起点不干净，对账问题几乎都出在终点没落凭据；记住这两个时刻，比记住箭头顺序更有用。

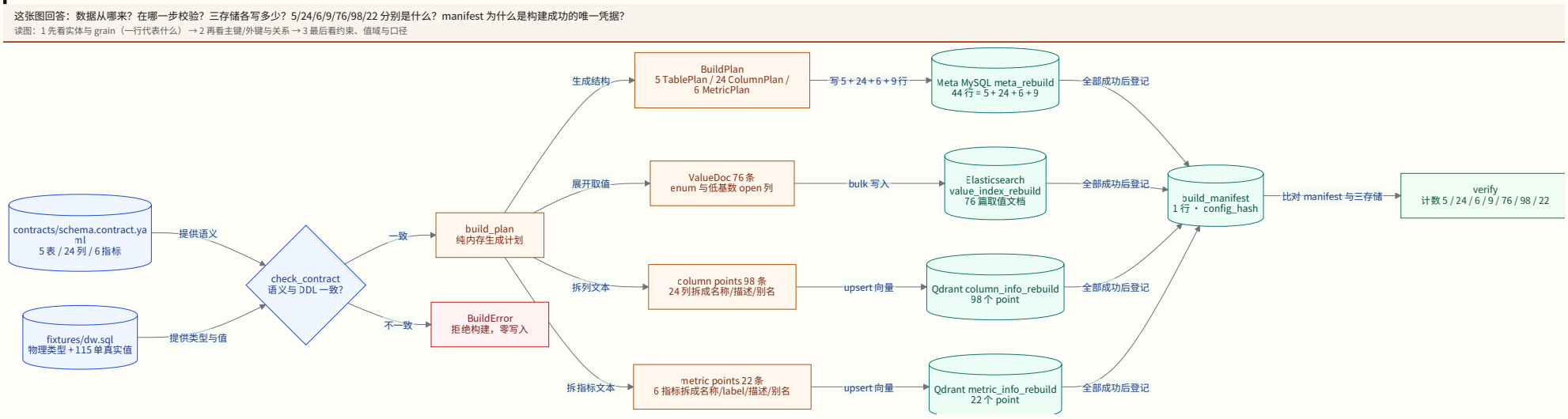


图 17 · S2 数据 lineage

3.5 输出

S2 的输出是三种存储 + 一份 manifest。计数必须背下来，因为它们会出现在后面每一次对账里：

目标	内容	计数	含义
Meta `table_info`	表	5	5 张表
Meta `column_info`	列	24	24 列
Meta `metric_info`	指标	6	6 个指标
Meta `column_metric`	列—指标关系	9	9 条
ES `value_index_rebuild`	enum 取值文档	76	76 条
Qdrant `column_info_rebuild`	列检索文本	98	24 列拆成 98 条

目标	内容	计数	含义
Qdrant `metric_info_rebuild`	指标检索文本	22	6 指标拆成 22 条
manifest	构建凭据	1	构建 ID、哈希、计数、状态

Meta 的 5 + 24 + 6 + 9 = 44 行；加上 ES 76、Qdrant 98 + 22、manifest 1，写入总数是 44 + 76 + 98 + 22 + 1 = **241**。这五个分量单位不同：44 是 Meta 行数，76 是 ES 文档数，98 与 22 是 Qdrant 点数，1 是 manifest 份数。241 只适合当构建日志的总数；对账时要回到 (44, 76, 98, 22, 1) 这组分量，不要用一个总数掩盖某一层的偏差。这个 241 是按 `build()` 的公式推导出来的，不是 `--fresh` 实测（本机 Docker socket 无权限），所以在书里标 ⓘ。`--dry-run` 能直接验证 5 / 24 / 6 / 9 / 76 / 98 / 22 这七个数，2026-10-29 本工作树已实跑，标 ✓。

两个真实例子能把「为什么是 98 和 76」讲清楚。

先排除一个容易产生的误会：98 不等于 24 × 5。每列的检索文本是「列名 + 描述 + 别名」去重后的条数：24 列里有 18 列各带两个别名、4 列各带三个、2 列各带一个，加上列名与描述各一条，合计 98 条。76 也不是 24 列的平均值：只有 `_should_sync()` 判定为维度、值域是 enum 或低基数开放列、且基数不超过 1000 的列，才把每个值写成一条 ES 文档；`order_amount` 这类度量列贡献 0 条。两个例子给的是单列的形状，总数要按这两条规则逐列累加。76 的逐列来源是：客户名 20、性别 2、会员等级 4、季度 1、年份 1、品牌 15、品类 6、商品名 15、国家 1、省 6、大区 5，相加正好 76。

例子一：`dim_region.region_name`。 它是维度列，`value_domain.kind = enum`，真实数据里有 5 个值：华东、华中、华北、华南、西南。`_should_sync()` 看到「维度 + enum + 基数 5 ⩽ 1000」，于是把 5 条 `ValueDoc` 写进 ES；同时该列的名、描述、三个别名（地区 / 区域 / 大区）去重后得到 5 条列文本，写进 Qdrant。所以你在 ES 里能精确命中「华北」，在 Qdrant 里能因「大区」这个别名召回同一列。Meta 里的 `examples` 是排序后的前 10 条，这一列只有 5 条，所以 examples 恰好等于全集；但这只是巧合，不是规律。

例子二：`fact_order.order_amount`。 它是度量列，`kind: range`，115 行事实里有 40 个不同金额。`_should_sync()` 第一道就把它挡掉：「只有维度列才同步值」。所以它在 ES 里有 **0 条**文档；但在 Qdrant 里仍有 5 条列文本（名、描述、三个别名），在 Meta 里 `examples` 只保留排序后的前 10 条（100.0、105.0、1198.0……），并显式带 `examples_truncated: true` / `examples_limit: 10`。这解释了两条纪律：不要因为 examples 里没有某个金额就判断数据里没有；也不要把「销售额」当成列名去 ES 查，它是 GMV 的指标别名。

两个例子合起来是一条判定规则：先看列的角色，再看值域的形状。维度列且值可枚举，才值得把每个值写进 ES；度量列的值是连续数字，写进去既贵又没用。这条规则会在图 19 的判定树里被展开，也会在 S3 的召回里反过来用：ES 命中一个值，说明它是某个可枚举列的合法取值。

这张图回答：region_name 和 order_amount 分别变成什么？为什么一个进 ES、一个不进？Qdrant 的 98 个 point 是怎么来的？Meta 的 examples 能不能当全集？
读图：1 先看实体与 grain（一行代表什么）→ 2 再看主键/外键与关系 → 3 最后看约束、值域与口径

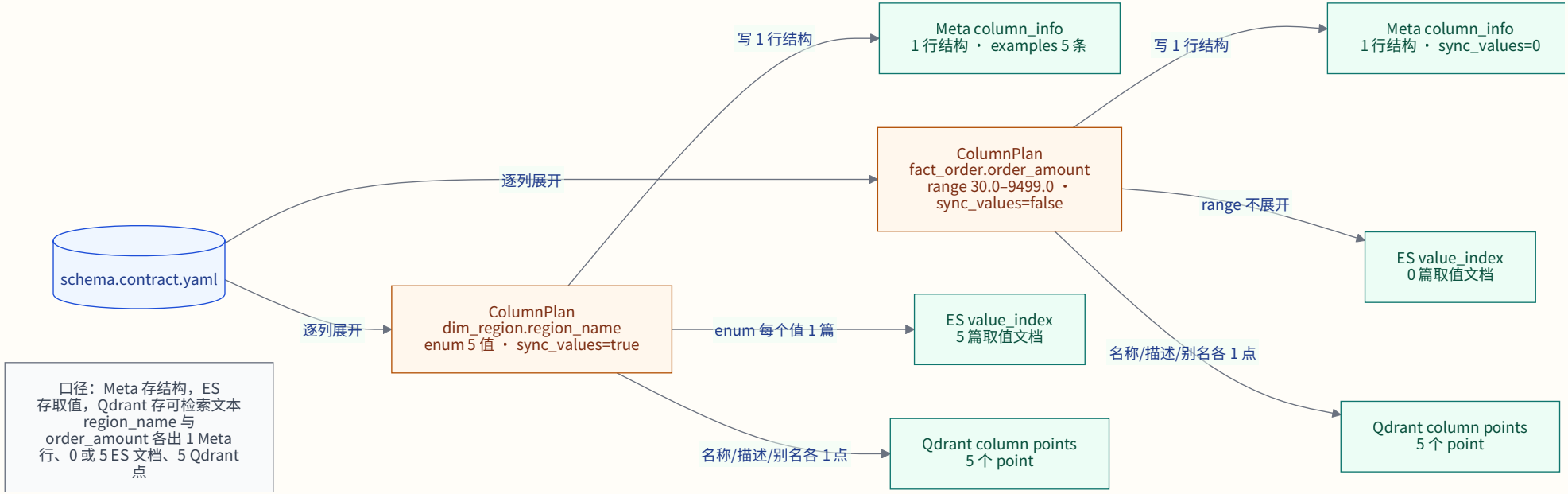


图 16 · 三存储知识布局

图 16 画出 Meta / ES / Qdrant 各自装什么；图 18 展开 Meta 的表结构与 manifest；图 19 则解释「一个值域该进哪种索引」的判定树。三张图合起来回答同一个问题：**同一份契约，为什么必须翻成三种形态?**

这张图回答：每张 Meta 表的字段和主键是什么？column_info 怎么指向表？column_metric 的复合主键是什么？manifest 记录哪些计数？
读图：1 先看实体与 grain（一行代表什么） → 2 再看主键/外键与关系 → 3 最后看约束、值域与口径

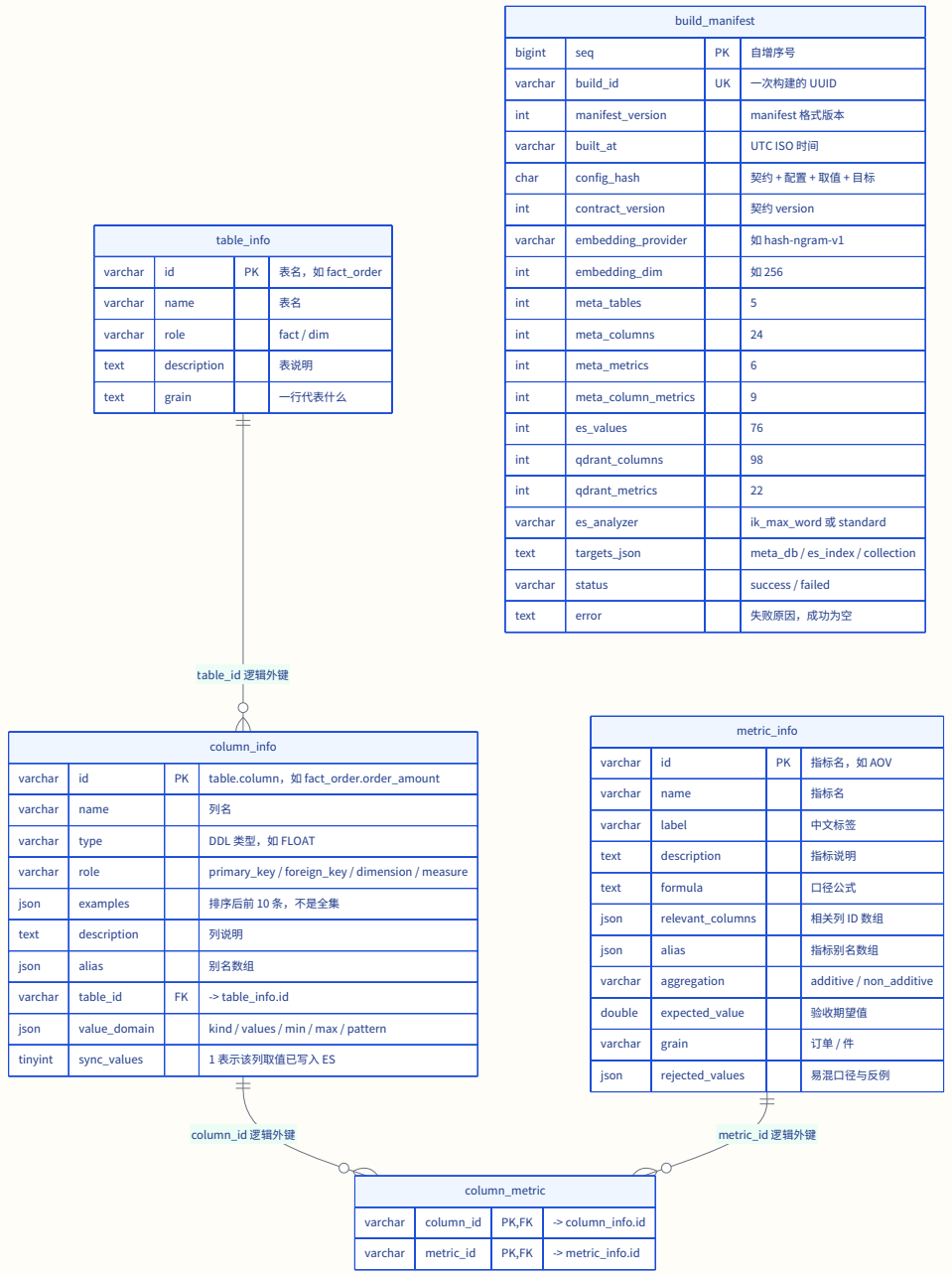


图 18 · Meta 表结构

这张图回答：enum 和 range 分别怎么处理？open 文本为什么有的进 ES 有的不进？sync_values 到底记什么？Met…
读图：1 先看实体与 grain（一行代表什么） → 2 再看主键/外键与关系 → 3 最后看约束、值域与口径

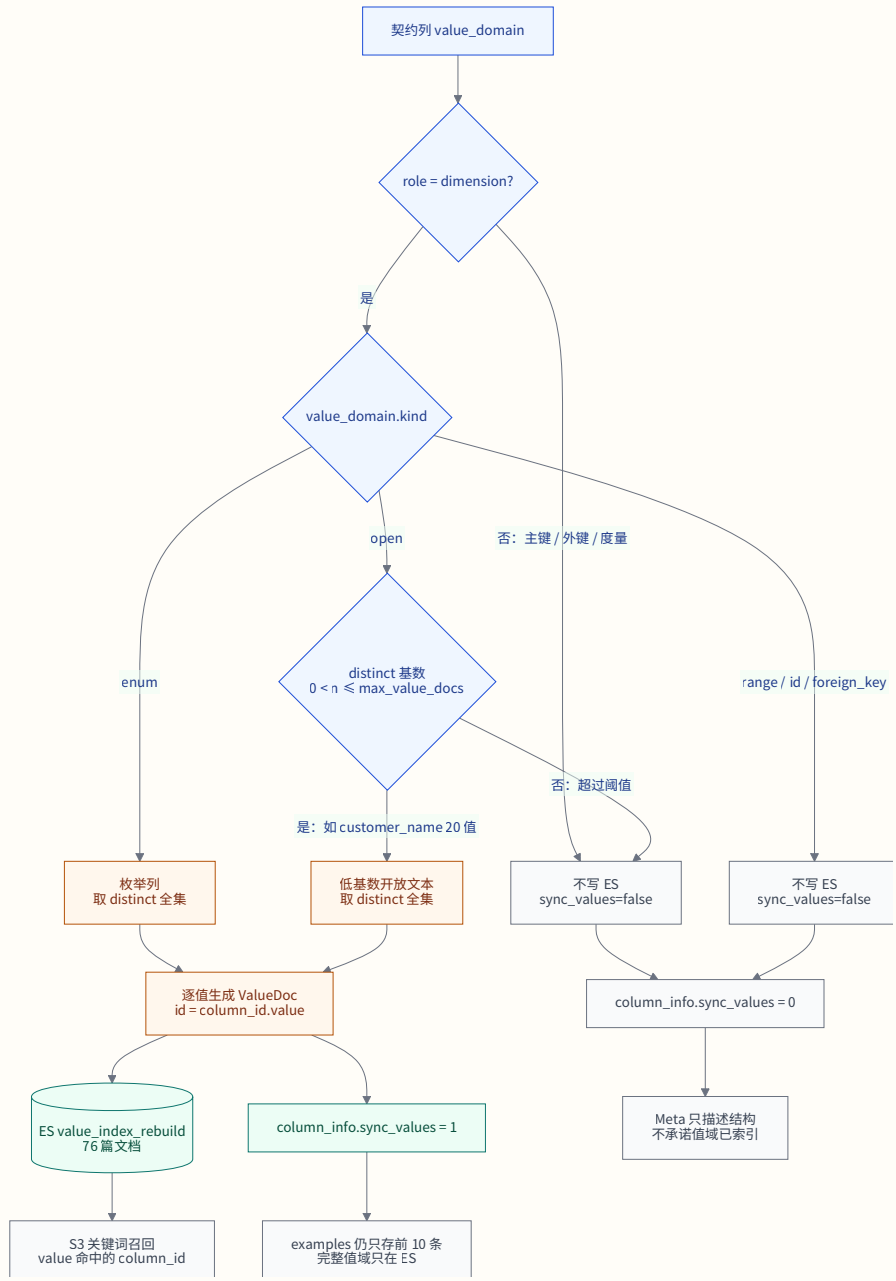


图 19 · 值域分类决策树

3.6 验收

```
make stage2
```

先看 `--dry-run`，它的输出不连任何存储：

```
source: contracts/schema.contract.yaml + fixtures/dw.sql (in-memory SQLite)
counts:
  Meta    tables=5 columns=24 metrics=6 column_metrics=9
```

```
  ES      values=76
  Qdrant  column_points=98 metric_points=22
DRY RUN: 0 writes (未连接 Meta/ES/Qdrant)
```

这段输出本工作树已实跑 ✓。它证明「计划」是对的，不证明「存储」是对的——后者要 `--fresh` 与 `--verify`。期望的关键输出还包括：

```
BUILT: writes=241 ...
S2 VERIFY PASS
NOOP: 0 writes ...
S2 IDEMPOTENT PASS
S2 PASS
```

读法：`BUILT` 说明写了 241 条；`VERIFY PASS` 说明 manifest、三存储计数、维度、analyzer、样例内容全部对得上；`NOOP: 0 writes` 说明第二次构建没有写任何东西；`S2 PASS` 是 Makefile 的总结。这里有一个常见误解：`--fresh` 那一步是**重建**；同一条 `make stage2` 里的 `--assert-idempotent` 那一步才是零写入断言。

前面问过「三份东西真的等价吗」，这里的「等价」指可观察的一致性：manifest、三存储计数、维度、analyzer 与样例内容对得上。它不覆盖语义等价：`--verify` 检不到样例之外的 payload 漂移，也检不到增删成对、计数不变的孤儿点，还检不到同名 analyzer 升级后的分词差异。要往语义等价靠近，得到 S3 的黄金集上比较召回集合，而不只是对计数。

失败含义：如果 `--dry-run` 就红，问题在契约 / DDL / 计划，和 Docker 无关；如果 `--fresh` 红，先查凭据与目标是否可达；如果 `--verify` 红，看它打印的 `key: manifest=... 实际=...`，那是三存储计数或维度不一致；如果 `--assert-idempotent` 红，说明第二次构建没有 noop，通常是 manifest 哈希变了或样例内容被外部改坏。

3.7 常见故障

故障	先查什么	怎么修
`Access denied`	当前 shell 是否 `source scripts/export-docker-env.sh`；`.env` 是否被换过	重新 `source`；改了 `.env` 后要重新导出，必要时 `docker compose up -d`
`--dry-run` 计数不对	契约与 `dw.sql` 是否同时改了；`REBUILD_MAX_VALUE_DOCS` 是否被改	以 dry-run 的计划为准修契约或数据；不要手改期望计数
`--verify` 报计数不一致	是否跑过 `--fresh`；是否有人删了索引 / collection	重跑 `--fresh`；不要用「补一条文档」掩盖差异

故障	先查什么	怎么修
第二次构建不是 noop	manifest 哈希、目标名、embedding 名 / 维度、值摘要是否变了	用 `--verify` 定位；配置变更本来就该触发重建，不是 bug
ES analyzer 不一致	是否装了 IK、是否回退 `standard`	当前镜像不含 IK，S2 会自动回退 `standard`；不要手改 mapping
Qdrant 维度不对	`REBUILD_VECTOR_SIZE` 与 collection 维度	默认 256；改维度必须 `--fresh` 重建，不能混写
目标指到旧系统	`REBUILD_META_DB` / `REBUILD_ES_INDEX` / collection 名	一律用 `*_rebuild`；不要为了「借用数据」改目标
Docker socket `permission denied`	当前账号是否在 `docker` 组	用有权限的账号，或 `sudo usermod -aG docker "$USER"` 后重新登录；这是系统权限变更，按环境决定

3.8 证据

结论	状态	证据
计划计数 5 / 24 / 6 / 9 / 76 / 98 / 22	✓	2026-10-29 本工作树 `--dry-run`
`dim_region.region_name` 进 ES 5 条、Qdrant 5 条	✓	本工作树用 `build_plan()` 读取计划实查
`fact_order.order_amount` 不进 ES、Qdrant 5 条、examples 截断为 10	✓	同上；`plan.py:_should_sync()` 与 `EXAMPLES_LIMIT`
写入总数 241	!	`build/pipeline.py:build()` 公式推导：44 + 76 + 98 + 22 + 1
`--fresh` 实际写入与 `--verify`	?	本机 Docker socket 无权限，未执行
第二次构建 `NOOP: 0 writes`	?	依赖 `--fresh`；历史文档记录过，本轮未复现
S2 测试 17 条	!	`docs/SETUP-FROM-SCRATCH.md` §3 的阶段性计数；本轮未跑全量

依赖 Docker 的存储层写入与 `--verify` 仍未验证，见对应行；计划层实查与测试记录的区别见表内证据列。

3.9 检查清单

- ☐ 能一句话说清：S2 是一次性 CLI，查询时不在场，留下的是三个存储和一份 manifest。
- ☐ 能背出 5 / 24 / 6 / 9 / 76 / 98 / 22，并知道 241 是公式推导。
- ☐ 能解释 `region_name` 为什么进 ES，而 `order_amount` 为什么不进。
- ☐ 知道 `examples` 是截断值域，不是全集；完整值在 ES（仅同步列）。

- [ ] 知道 `make stage2` 是重建，`--assert-idempotent` 才是零写入断言。
- [ ] 所有目标都带 `_rebuild`，且 `--fresh` 不碰旧系统。

4. S3 · 检索

S3 要解决的是「召回不可回归」：它不证明召回质量已经好，只把召回变成一套可重复的验收。S2 把知识放进了三个存储，但「放进去」不等于「找得到」。用户问「华北销售额」，系统必须同时找到 `dim_region.region_name = 华北`、GMV 指标、`fact_order.order_amount` 与连接所需的 `region_id`；少一个，S4 就写不出正确的 SQL；多一堆噪声，S4 的上下文就会超长、跑偏。S3 用 20 题黄金集把这件事变成可重复的验收，而不是靠「试几个问题感觉还行」。

4.1 问题

检索层最容易犯两种错：召回不足与召回过量。召回不足时，生成器只能拒答或猜；召回过量时，模型被无关表列淹没，反而更容易写错。S3 的设计是「批量召回 + 确定性裁剪」：先用 Qdrant 找列和指标、用 ES 找取值、用 Meta 补事实，再用主外键、指标依赖、直接证据做确定性过滤。整个过程不调用 LLM，所以同一个问题在同一份索引上得到同一份 YAML（除了运行日期）。

先说边界：S3 证明的是召回流水线成立，不是召回质量；它在生产主链路里的接入状态，§4.2 会说明。

4.2 实体与边界

S3 的实体是 **Keyword**（关键词与提示）、**Candidate**（候选列 / 指标，带向量分与词法分）、**ValueHit**（ES 命中的取值）、**ColumnContext / TableContext / MetricContext**（合并后的事实）、**RecallState**（整次检索的状态与 trace）。trace 记录 ES / Qdrant / Meta 各往返几次、候选多少、最终保留多少。它是调试证据，不进契约。

三个存储的分工必须说清：

- **ES** 负责「值」：`value_index_rebuild` 里是 enum / 低基数开放列的取值文档。它回答「华北是不是一个真实取值，属于哪一列」。
- **Qdrant** 负责「语义文本」：`column_info_rebuild` 与 `metric_info_rebuild` 里是列 / 指标的名、描述、别名。它回答「哪些列和指标跟这个问题的文本最接近」。

- **Meta** 负责「事实」：`table_info` / `column_info` / `metric_info` / `column_metric`。它回答「这个候选的粒度、类型、公式、期望值、依赖列是什么」。

S3 的边界：只读、不写任何存储、不生成 SQL、不调用 LLM。它输出的是「候选上下文」，不是答案。另一个关键边界是：在当前生产主链路里，`/api/query` 默认走 `OfflineContextBuilder`（契约离线裁剪），S3 的真实 ES/Qdrant 召回只在 `make stage3` 与集成测试中使用，尚未接入线上问数。这一点在 README「数据与存储」里写得很明确，不能用「S3 通过」推断「前端已经能真实召回」。

4.3 输入

输入	内容	证据
用户问题	单轮自然语言，例如「华北销售额」	`recall/pipeline.py:recall()`
S2 三存储	Meta 快照、ES 索引、两个 Qdrant collection	`recall/clients.py:RecallStack`
契约 Catalog	表列顺序、主外键、指标依赖、值域	`recall/catalog.py:default_catalog()`
黄金集	20 题，含 `expected_*` 与 `must_not`	`backend/tests/recall/golden.yaml`

黄金集不是「20 个随便的问题」。每题都写清必须出现的表、列、指标、取值，以及**必须不出现**的噪声（`must_not`）。例如 `q01`「华北销售额」要求出现 `dim_region`、`fact_order`、`region_name`、`region_id`、`order_amount`、GMV、`华北`，同时禁止 `AOV`、`销量`、`province`、`city`。这样测试既防召回不足，也防「把所有表列都塞进去」的作弊式全召回。

例题：`must_not` 为什么比 `expected_*` 更难作弊

题目：黄金集里，`expected_*` 列出必须出现的表、列、指标与取值，`must_not` 列出必须不出现的噪声。只保留前者，测试会不会更容易通过？如果会，为什么还要保留后者？

解答：会，而且容易得多。只检查 `expected_*` 时，一个把所有表、所有列、所有指标都塞进上下文的实现可以拿满分：反正要的都在里面。这种实现回到线上，会把几十个无关列一起交给生成器，上下文变长、噪声变多，SQL 更容易跑偏。`expected_*` 检查召回够不够，`must_not` 检查召回准不准，两个方向缺一不可。

以 `q01`「华北销售额」为例：期望出现 `dim_region`、`fact_order`、`region_name`、`region_id`、`order_amount`、GMV 与 `华北`；同时禁止 `AOV`、`销量`、`province`、`city`。如果实现把 `province` 也召

回，`expected_*` 不会红，`must_not` 会。`city` 在契约里不存在，把它列进 `must_not`，等于同时守住不要编造粒度这条线。

结论：`expected_*` 防召回不足，`must_not` 防召回过量；前者让系统能答，后者让系统不乱答。只做前者，测试会奖励全召回这种最省事也最没用的实现。

4.4 动作

```
source scripts/export-docker-env.sh
make stage3
```

`make stage3` 的实际步骤是：`--verify` 对账 S2；`REBUILD_INTEGRATION=1 pytest -q tests/recall` 跑 20 题黄金集与集成测试；`sh scripts/recall-demo.sh /tmp/rebuild-s3-recall.yaml` 生成演示 YAML；`head -n 40` 看前 40 行；最后跑全量后端测试。注意它**不执行** `stage2` 或 `--fresh`，S3 是只读验收，不能顺手重建索引。

手动看单题：

```
cd backend
uv run python -m app.scripts.recall "华北销售额"
```

stdout 是 LLM 将看到的 YAML，stderr 是检索统计。代码路径：

- `backend/app/recall/pipeline.py:recall()`：固定顺序串起 8 个节点。
- `backend/app/recall/nodes.py:extract_keywords()`：抽关键词与值 / 日期提示。
- `recall_column()` / `recall_value()` / `recall_metric()`：三个批量召回节点。
- `merge_retrieved_info()`：一次 Meta 快照，补齐事实与主外键。
- `filter_table()` / `filter_metric()`：确定性裁剪噪声。
- `add_extra_context()`：注入日期、星期、季度、数据库版本。
- `backend/app/recall/clients.py`：ES `_msearch`、Qdrant `batch_search`、Meta 批量读。

批量是硬约束，不是优化建议：20 题里每题 ES `_msearch` 不超过 2 次、Qdrant 列与指标各 1 次 batch（合计 2 次）、Meta 恰好 5 条

SQL。测试 `test_golden_roundtrips_are_batched()` 逐题断言这些上界，防止有人把循环查询写回来。

`_msearch` 是 ES 的批量搜索接口：一次请求里放多条查询，返回多条结果；`batch_search` 是 Qdrant 的批量检索，一次带上多个向量。两者解决同一个问题：别把每列查一次写成循环。往返次数是延迟的大头，批量属于能不能在可接受时间内完成一次召回的前提，不是微优化。

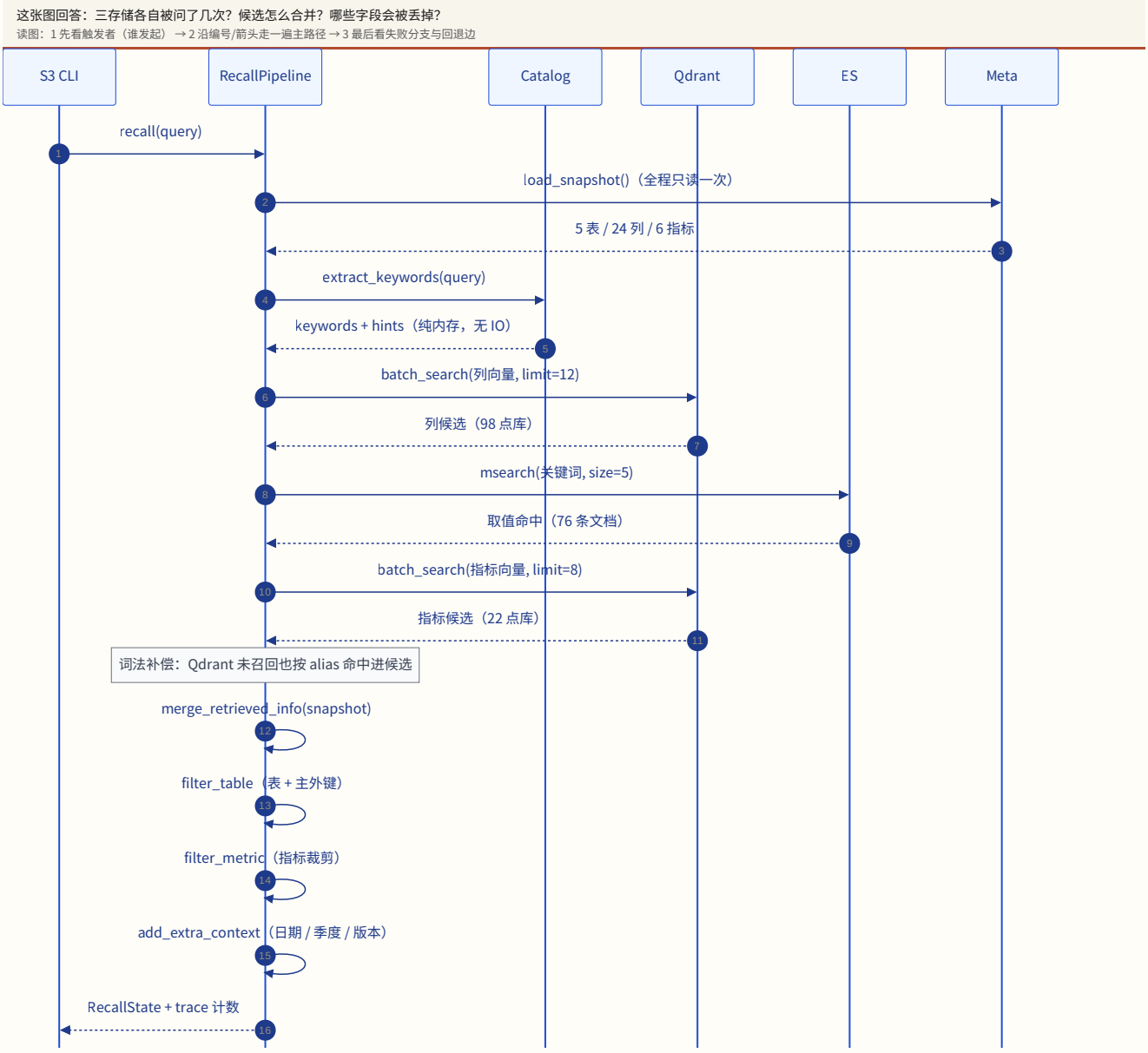


图 09 · S3 召回流水线时序

图 09 的读法是「一次 HTTP 里带多个关键词」：ES 用 `_msearch` 一次问多词，Qdrant 用 `batch_search` 一次查多向量，Meta 用一次连接读四张表。时序图上的每一次往返都能在 trace 里对上数字。

4.5 输出

S3 的输出是一份稳定 YAML + 一份 stderr 统计。YAML 的顶层包括 `query`、`contract_version`、`dataset`、`keywords`、`extra_context`、`tables`、`metrics`、`values`、`retrieval`、`examples_policy`。每个入选列都保留契约字段：`id`、`name`、`type`、`role`、`description`、`aliases`、`value_domain`、`examples`、`examples_truncated`、`examples_limit`、`matched_values`。指标保留 `formula`、`expected_value`、`relevant_columns`、`aliases`、`rejected_values`。

这些字段都有下游用途。S4 生成 SQL 时要用 `formula` 决定聚合，用 `value_domain` 判断取值是否合法，用 `rejected_values` 避免把 448.81 当成 AOV。S3 的过滤层可以裁掉不需要的 join 边，但**契约关键字段不会被裁掉**；`test_golden_yaml_preserves_contract_fields()` 就是在守这条线。

示例统计（`docs/SETUP-FROM-SCRATCH.md` §2.6）：

```
recall stats: query='华北销售额' keywords=3 es_roundtrips=1 qdrant_roundtrips=2
(column=1 metric=1) meta_roundtrips=5 candidates[columns=12 values=1 metrics=4]
final[tables=2 columns=5 metrics=1 values=1]
```

读法：ES 1 次、Qdrant 2 次、Meta 5 条；候选 12 列 / 1 值 / 4 指标；最终保留 2 表 / 5 列 / 1 指标 / 1 值。候选到最终的数字变化，就是 `filter_table()` / `filter_metric()` 的工作量。

4.6 验收

```
make stage3
```

期望输出：

```
38 passed
recall stats: ... es_roundtrips=1 qdrant_roundtrips=2 ... meta_roundtrips=5 ...
90 passed, 23 skipped
S3 PASS
```

这几组数字要分开看：

命令	跑哪些	结果
`make stage3` 的集成那一步	`tests/recall` 全量	38 passed
全量后端（S3 阶段快照）	全部用例；23 条 S3 集成测试被 skip	90 passed / 23 skipped

23 条是 38 条里需要真实三存储的那部分：默认全量把它们 skip，`make stage3` 用 `REBUILD_INTEGRATION=1` 打开后，`tests/recall` 的 38 条全过。90 是 S3 阶段全量后端的快照；作为对照，S4 阶段 `tests/generate` 的快照是 108 passed / 5 skipped。两者来自不同阶段、不同命令，108 不能和 90 直接比大小。不要把 23 条 skip 当成失败，也不要把它们当成已通过。

失败含义：如果 `--verify` 红，先回到 S2，不要在 S3 修索引；如果黄金集某题红，看是 `expected_*` 少了（召回不足）还是 `must_not` 多了（噪声过量）；如果往返次数超上界，检查是否有人把批量查询改成了循环；如果 YAML 字段缺失，查 `merge_retrieved_info()` 是否在裁剪时丢了契约字段。

常见误解：黄金集全过，不代表召回质量好

黄金集是回归测试，不是质量评估。它固定了 20 个问题与期望，能防改坏了，但不能回答开放问法下召回率多高。当前向量是 `hash-ngram-v1`，256 维，靠字符片段相似而不是语义理解；换一种问法、换一批业务词，召回结果可能完全不同。把 20/20 当成召回已经解决，会在 S8 接真实问题时被现实纠正。这一章能证明的是流水线成立、批量上界成立、契约字段保真；召回质量属于后续的开放问题。

4.7 常见故障

故障	先查什么	怎么修
`--verify` 失败	S2 是否成功构建、目标是否同一组变量	重跑 S2；S3 与 S2 必须用同一组 `REBUILD_*`
20 题里某题召回为空	索引是否有数据、问题关键词是否被正确抽取	先手工查 ES / Qdrant；再看 `extract_keywords()` 的提示
`must_not` 里的列仍出现	`filter_table()` 的直接证据阈值、主外键补齐	调过滤规则前先确认黄金集期望是否合理；不要为了让测试绿而删 `must_not`
往返次数变多	是否改成了逐关键词循环；是否重复打开连接	恢复批量 `_msearch` / `batch_search`；Meta 用一次快照
Qdrant 维度错误	collection 维度与 `REBUILD_VECTOR_SIZE`	用 S2 `--fresh` 重建；不要混写不同维度
ES 搜不到中文	analyzer 是否回退 `standard`、分词是否符合预期	当前镜像不含 IK；先接受 `standard` 的边界，再评估替换镜像

故障	先查什么	怎么修
YAML 字段被裁掉	`_column_context_from_meta()` 与过滤层	契约字段必须保留；过滤只裁候选，不裁事实
日期上下文每天变	`extra_context.current_date` 来自运行日	这是预期行为；需要固定日期时在代码里注入 `now`，CLI 暂不暴露

`filter_table()` 的「直接证据」指列命中了一个取值、词法分过线、向量分过线，或它是指标依赖列；阈值常量在 `recall/nodes.py` 里。向量分没到线、又没有取值、词法或依赖证据的列，会在过滤层被裁掉。

4.8 证据

结论	状态	证据
黄金集是 20 题，含 expected 与 must_not	✓	`backend/tests/recall/golden.yaml`；`test_golden_has_twenty_unique_questions`
每题 ES ⩽ 2、Qdrant 列 1 + 指标 1、Meta 5 次	✓	`test_golden_roundtrips_are_batched()`
单题 stats 为 es=1 / qdrant=2 / meta=5	✓	`docs/SETUP-FROM-SCRATCH.md` §2.6 记录
20/20 通过、38 条集成测试	!	文档记录；本机无 Docker 权限，本轮未复现
全量 90 passed / 23 skipped	!	同上；23 条 skip 是真实存储集成测试
三存储分工	✓	`build/plan.py:_should_sync()`、`recall/nodes.py`、`recall/clients.py`
S3 不在生产主链路	✓	`README.md`「数据与存储」；`main.py:default_runner_factory()` 未注入 recall

依赖 Docker 的复现以表内证据列为准；20/20 与集成测试另标推断。

4.9 检查清单

- ☐ 能分别说出 ES、Qdrant、Meta 各回答什么问题，且知道不能互相替代。
- ☐ 能解释 20 题黄金集为什么同时有 `expected_*` 与 `must_not`。
- ☐ 知道一次检索的往返上界：ES ⩽ 2、Qdrant 2、Meta 5。
- ☐ 知道 trace 是调试信息，不是业务契约；稳定的是 YAML 与契约字段。
- ☐ 知道 S3 通过 ≠ 前端能真实问数；生产主链路仍走离线上下文。
- ☐ 能定位「召回不足」与「召回过量」分别该看哪一层。

5. S4 · 生成 SQL

S4 要消灭的是「产物不确定性」：上下文再完整，最后也必须变成一条 SQL，而 SQL 可能语法错、引用不存在的列、投影常量、排名却不排序，甚至根本不是只读语句。旧系统把「只读」寄托在一句提示词上，于是 UPDATE 能过 `EXPLAIN`、多语句第二句照常执行。S4 的做法是把生成结果限制成两种形状（`sql` 或 `refusal`），再用 AST 静态校验把语义硬规则变成代码；不合格的 SQL 不会被「修一修继续用」，而是降级成有违规码的拒答。

5.1 问题

生成层最危险的不是「生成不出来」，而是「生成出一个看起来对的错东西」。一条 `SELECT` 可能引用了契约里不存在的表，可能把 `order_quantity` 当订单数，可能在没有 `ORDER BY` 的情况下返回「Top 3」，也可能在 SELECT 列表里塞一个常量伪装成数据。S4 的答案是两道关：先校验**形状**（必须是 `sql_or_refusal`，不能有第三种），再校验**语义**（单语句、只读、表列存在、无常量投影、排名有 ORDER BY / LIMIT）。两道关都是运行时的，不依赖模型自觉。

5.2 实体与边界

S4 的实体：**GenerateContext**（离线裁剪或 S3 召回得到的上下文）、**Slots**（7 个提示词槽位）、**GenerationDraft**（还没过静态校验的草稿）、**GenerationResult**（最终的 SQL 或拒答）、**Refusal**（码 + 原因）、**GuardReport**（违规列表）。`GenerationDraft` 单独存在，是为了让 S6 的校正闭环能拿到「被拒的那条 SQL」；S4 本身不需要校正。

S4 的边界：它只到「生成 + 静态校验」，**不执行 SQL**。执行是 S5 的事；失败通道（错误码、类别、校正、取消）是 S6 的事。S4 的静态校验也不是安全策略：它检查表列是否在上下文里、是否只读，但不做白名单、超时、注入语料、只读账号。把 S4 当 S5 用，会漏掉「策略重新序列化」和「执行账号权限」这两层。

AST 这个词在这里值得拆开。SQL 先被解析成一棵树：根是语句类型，子节点是选择列表、来源表、过滤条件、排序与限量；树上的每个节点都有确定的类型。策略与静态校验都在这棵树上做判断，字符串上的正则匹配做不到这一点：注释、大小写、括号和换行都能绕过字符串匹配；解析器把这句话到底在干什么固定成一棵树，这些写法就绕不过去了。

5.3 输入

输入	内容	证据
上下文	`build_offline_context()` 或 `context_from_recall_state()`	`generate/context.py`
提示词	`contracts/prompts/generate_sql.prompt`，7 个槽位	prompt frontmatter
输出契约	`contracts/sql.output.schema.json`，只允许 `sql_or_refusal`	JSON Schema
基线	10 题：8 题来自黄金集，2 题越界	`app/scripts/generate.py --baseline`
生成器	默认确定性；可选 `HttpLlmGenerator`	`generate/provider.py`

7 个槽位是 `query`、`table_infos`、`column_infos`、`metric_infos`、`value_infos`、`date_info`、`db_info`。槽位声明在提示词 frontmatter 里，`build_slots()` 按同一顺序渲染；少一个槽就抛 `PromptSlotError`，绝不静默补空串。所以提示词在这里是一份有输入契约的程序，改槽位等于改接口。

5.4 动作

```
make stage4
```

它做三件事：跑 10 题基线、跑 `tests/generate`、跑全量后端测试。
单独看基线表格：

```
make baseline
```

单题：

```
cd backend
uv run python -m app.scripts.generate "华北销售额"
```

代码路径：

- `backend/app/generate/pipeline.py:draft_sql()`：构建上下文、渲染 7 槽、调用生成器、校验输出形状。
- `backend/app/generate/pipeline.py:validate_draft()`：静态校验，不通过降级成拒答。
- `backend/app/generate/guard.py:check_sql()`：五条硬规则。

- `backend/app/generate/provider.py:DeterministicGenerator` / `HttpLlmGenerator`：两种实现，同一输出 schema。
- `backend/app/generate/output_schema.py:validate_payload()`：形状校验。
- `backend/app/generate/intent.py`：排名意图、Top N、时间序列维度。
- `backend/app/generate/deterministic.py`：离线生成器本体。

确定性生成器 vs LLM，是 S4 最需要讲清的一组对比：

维度	确定性生成器	`HttpLlmGenerator`
默认	是，不需要任何配置	否，需三个环境变量齐全
可复现	同一上下文同一 SQL	受模型与采样影响（temperature=0 仍不保证）
网络	不需要	需要 OpenAI 兼容端点
输出形状	`sql_or_refusal`	同左，非 JSON 直接拒答
静态校验	必须过	必须过，**不放宽任何一条规则**
本机状态	✓ 已实跑	? 无端点 / 无 Key

`build_slots()` 产出的 7 个槽位是两条路共用的；`validate_draft()` 也是两条路共用的。所以「换成真模型」只换生成器，不换契约、不换校验、不换输出形状。这也是为什么 S4 可以在没有 LLM 的机器上验收：它验的是「提示词 + 输出契约 + 静态校验」这条骨架，而不是某一次模型表现。

接一个真实模型：三个变量，缺一不可。

`HttpLlmGenerator.from_env()` 只认这三个变量（`backend/app/generate/provider.py:180`）：

变量	含义	DeepSeek 示例
`REBUILD_LLM_BASE_URL`	OpenAI 兼容端点根地址	`https://api.deepseek.com`
`REBUILD_LLM_API_KEY`	API Key	从环境注入，**不写进仓库**
`REBUILD_LLM_MODEL`	模型名	`deepseek-flash`（或 `deepseek-chat`）

三个变量必须同时非空；缺任何一个，`from_env()` 直接抛错，主程序回退确定性生成器。最常见的坑是把 Key 写成 `DEEPSEEK_API_KEY:sk-...` 这种**冒号**格式——shell 不认，变量等于没设。正确写法是 `export DEEPSEEK_API_KEY=sk-...`，再映射到

`REBUILD_LLM_API_KEY`。验证只打印模型名与 Base URL，不打印 Key：

```
export REBUILD_LLM_BASE_URL=https://api.deepseek.com
export REBUILD_LLM_MODEL=deepseek-flash
export REBUILD_LLM_API_KEY="$DEEPSEEK_API_KEY"
cd backend && uv run python - <<'PY'
from app.generate.provider import HttpLlmGenerator as G
g = G.from_env()
print(g.model, g.base_url)
PY
```

接上真模型之后，校正才真正有意义：没有校正器时，`correct_sql` 这一步会显式 `skipped`，不会假装能改。

静态校验的五条规则，逐条对应一个真实故障：

1. **单语句**：解析后必须恰好一条，否则 `SQL_MULTIPLE_STATEMENTS`。
2. **只读**：根节点必须是 SELECT、UNION（含 UNION ALL），或整条被括号包起来的子查询；CTE 按主查询判定。任何 DML / DDL / INTO 都拒绝，码是 `SQL_NOT_READ_ONLY`；词法层还会兜底扫写关键字，防止 AST 解析不了的 `INTO OUTFILE` 被当成「语法错误」而不是「不是只读」。
3. **表列存在**：引用的表、列必须在契约上下文里，否则 `SQL_UNKNOWN_TABLE` / `SQL_UNKNOWN_COLUMN`。
4. **无常量投影**：SELECT 列表里不得出现字面量或裸 NULL，否则 `SQL_CONSTANT_PROJECTION`；聚合表达式里的 `CAST(... AS DECIMAL(10,2))` 属于类型声明，不算常量。
5. **排名有 ORDER BY / LIMIT**：问题含排名意图却没有排序，`SQL_MISSING_ORDER_BY`；限定了名次却没有 LIMIT，`SQL_MISSING_LIMIT`。

五条都是硬规则：违规就降级成拒答，`GenerationResult.outcome` 只会是 `sql` 或 `refusal`。下游永远拿不到「一条没过校验的 SQL」。

词法层不是简单的关键字子串匹配：它先遮蔽字符串字面量与反引号标识符，再剥注释，最后按词边界扫写关键字。所以 `WHERE product_name = 'DROP'` 不会被误拒，而 `INTO OUTFILE` 会被拦下。这两个例子都实测过；词法层只是 AST 的兜底，不单独承担安全。

补充：`correct_sql` 为什么是条件步骤

`correct_sql` 的条件执行值得展开，因为它是步骤目录里唯一的条件步骤，也是 S4 与 S6 的交界。`correct_sql` 在 `contracts/`

`steps.yaml` 里标 `conditional: true`，触发条件写在 `contracts/errors.yaml` 的 `correction` 段：`max_attempts: 1`、`revalidate: true`、重新过 `guard` 与 `policy`。`AgentRunner._run()` 的循环是「校验 → 失败 → 判断是否可重试且未超次数 → 校正 → 回到校验」；`SQL_REJECTED` 永远不可重试，执行阶段的失败也不再校正。这么严有具体原因。拿被安全策略拒绝的 SQL 去问模型改一下，等于给注入开重试口子；而执行阶段的数据库错误（列不存在、超时）不是「最小必要修正」能可靠修好的，再问一次只会得到一条说不准对不对的 SQL。没有配置 LLM 时，`_build_corrector()` 返回 `None`，步骤显式标 `skipped`，这是诚实，不是缺陷。S4 本身不跑校正；校正属于 S6 的失败通道，但它的策略契约在 S4 就已经被引用。

例题：为什么先静态校验，再谈执行

题目：SQL 最终要在数据库上跑，能不能干脆直接执行、让数据库报错，再把错误翻译成用户文案，省掉一套校验？这个方案在 S4 为什么不成立？

解答：执行是成本最高、副作用最大的一步。它要连库、要占连接、要扫数据；一条扫全表的语句可能拖垮实例。数据库的报错又是执行视角的：它能告诉你列不存在，却不会告诉你这条 SQL 是不是只读、是不是单语句、是不是把常量伪装成数据、排名有没有排序。把这些判断留给数据库，等于把语义是否可接受交给一个只懂语法的执行器。

S4 的顺序是：先用输出契约卡形状（只允许 `sql` 或 `refusal`），再用 AST 卡语义（单语句、只读、表列存在、无常量投影、排名有排序与限量）。两道关中任何一道不过，产物降级成带码的拒答。执行留给 S5，而且执行的是策略重新序列化后的 SQL。这样每一层只回答自己那一类问题：S4 回答这像不像一条可接受的查询，S5 回答这条查询能不能安全地跑。

结论：先静态校验再执行，理由在于把形状、语义、安全、执行四类判断分开；省数据库的报错只是附带效果。混在一起，错误会以数据库报错的形式出现，用户和排障者都拿不到正确的层级。

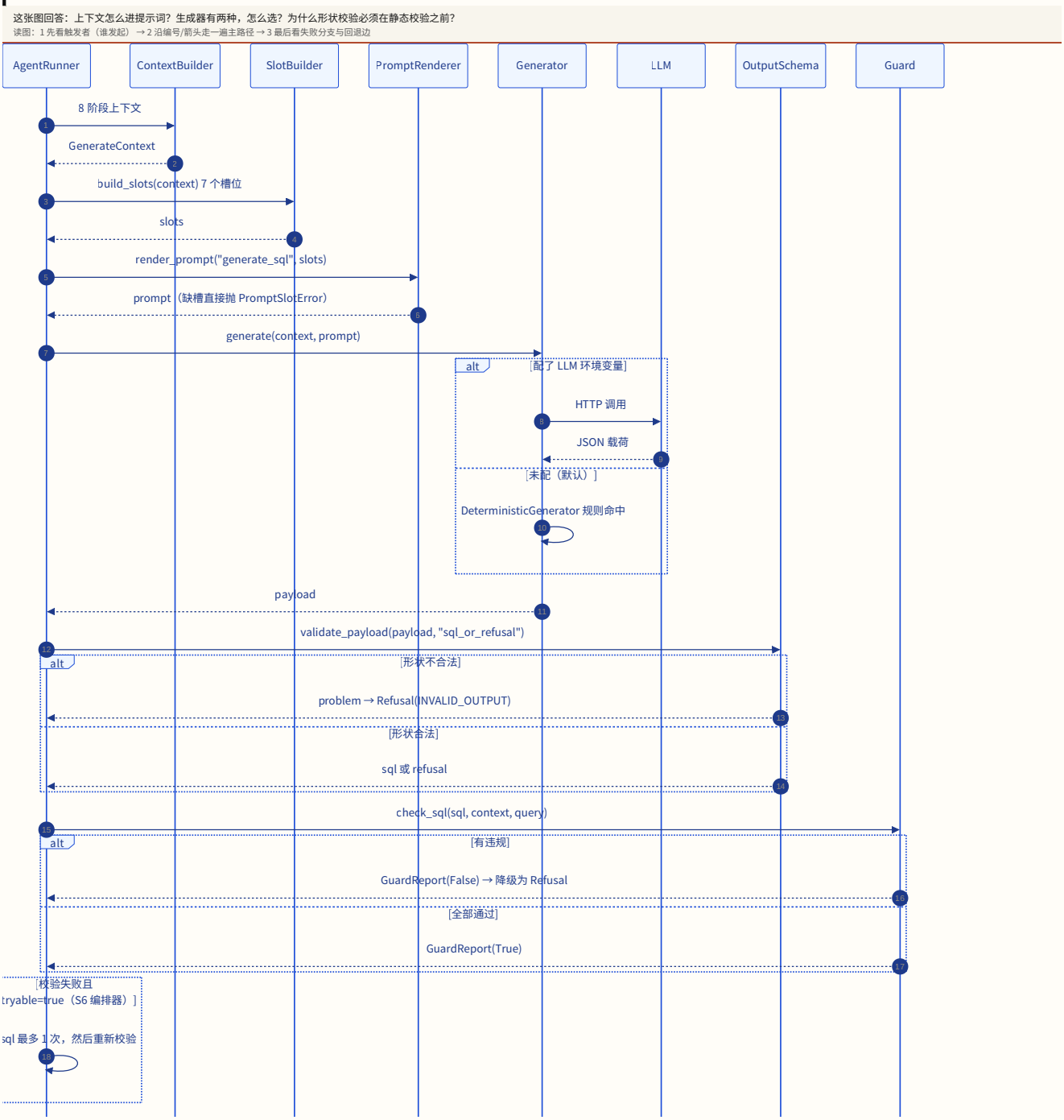


图 10 · S4 生成 SQL 流水线

图 10 的读法是「两道关顺序不能反」：形状不对的产物根本进不到静态校验；静态校验不过的产物也进不到执行。S4 只画到校验为止，执行是下一阶段的图。

5.5 输出

S4 的输出只有两种形状：

```
{"sql": "SELECT ..."}
```

```
{"refusal": {"code": "OUT_OF_RANGE", "reason": "数据只覆盖 20250101 ~ 20250331，问题里的 2024 年超出数据边界。"}}
```

10 题基线的结果是 10/10 通过，其中 8 条 SQL、2 条拒答。两条拒答很有代表性：「2024 年销售额」超出数据覆盖范围，码是 `OUT_OF_RANGE`；「各城市销售额」需要的城市粒度在契约里不存在（`dim_region` 只到省），码是 `UNSUPPORTED_INTENT`。这两条不算失败：拒答是契约允许的输出，也是生成器在边界内能做的正确选择。

先给三行摘录，完整 10 题在生成层测试的基线 YAML 里：

问题	产物	判定
华北销售额	一条聚合 SQL（下方完整代码块）	PASS：口径来自契约，时间范围已裁
各品类销量	一条分组 SQL	PASS：按商品维表的 `GROUP BY dim_product.category` 分组，没有混入地区表
2024 年销售额	拒答 JSON	PASS：`OUT_OF_RANGE`，不是硬编 SQL

华北销售额的完整产物（本工作树实跑）：

```
SELECT SUM(fact_order.order_amount) AS `销售额`
FROM fact_order
JOIN dim_region ON fact_order.region_id = dim_region.region_id
WHERE dim_region.region_name = '华北' AND fact_order.date_id BETWEEN 20250101 AND 20250331
```

这条 SQL 值得逐行看：`SUM(fact_order.order_amount)` 来自 GMV 的契约公式；`JOIN dim_region` 是为了把「华北」落到 `region_name`；`BETWEEN 20250101 AND 20250331` 是数据边界裁剪；没有 `GROUP BY`，因为问题问的是一个总数。`AS` 后面的中文名是列别名，不是常量投影；规则 4 禁止的是 SELECT 列表里出现字面量或裸 NULL（列别名不算）。「各城市销售额」被拒，是因为表的最细粒度到省（`province` 列），没有城市列；`region_name` 是省之上的大区分组。

上面那张表只写结论；判断口径要对回 SQL：有没有 `ORDER BY`、有没有把 `order_amount` 当单价、有没有对 `date_id` 做范围裁剪。这些细节才是「口径正确」的证据。

> **补充：拒答是输出，不是异常。** 两条拒答各自有明确的码：`OUT_OF_RANGE` 表示问题超出数据覆盖范围，`UNSUPPORTED_INTENT` 表示问题要求的粒度在契约里不存在。在当前数据与契约下，正确回答就是拒绝。把拒答改写成一条勉强能跑的 SQL，表面上是提高命中率，实际上是在制造一个没有依据的数字。

把这条放进全书的主线：S4 只负责生成层的有名字拒绝；执行层的失败（超时、0 行、取消）要到 S6 才有完整的错误码与类别。两套拒绝不要混为一谈。

5.6 验收

```
make stage4
```

期望输出（本工作树 2026-10-29 实跑）：

```
基线：10/10 通过（拒答 2）
108 passed, 5 skipped
S4 PASS
```

`10/10` 是基线，`拒答 2` 说明两条越界题按预期拒答而不是硬编 SQL；`108 passed, 5 skipped` 是 `tests/generate` 的结果，5 条 skip 是「需要真实 LLM / 真实存储」的按需测试。全量后端测试随后还会再跑一遍，数字以终端为准；不要把 `108` 当成全量。

10/10 是构造性基线：8 题来自黄金集，2 题是人为越界；题目按生成器能处理的意图设计。它证明「在这组题上，生成器按契约产出正确形状」，不证明「开放问法的生成能力」；后者要等接入 LLM 后再评估。

失败含义：如果基线不是 10/10，先看是哪一题、是 SQL 错还是拒答码错；如果 `tests/generate` 红，看是提示词槽位、输出形状还是某条 guard 规则；如果 skip 数变了，检查环境变量是否意外启用了 LLM。S4 不需要三存储，所以它红了基本可以排除 MySQL / ES / Qdrant。

5.7 常见故障

故障	先查什么	怎么修
`PromptSlotError`	提示词 frontmatter 的 `slots` 与 `build_slots()` 的键是否一致	两边同改；少槽不能补空串，必须显式失败
`INVALID_OUTPUT`	模型是否返回非 JSON / 代码块 / 多余字段	保留剥代码块逻辑；仍不合法就拒答，不要猜
`SQL_UNKNOWN_COLUMN`	上下文是否缺列；SQL 是否用了别名或拼写错误	先查 S1 契约与上下文裁剪；不要把校验放宽成「未知列也放行」
`SQL_MISSING_ORDER_BY`	问题是否有排名意图（最高 / Top N / 排名）	让生成器补 `ORDER BY`；不要删意图判断来「修绿」
`SQL_NOT_READ_ONLY`	SQL 是否含 UPDATE / DROP / INTO；是否多语句	这是硬拒绝；不要用「只检查第一句」绕过
基线某题 SQL 变了	确定性生成器、契约或提示词是否被改	先 diff 契约与生成器；基线是可复现证据，变化必须有原因
配了 LLM 但没走真模型	三个环境变量是否同时非空	`HttpLlmGenerator.from_env()` 缺一就抛错，主程序回退确定性生成器
`correct_sql` 一直灰着	是否配置了 LLM、错误是否 `retryable`	没配置 LLM 时显式 `skipped` 是正确行为；不要假装能校正

5.8 证据

结论	状态	证据
7 个提示词槽位，缺槽显式报错	✓	`contracts/prompts/generate_sql.prompt`； `provider.py:build_slots()`
输出只允许 `sql` / `refusal`	✓	`contracts/sql.output.schema.json`； `pipeline.py:draft_sql()`
静态校验五条硬规则	✓	`generate/guard.py:check_sql()`
10 题基线 10/10、拒答 2	✓	2026-10-29 本工作树 `--baseline`
`tests/generate` 108 passed / 5 skipped	✓	2026-10-29 本工作树 `pytest -q tests/generate`
真实 LLM 生成	?	无端点 / 无 Key； `HttpLlmGenerator` 未对真实服务执行
`correct_sql` 闭环	?	依赖 LLM；未配置时按设计 `skipped`
执行 SQL / 安全策略	?	属于 S5，不在 S4 边界内

5.9 检查清单

- ☐ 能说出 S4 只到「生成 + 静态校验」，不执行 SQL。
- ☐ 能区分确定性生成器与 LLM：同接口、同 schema、同校验。
- ☐ 能背出五条 guard 规则及对应错误码。

- [ ] 知道输出只有 `sql` 与 `refusal` 两种形状，没有第三种。
- [ ] 能解释 `correct_sql` 为什么是条件步骤：只有可重试错误、最多一次、改完重新校验。
- [ ] 知道 10 题基线里 8 条 SQL、2 条拒答，以及两条拒答各自的原因。

6. 走到这里，系统能做什么、还不能做什么

走到 S4，系统已经有一条**可复现的离线链路**：契约能校验，三份元知识能按计划构建（计数可验证），20 题能召回出稳定上下文，上下文能生成一条 SQL 或一次有名字的拒答，SQL 能过五条 AST 静态规则。它足以教学、足以评审、足以做回归；在没有任何 MySQL、LLM、API Key 的机器上，S0、S1、S4 都能跑出确定结果。

但必须明确它**还不能**做什么，否则读者会把「生成成功」误当成「可以上线」。

第一，不能安全执行。 S4 的 guard 只检查「表列在上下文里、语句只读、非常量投影、排名有排序」，它不做表白名单、列白名单、危险函数、集合运算、注释剥离、自动 LIMIT、执行超时、只读账号。一条过了 S4 的 SQL 仍可能扫全表、仍可能调用危险函数、仍可能因为账号权限过大而被滥用。这些是 S5 才补的：六道检查在连库之前，执行的是策略重新序列化后的 SQL，数据库账号 `dw_reader` 只有 `SELECT`，并叠加 `max_execution_time`。在 S4 停下来，你拥有的是一条「语义上像 SQL」的字符串，不是一次安全的查询。

第二，不能表达失败。 S4 的拒答只是生成层的一种输出；它还没有 16 个错误码、3 个类别（`failure` / `empty` / `cancelled`）、`retryable` 策略、校正闭环、取消、SSE 的 `error` 收尾。执行返回 0 行时，S4 没有 `EMPTY_RESULT` 这个概念；用户取消时，也没有独立的 `CANCELLED` 状态。这些是 S6 才补的失败通道。在 S4 停下来，失败要么是拒答，要么是一个异常；它还没有「名字、位置、通道」三件套。

第三，真实召回与真实 LLM 仍未接入主链路。 S3 的 ES/Qdrant 召回目前只在 `stage3` 与集成测试中使用，`/api/query` 默认走 `OfflineContextBuilder`；真实 LLM 需要三个环境变量齐全，且本机没有端点可验。「S3 通过」证明召回流水线成立，「S4 通过」证明生成与静态校验成立，但两者都还没有在生产请求里串起来跑真实数据。

第四，计数与写入仍有证据缺口。 `--dry-run` 的 5 / 24 / 6 / 9 / 76 / 98 / 22 已实测；241 是公式推导；`--fresh`、`--verify`、`NOOP: 0`

`writes` 因 Docker 权限未在本轮复现。读者可以照着命令跑，但不要把书里的 ⚠ 当成 ✓。

所以 S0–S4 的准确结论是：**骨架成立、契约成立、知识计划成立、召回流水线成立、生成与静态校验成立；安全执行与失败通道尚未成立。** 这也是下一部分从 S5 开始的原因：先把「能生成」变成「能安全执行」，再把「会失败」变成「失败有名字」。

把五个阶段压成一页纸，方便在评审或排障时快速定位：

阶段	消灭的不确定性	留下的产物	查询时是否在场	可停点 / 验收
S0	前后端是否说同一种事件语言	步骤目录、事件 schema、收尾规则	在：步骤目录与 schema 是运行时契约	前端能解析到 `result` / `error`；`make stage0`
S1	表、列、指标、值域以谁为准	契约、人读版契约、负向夹具结论	在：契约被 S2/S3/S4 读取	契约与真实数据对账；`make stage1`
S2	同一份契约翻成三种存储后是否等价	Meta、ES、Qdrant、manifest	在：三种存储就是查询时的知识	计数与 manifest 对上；`make stage2`
S3	检索能不能找到、会不会夹带噪声	候选上下文、trace、黄金集结果	不在主链路：当前用离线上下文	20 题黄金集全过；`make stage3`
S4	产物形状与语义是否可接受	SQL 或带码拒答、静态校验结论	在：生成与校验就是查询时的一步	基线全过、拒答按预期；`make stage4`

6.1 本部分小结

- S0 先让管道跑通。12 个步骤、1 个条件步骤；前端能解析 3 种 SSE 事件，才算真的跑通。
- S1 契约是唯一事实源。5 表 / 24 列 / 6 指标；旧系统的负向夹具必须红。
- S2 元知识三份存储。241 = 44 + 76 + 98 + 22 + 1，是公式推导，标 ⚠。
- S3 检索要能被证伪。20 题黄金集，每题都有 `must_not`。
- S4 生成 SQL 或明确拒答。基线 10/10，拒答 2。

如果只记一句：阶段不是进度条，是可停点；没有验收命令的阶段，不算完成。

把这张表倒过来读，就是排障顺序：先确认最后一条绿的验收命令是哪条，再决定往上看哪一层。S4 红了往生成层看，S3 红了往召回与存储看，S2 红了往构建与凭据看，S1 红了往契约与数据看，S0 红了往协议与前端看。每一层都有自己的失败模式，混着查只会把时间花在错误的层。

7. 复习题

下面十二题分三档：易题考概念与命令，中题考机制与故障定位，难题考设计取舍。建议先闭卷做易题，再开卷做中题；难题没有唯一答案，能说清权衡即可。参考答案不放在本章。

易题

1. S0 为什么要求「每条流必须以 `result` 或 `error` 收尾」？如果只要求「有事件」会漏掉什么故障?
2. `contracts/steps.yaml` 里为什么只有 `correct_sql` 是 `conditional`？如果前端把它渲染成永远 pending 的灰块，用户会误以为什么?
3. 用 `dim_region.region_name` 说明「粒度」与「值域」的区别；为什么它不能用来回答「各城市销售额」？
4. `fact_order.order_amount` 的 `role` 是 `measure`、`value_domain.kind` 是 `range`，这两点分别如何影响 S2 的 ES 同步与 Meta 的 `examples`？

易题做完后，用这四个判据自检：第 1 题看流是否以 `result` 或 `error` 收尾；第 2 题看 `conditional: true` 是否只出现在 `correct_sql`；第 3 题看 `region_name` 是不是唯一键、有没有城市粒度；第 4 题看 `role` 与值域形状是否同时满足同步条件。

中等题

1. 为什么 `--fresh` 那一步是重建，不能拿它当幂等检查？同一条 `make stage2` 里，哪一步才是零写入断言?
2. S2 的 manifest 里为什么要同时存 `config_hash`、`contract_version`、`targets`、`embedding_provider` 与计数？少一个会放过哪种「计数对但内容错」？
3. ES、Qdrant、Meta 在 S3 里各自负责什么？如果只保留 Meta，20 题黄金集最可能先坏在哪一类问题上?
4. 20 题黄金集为什么要同时有 `expected_*` 与 `must_not`？只有 `expected_*` 会鼓励哪种作弊式实现?
5. S4 的输出为什么只允许 `sql` 与 `refusal` 两种形状？「模型把错误信息写成一行数据」在这个契约下为什么不可能发生?

难题

1. 确定性生成器与 `HttpLlmGenerator` 共用哪三样东西？为什么「换成真模型」不会放宽任何一条硬规则?

2. `correct_sql` 的触发条件是什么？为什么 `SQL_REJECTED` 永远不可重试，而执行阶段的失败也不再校正？
3. 用一句话回答：S4 通过之后，系统还缺哪两件事才能算「能安全地问数」？它们分别由哪个阶段补？

> 复习题的参考答案不放在本章，避免读者先看答案再思考；它们会在附录或下一章的「阶段回看」里给出。

03

第三部分 ·

S5–S8：从能跑到敢用

旧系统会把错误文案当成数据发回来。模型生成的 SQL 是 `SELECT '无法生成SQL…' AS error_message`，它通过了 `EXPLAIN`；后端把这一行当成正常结果，前端渲染成 1 行 1 列的表。屏幕上没有红色，用户以为查到了数据。

先停一秒，问一个看起来矛盾的问题：`EXPLAIN` 都通过了，这条 SQL 为什么还是不该执行?

`EXPLAIN` 回答的是「数据库能不能解析并给出执行计划」，它不评价这句话该不该说出口。`SELECT '无法生成SQL…' AS error_message` 是一条完全合法的查询：不写库、不越权、不拖垮服务，数据库没有理由拒绝它。问题出在语义层：它把「模型放弃了」这件事伪装成了一行数据。按 S4 的规则，这条 SQL 属于常量投影，本该在生成质量那一层被拦下；旧系统把校验写成了 `explain {sql}` 的字符串拼接，既不检查投影，也不检查语句条数。于是它一路走到执行器，带着一句道歉变成了结果。

S5–S8 要解决的就是这种「看起来成功」的失败：拒绝要可判定，错误要有名字，类型要保真，部署边界要说清。

S0–S4 解决的是「系统能不能产出一条看起来像样的 SQL」。S5 开始，问题换了：这条 SQL 凭什么可以碰数据库？出错时用户能不能知道发生了什么？界面会不会把失败画成成功？把它装进容器以后，还有哪些事没有证明？四个问题分别对应 S5 安全边界、S6 失败通道、S7 前端收口、S8 生产化，也是把「能跑」变成「敢用」的四次交接。

这一部分每节的结构一样：先说上一阶段留下了什么风险，再说本阶段新增了什么不确定性，最后用一条可复现的命令证明这个不确定性已经被压住。凡是本设计工作树没有实际执行的，一律标 ?；凡是只有文档声称、没有本机输出的，一律标 !；凡是 2026-10-29 在本工作树跑过、输出可对照，或文件明确存在且可逐行引用的，才标 ✓。

> 拒绝要可判定，错误要有名字，类型要保真，部署边界要说清。

学习目标

读完这一部分，你应该能：

- 说出 S4 静态校验与 S5 安全策略各自回答的问题，并解释为什么前者不能替代后者；
- 沿着一条 SQL 走完 S5 的判定链，指出哪一步保证「执行的就是判定过的那条」；
- 用一张表说明 S6 的三种 `class` 如何决定界面，并解释空结果与取消为什么都不算失败；
- 从一段 SSE 字节流还原出前端的状态迁移，指出未知事件与未知步骤的处理为什么相反；
- 画出 S8 的启动依赖链，说明 `service_started` 为什么不够，并列出「换一台机器」的检查顺序；
- 分清已验证、推断与未验证，知道这一部分哪些结论还缺证据。

读之前需要什么

你应当读过上一章（S0–S4），知道系统能生成 SQL、能做静态校验；会 Python、SQL 与 HTTP/JSON；对容器、镜像、端口、卷有基本概念。AST 策略、SSE 解析细节、MySQL 权限模型、nginx 缓冲这四件事，本章不假设你会，首次出现时会拆开讲。

术语约定

本章反复用几个词：**契约**指仓库 `contracts` 目录下的事实源；**生成物**指由契约生成、不能手改的前端文件；**夹具**指录下来供逐字比对的真实事件流；**门禁**指 `make stage5` 这类可重复的验收命令；**语料**指注入测试用的 SQL 样本；**通道码**是暴露给前端的错误码，**原因码**是策略内部的具体违规码。

这一部分在全书的位置

S0–S4 证明系统「能生成」；S5–S8 证明它「敢用」。但「敢用」有边界：本章只讲到离线门禁与证据边界，不把 S8 写成已经上线。S8 之后的容量、可观测性、身份与传输安全属于后续工作，本章只标出缺口。

四道交接，一句话各自管什么

- **S5 管入口**：执行前判定，拒绝时不连库。
- **S6 管失败**：每种失败有名字、有类别、有步骤、有能否重试的结论。
- **S7 管呈现**：后端的契约直接变成前端的类型、步骤与界面状态。

- **S8 管交付**：把本机能跑变成换一台机器也能起，并把没证明的部分标出来。

四次交接问的是同一个句式：上一阶段留下了什么风险，这一阶段新增了什么不确定性，用什么命令证明它被压住。

0. 这一部分的地图

表 3-1 · **S5–S8 的阶段地图**

阶段	新增能力	新增的不确定性	可停点	前置
S5 安全边界	SQL 在连库前过 AST 策略；只读账号执行	生成出来的 SQL 可能带写操作、多语句、越权表列或外带构造	`make stage5`：36 条注入全拒、11 条正常查询零误杀；`make stage5-live`：只读账号写不了	S4 能产出 SQL；`contracts/schema.contract.yaml` 与 `sql.policy.yaml` 可读
S6 失败通道	16 个错误码、3 个 class、校正闭环、取消	失败可能没有名字、被归错类、被无限重试，或者取消被当成完成	`make stage6`：6 类失败逐条对上码 / 类 / 步骤；`make stage6-ui`：界面状态互不相同	S5 的拒绝已经是一个可判定的结论
S7 前端收口	契约生成物、SSE 三事件消费、StepRail / ErrorCard / 类型保真	前端可能按旧契约渲染、把未知事件当失败、把 Decimal 悄悄降级成字符串	`make stage7`：生成物新鲜度、14 条夹具逐字对账、前端契约测试与构建；`make stage7-ui`：13 条 F 缺陷	S6 的错误已经带 `code` / `class` / `step`
S8 生产化	Compose 六服务、健康检查、端口收敛、非 root 镜像	本机能跑不等于换一台机器能起；单 worker、无鉴权、无 TLS、无指标都是边界	目前没有已完成的 S8 性能门禁；Docker 全栈在本工作树标 ⓘ	S5–S7 的离线门禁与契约

可停点的意思是「这一类不确定性有了可重复的判据」，不是「功能做完」。S5 的判据是 36/36 与 11/11；S6 的判据是 16 个码和 6 类失败；S7 的判据是 14 条夹具逐字一致；S8 之所以还不能算完成，正是因为它还没有这样一条判据。

读这张表时，把「可停点」和「证据等级」分开看。可停点回答「这一类不确定性有没有判据」，证据等级回答「这条判据在本工作树跑没跑过」。S5 的判据写在表里，也真的在本机跑出了 36/36；S8 连判据都还没有成形，所以它的停点是空的。这两件事不能合并成一句「做完了」或「没做完」。

表里的标记只描述本工作树 2026-10-29 的状态，不描述设计质量。一个标为未验证的条目可能是设计正确、只是环境不具备；一个标为已验证的条目也可能只证明了一条窄路径。读后续各节时，先看结论后面的证据，再看结论本身。

表 3-1a · 四道交接的一页纸

交接	上一阶段留下的风险	这一阶段压住的不确定性	判据长什么样
S4 → S5	SQL 看起来对，但可能写库、越权、拖垮服务	生成文本能否在执行前被判定为安全	注入全拒、正常查询零误杀
S5 → S6	拒绝是一个结论，但用户看不到结论	失败能否变成有名字、有类别的事件	每类失败对上码、类、步骤
S6 → S7	事件有名字，但界面可能不认	契约能否直接约束类型、步骤与渲染	生成物与夹具逐字一致
S7 → S8	开发机上能跑、能失败、能展示	同一套文件能否在另一台机器上起来	启动链与验收脚本

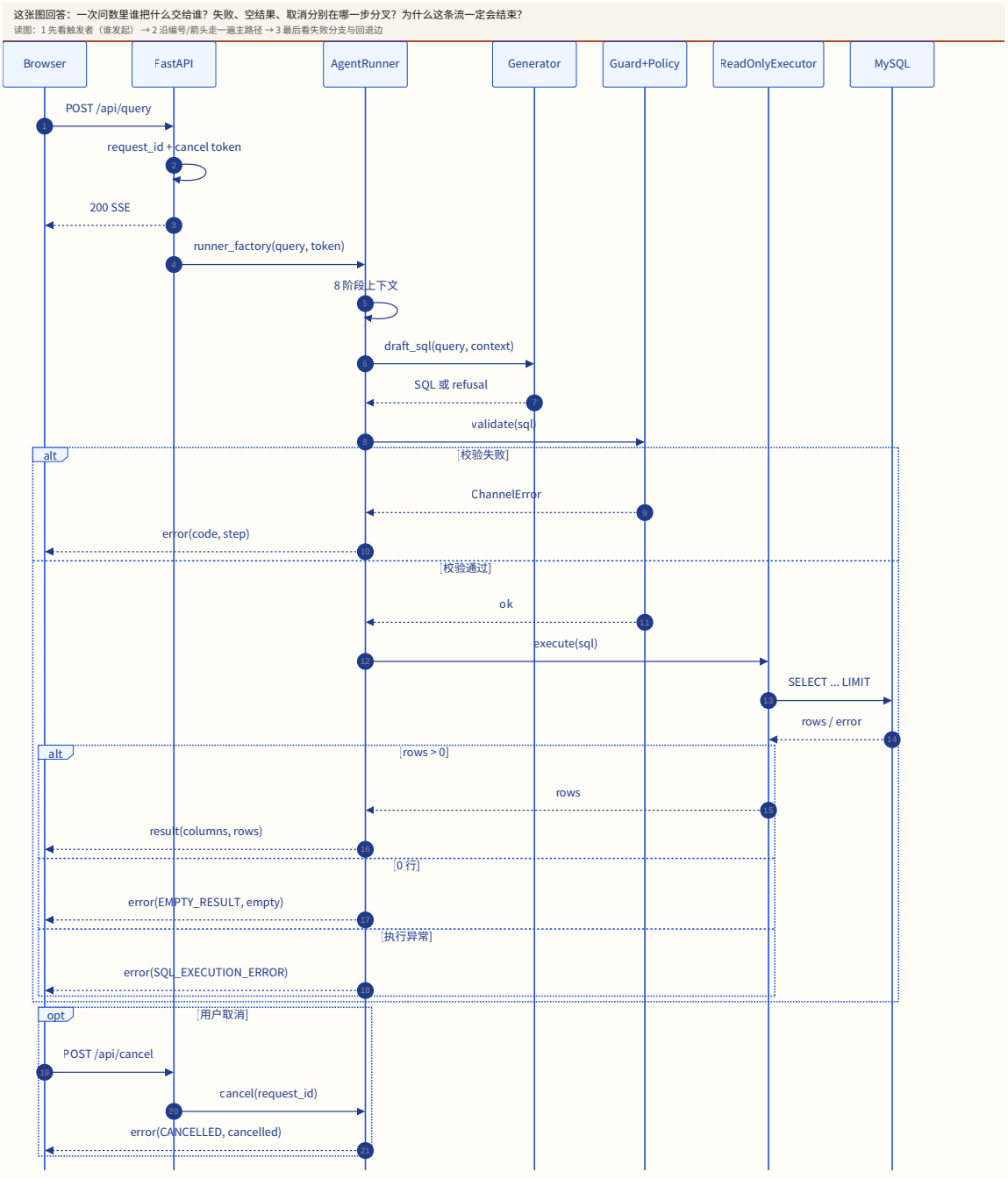


图 07 · 端到端请求生命周期

图 07 把 S5–S7 放进同一条请求：S5 负责执行前的闸，S6 负责失败怎么变成事件，S7 负责事件怎么变成界面。三者共用一条 SSE 管道，任何一层的契约漂移都会在下一层显形。

1. S5 · 安全边界

1.1 问题：为什么 S4 的 SQL 不能直接执行

S4 结束时，系统已经能生成 SQL，也能做 AST 静态校验。但 S4 的校验回答的是「这条 SQL 像不像这个问题的答案」，不是「这条 SQL 有没有资格碰数据库」。这两件事必须分开，因为它们的输入不同、失败后果也不同。

把两件事混在一起，最常见的写法就是「先 `EXPLAIN` 一下，能过就执行」。这条路径有三个盲点。`EXPLAIN` 只编译第一句，多语句里的第二句不进入计划；它不检查账号有没有写权限，只检查语法与对象是否存在；它也不关心投影出来的是不是一行伪装成数据的错误文案。换句话说，`EXPLAIN` 证明的是「数据库看得懂」，不是「这句话该被执行」。

旧系统的教训写在 `fixtures/injection/sql-injection.yaml` 的文件头里：旧 `validate()` 做的是 `f"explain {sql}"`，只给第一句加 `EXPLAIN` 前缀，不检查语句条数。MySQL 对 DDL 自动提交，所以下面这条语句在旧系统里会真的把 `fact_order` 删掉：

```
SELECT 1 AS a FROM fact_order LIMIT 1; DROP TABLE fact_order
```

问题出在校验和执行不是同一条语句，跟模型偶尔胡说无关。`EXPLAIN` 只看第一句，第二句照原样执行；只要生成器被诱导一次，整张事实表就没了。这条语句之所以能过旧校验，是因为 `EXPLAIN` 与执行分属两次调用：校验时把整段文本加上前缀交给数据库，执行时又把原始文本原样交给驱动，两次调用之间没有任何东西保证它们是同一条语句。S5 的核心不变式就从这里来：**执行的文本必须由被判定过的那棵 AST 生成**，不能是「同一段字符串再拼一次」。

S4 的静态校验能挡住一部分「生成质量问题」，但它不是执行安全层：它不知道数据库账号有没有写权限，也不负责把「校验通过的那条 SQL」和「真正执行的那条 SQL」钉成同一条。

所以 S5 的问题可以写成一句话：**在打开数据库连接之前，如何把「一条 SQL 文本」变成一个可解释的放行 / 拒绝结论，并保证执行的就是被判定过的那条 SQL？**

这句话里有三个词要兑现。「可解释」意味着拒绝要带原因码，不能只说不行；「放行 / 拒绝」意味着结论只有两类，执行错误是放行之后的另一件事；「执行的就是被判定过的那条」意味着判定必须产出可执行的 SQL，不能只产出一个布尔值。

1.2 实体与边界：策略、执行器、账号

S5 有四个实体，边界清楚才不会互相冒充。

第一是**策略**。`backend/app/security/policy.py` 的 `enforce()` 是纯函数式的判定器：输入 SQL 文本、白名单、策略配置，输出 `PolicyDecision`。它不连库，不抛「拒绝异常」，而是把违规收进 `violations`，让调用方决定怎么降级。拒绝因此是一个结论，不是一次崩溃。

第二是**执行器**。`backend/app/security/executor.py` 的 `ReadOnlyExecutor` 负责连接、会话超时、执行和取行。它在 `execute()` 的第一行先调用 `enforce()`；策略不通过时直接返回 `SQL_REJECTED`，**连连接都不打开**。这一条是 S5 的硬边界：没有数据库往返，就没有「校验过了但跑的是另一条」的机会。

第三是**账号**。执行器只使用 `dw_reader` 连接 `dw`，这个账号在 `dw` 上只有 `SELECT`。策略是代码，账号是数据库；两者独立生效。旧系统的 `didilili` 是 `GRANT ALL PRIVILEGES ON dw.*`，这正是 D6 能发生的根本原因。新系统不复用它，只把 `didilili` 留给构建与初始化，把查询路径收窄到 `dw_reader`。

第四是**白名单**。哪些表、哪些列可以出现，不在 `sql.policy.yaml` 里再抄一遍，而是从 `contracts/schema.contract.yaml` 派生。`SqlWhitelist.from_catalog()` 把契约里的 5 张表、24 个列变成执行期的集合，避免「契约改了、白名单没改」的漂移。

四个实体各守一条边：策略管「文本能不能执行」，执行器管「这条路径必须先问策略」，账号管「即使绕过程序也写不了」，白名单管「哪些表列算数」。任意一条单独拿出来都不够。策略写对了但执行器先连库，连接已经发生在判定之前；执行器顺序对了但账号仍有写权限，一次漏判就是一次真实写入；账号收窄了但白名单靠手写，契约一改就会漂移。

这张图回答：策略在哪两个地方各查一次？被拒绝时会不会连库？超时是怎么产生的？
读图：1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边

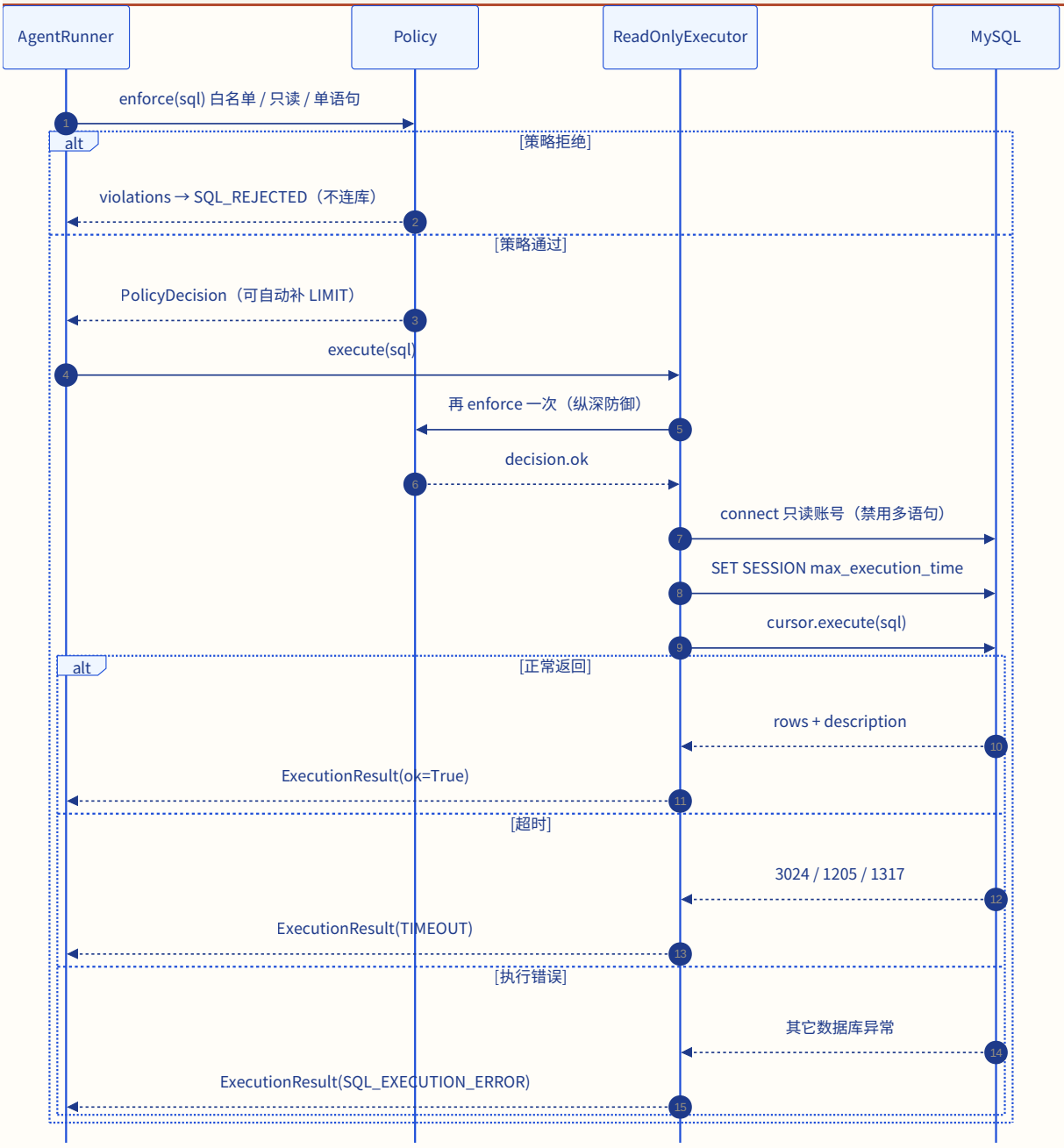


图 11 · S5 安全校验与执行

图 11 的时序值得逐段读：`AgentRunner` 先问 `Policy`，拒绝就返回 `SQL_REJECTED`，根本不进入 `MySQL`；放行才把 `PolicyDecision.sql` 交给 `ReadOnlyExecutor`。执行器在连库前又跑一次 `enforce()`，这是纵深防御，不是重复劳动：自定义执行器、校正后的 SQL、未来新增的调用方，都可能绕过第一道调用点。

这张图回答：每张表一行代表什么？主键是什么？事实表如何关联维度？哪些列是真实存在、哪些是常见误问（city、price）？
读图：1 先看实体与 grain（一行代表什么）→ 2 再看主键/外键与关系 → 3 最后看约束、值域与口径

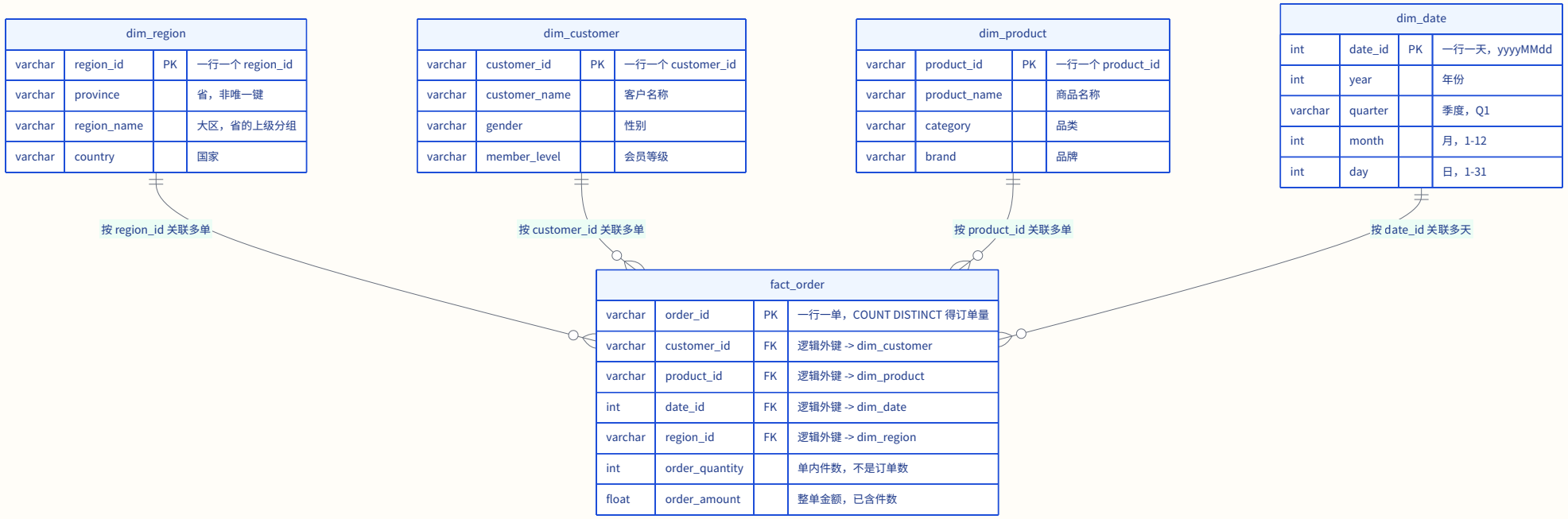


图 14 · 数仓 ER 图（5 表）

图 14 是白名单的现实依据。`dw` 里只有 `dim_region`、`dim_customer`、`dim_product`、`dim_date`、`fact_order` 五张表；`fact_order` 的 `order_quantity` 是单内件数，`order_amount` 是整单金额。策略拒绝 `information_schema.tables`，理由是 `information_schema` 这个库不在 `dw` 白名单里，跟名字可不可疑无关；策略拒绝 `secret_table`，是因为契约里根本没有这张表。每条拒绝都能落回一张真实的表或一个真实的列。

1.3 输入：SQL 文本 + 契约 + 策略

S5 的离线判定只需要三样输入，都不依赖数据库：

1. **SQL 文本**：来自 S4 的生成结果，或校正闭环给出的候选 SQL。

2. **策略契约**：`contracts/sql.policy.yaml`。它声明单语句、根节点只允许 `Select`、禁止集合运算、禁止可执行注释、写操作禁词、危险函数禁词、`default_limit: 1000`、`max_limit: 1000`、`max_execution_time_ms: 5000`、只读账号 `dw_reader`，以及对外通道码 `SQL_REJECTED`。
3. **Schema 契约**：`contracts/schema.contract.yaml`。它提供表白名单与列白名单；库名 `dw` 也来自策略契约，不在代码里再写一遍。

这三样都是仓库里的文件，所以 `make stage5` 在任何一台有 Python 依赖的机器上都能给出同样的结果。这也是为什么 S5 的离线门禁不需要 MySQL：策略判定不查库，只查契约。

需要数据库的是「只读账号真的写不了」这一条，拆在 `make stage5-live`。它需要 `REBUILD_DW_RO_PASSWORD`，密码不在仓库里，由部署环境提供。这个拆分是刻意的：**策略的正确性**可以离线证明，**账号的权限**必须在线证明，两者不能互相代替。

所以 `make stage5` 通过只证明判定器，不证明账号；两者都过，才叫「恶意 SQL 进不来」。

1.4 动作：AST 解析 → 策略判定 → 只读执行

`enforce()` 的判定顺序是排过的：

1. 可执行注释 `/*! ... */`：服务端会执行，解析器会忽略，必须在解析前先抓。
2. 解析：`sqlglot.parse(text, read="mysql")`。解析失败时，先看词法禁词，再退到 `SQL_SYNTAX`。
3. 单语句：过滤掉 `None` 与 `Semicolon` 后，如果多于一条，直接 `SQL_MULTIPLE_STATEMENTS`。
4. 集合运算：`Union` / `Except` / `Intersect` 一律拒绝，原因码 `SQL_FORBIDDEN_CONSTRUCT`。
5. 根节点：只允许 `Select`；`Update`、`Delete`、`Drop` 等根节点给 `SQL_NOT_READ_ONLY`。
6. 任意位置的写节点：再遍历一次 AST，防止写操作藏在子查询或 CTE 里。
7. 危险函数：`SLEEP`、`BENCHMARK`、`LOAD_FILE`、`GET_LOCK`、`EXTRACTVALUE`、`UPDATEXML`。
8. 词法兜底：AST 看不见的 `INTO OUTFILE` 之类，交给 `scan_keywords()`。
9. 表白名单：库名必须是 `dw`，表名必须在契约的 5 张表里。

10. 列白名单：限定列按别名解析到真实表；无限定列允许命中投影别名（例如 `ORDER BY 销售额`）。
11. LIMIT：没有就补 `default_limit`，超过 `max_limit` 就截断。
12. 重新序列化：剥掉注释，生成最终 SQL。

把这十二步当成一条流水线，而不是一张检查清单：每一步只回答一个问题，多数步骤一旦给出确定结论就短路返回。第 2 步解析失败、第 3、4 步命中时直接返回；第 1、5、6、7、8、9、10 步把违规记进 `violations`，最后统一拒绝。短路的方向也排过：先抓服务端会执行、解析器会忽略的可执行注释，再解析；解析失败时先看词法禁词，再退到语法错；集合运算先于根节点判定，因为 `Union` 的根节点本身是只读的，但它是外带数据的载体，原因码必须说「构造不允许」；写节点再遍历一次，是因为写操作可以藏在子查询或 CTE 里；词法兜底放在 AST 之后，是为了让原因码更准——`SELECT 1; DROP TABLE t` 应该说「不是单条语句」，而不是「出现了 DROP」。

也有些步骤先记账、最后一起返回：可执行注释与白名单违规会累积进 `violations`，等所有检查跑完再统一拒绝。两种写法都保证同一个结果：只要 `violations` 非空，就没有执行路径。

例题：一条正常查询走完判定链

拿一条正常查询走一遍。输入 `SELECT SUM(fact_order.order_amount) AS gmv FROM fact_order`：第 1 步没有可执行注释；第 2 步解析出一条 `Select`；第 3 步的单语句检查通过；第 4 步没有集合运算；第 5 步根节点是 `Select`；第 6 步遍历不到写节点；第 7 步没有危险函数；第 8 步词法兜底不触发；第 9 步库名缺省，表 `fact_order` 在契约的五张表里；第 10 步限定列 `order_amount` 解析到 `fact_order`，在二十四列之内；第 11 步没有 LIMIT，补上默认上限；第 12 步剥掉注释、重新序列化，得到可执行文本。每一步只做一个判断，判定链结束时以 `violations` 是否为空收口。

第 12 步最关键：策略返回的 `decision.sql` 是 AST 重新序列化后的产物，不是原始文本。执行器只执行这个产物。这样「校验的那条」和「跑的那条」在结构上就是同一条；否则 D6 的漏洞只是换了一个地方复发。

关键代码不长，第 3 步的判定可以对照 `backend/app/security/policy.py:enforce` 读：

```
statements = sqlglot.parse(text, read=dialect)
meaningful = [
    statement for statement in statements
    if statement is not None and not isinstance(statement, exp.Semicolon)
]
```

```
    if cfg.single_statement and len(meaningful) > 1:
        violations.append(PolicyViolation(
            "SQL_MULTIPLE_STATEMENTS",
            f"SQL 必须只有一条语句，实际解析出 {len(meaningful)} 条",
        ))
        return _reject(original, violations)

    tree = meaningful[0]
```

最后两步决定了这条流水线的产物，同样可以在同一个函数末尾读到：

```
        existing = _existing_limit(tree)
        if existing is None:
            tree = tree.limit(cfg.default_limit)
            applied_limit = cfg.default_limit
        elif existing > cfg.max_limit:
            tree = tree.limit(cfg.max_limit)
            applied_limit = cfg.max_limit
        else:
            applied_limit = existing

        final_sql = _strip_comments(tree).sql(dialect=dialect)
```

`decision.sql` 是这棵 AST 重新序列化的结果，不是原始文本。原文里的注释、大小写、空白都不保证保留；不变式要求的是「执行文本由这棵 AST 生成」，不是「字节不变」。基线那句话在这里落地：**执行的就是判定过的那条**。这条不变式还有一个推论：策略可以改写 SQL，而且改写是显式的。补 LIMIT、截 LIMIT、剥注释都会改变用户看到的文本，但不会改变语义边界。

AST 与词法兜底的分工，可以压成一句话：AST 负责「我认得的结构」，词法负责「我认得但解析器看不见的写法」。`INTO OUTFILE` 这类构造经常让解析器直接失败，此时 AST 什么都给不了，只能靠 `scan_keywords()` 在文本里找禁词；可执行注释则相反，解析器会把它当普通注释丢掉，必须在解析前用 `has_executable_comment()` 抓住。解析前抓可执行注释，解析失败后与 AST 判定后各扫一次词法禁词；三层各管一段，不互相替代。

执行器这一侧有四道闸，代码在 `backend/app/security/executor.py:ReadOnlyExecutor`：

```
    decision = enforce(sql, whitelist=self.whitelist, config=self.config)
    if not decision.ok:
        return ExecutionResult(ok=False, sql="", error=ExecutionError(
            code=self.config.channel_code,
            message=first.message,
```

```
            reason_code=first.code,
        ))
    return self._run(decision)
```

`_run()` 里再设置 `SET SESSION max_execution_time = 5000`，并用 `CLIENT.CAPABILITIES & ~CLIENT.MULTI_STATEMENTS` 显式去掉多语句标志。注意最后这一句：即使策略被绕过，连接本身也只可能跑一条语句。这是「代码」和「驱动」两层独立生效，不是同一道检查写两遍。

「先连库再判定」是一个值得走一遍的反事实。假设执行器把顺序倒过来：先连库，再跑 `enforce()`。功能上它仍然会拒绝，测试也可能全绿，但风险结构变了。判定之前就消耗了一次数据库连接与一次会话初始化，恶意请求可以用合法形状的 SQL 让每一次拒绝都先付一次连接成本。更麻烦的是，一旦某个调用方在判定与执行之间插入自己的逻辑，判定结果与执行文本就可能分叉，D6 的漏洞换一个地方复发。连接建立后如果先做会话设置，那段语句也发生在判定之前，等于让未判定的请求先影响了数据库会话。S5 把 `enforce()` 放在 `execute()` 的第一行，就是为了让「拒绝」与「没连过」成为同一件事。

1.5 输出：放行 / 拒绝 / 执行错误

S5 有三类输出，不能混成一句「失败了」。

放行：`PolicyDecision(ok=True, sql=..., applied_limit=1000)`。`sql` 是去掉注释、补过 LIMIT 的最终文本。比如 `SELECT SUM(fact_order.order_amount) AS gmv FROM fact_order` 会被改写成带 `LIMIT 1000` 的版本；聚合查询本来只返回一行，但策略仍然按统一规则补上上限。

拒绝：`ok=False`，`violations` 至少一条，`reason_code` 是第一条违规的原因码。对外通道码统一是 `SQL_REJECTED`，细粒度原因放在 `reason_code`。这是刻意的：如果「多语句」「写语句」「非白名单表」各自变成一个对外通道码，下游就要为每种拒绝写一套分支，失败又会长得不一样。原因码保留在 `detail.reason_code` 里，机器仍然能判定，人也能读懂。

执行错误：策略通过后，数据库仍可能报错。`ExecutionResult.error` 用 `code` 表示通道码，用 `reason_code` 表示数据库类别。MySQL 错误码的映射写在 `_ERROR_CODE_MAP` 里：`1064` → `SQL_SYNTAX`，`1146` → `SQL_UNKNOWN_TABLE`，`1054` → `SQL_UNKNOWN_COLUMN`，`3024` / `1205` / `1317` → `TIMEOUT`，其余 → `SQL_EXECUTION_ERROR`；客户端侧的 `2006` / `2013` 也按超时处理。这

是实现层的归类，不是对错误本质的判断：2006 / 2013 本质是连接中断，文案写成超时会把排查带偏。

三类输出互斥：`ok=True` 只有一种，`ok=False` 只有一种，执行错误只在 `ok=True` 之后出现。把它们混成「失败了」会让下游失去判断依据——策略拒绝可以教育用户改问题，执行错误应该重试或报障，两者的处置完全不同。

这里有一个容易混淆的点：`SQL_TABLE_NOT_ALLOWED`、`SQL_COLUMN_NOT_ALLOWED`、`SQL_FORBIDDEN_CONSTRUCT`、`SQL_COMMENT_OBFUSCATION` 是**策略原因码**，不是 `contracts/errors.yaml` 里的 16 个通道码。对外只出现 `SQL_REJECTED`，原因码在 `detail` 里。S6 的错误码表不会为这四种原因各开一个通道；它只要求「拒绝」这件事有一个稳定的名字。

这条分层在验收里也能看见：注入语料核对的是 `reason_code`，通道码只出现一次。原因码是给人看的诊断，通道码是给机器看的分支。把原因码提升成通道码，等于让每种攻击手法各占一个对外分支，攻击面越宽，下游越不稳。

1.6 验收：注入语料 100% 拒绝、零误杀、只读账号写不了

离线门禁的命令来自 `Makefile` 的 `stage5`：

```
make stage5
```

它依次做四件事：

```
cd backend && uv run python -m app.scripts.security --check
cd backend && uv run python -m app.scripts.security --corpus
cd backend && uv run python -m pytest -q tests/security
cd backend && uv run pytest -q
```

`uv` 是 Python 的包与环境管理器；它按锁文件准备环境再执行，避免「本机装了什么就跑什么」。

四条命令各证明一层：`--check` 证明语料与原因码，`--corpus` 证明注入/放行两组语料的整体判定，安全测试证明边界，全量测试证明这次改动没有碰坏别处。任何一条单独绿都不够；四条一起绿，才叫「离线门禁通过」。

2026-10-29 在本工作树执行 `--check`，输出如下（✓）：

```
== 1/3 注入语料（必须 100% 拒绝） ==
  注入语料：36/36 条按预期拒绝
== 2/3 零误杀语料（必须全部放行） ==
  零误杀语料：11/11 条放行
== 3/3 S4 基线零误杀 ==
  S4 基线：8/8 条 SQL 通过策略
S5 PASS
```

「36 条全拒」不够，还要「原因码精确匹配」。否则把所有东西都拒掉也能得到 36/36，但正常问题会被误杀。所以 `_check_corpus()` 同时核对 `decision.reason_code == case.reason_code`；`_check_allow()` 再跑 11 条「长得像正常查询」的语料，必须全部放行。

表 3-2 · **五条真实注入 SQL 与本机判定结果（✓ 2026-10-29）**

语料 id	SQL	判定	原因码	为什么被拒
`multi_statement_drop`	`SELECT 1 AS a FROM fact_order LIMIT 1; DROP TABLE fact_order`	拒绝	`SQL_MULTIPLE_STATEMENTS`	`sqlglot.parse` 解析出 2 条语句；策略在单语句检查处直接返回，第二句没有机会执行
`update_statement`	`UPDATE fact_order SET order_amount = 0 WHERE 1 = 0`	拒绝	`SQL_NOT_READ_ONLY`	根节点是 `Update`，不在 `allowed_roots`；同时遍历到写节点 `Update`，给同一条原因码
`union_exfiltration`	`SELECT order_amount FROM fact_order UNION SELECT customer_id FROM dim_customer`	拒绝	`SQL_FORBIDDEN_CONSTRUCT`	`Union` 是集合运算，可能把别的表的数据并进结果；策略在根节点判定前先拒绝它
`executable_comment_drop`	`SELECT 1 FROM fact_order; /*! DROP TABLE fact_order */`	拒绝	`SQL_COMMENT_OBFUSCATION`	`/*! ... */` 是 MySQL 可执行注释：服务端会执行，解析器会忽略；第 1 步就抓
`information_schema_tables`	`SELECT table_name FROM information_schema.tables`	拒绝	`SQL_TABLE_NOT_ALLOWED`	库 `information_schema` 不在白名单（只允许 `dw`）；列 `table_name` 也不在契约 24 列里

对应的本机 CLI 输出可以逐条复现：

```
cd backend
uv run python -m app.scripts.security --policy "SELECT 1 FROM fact_order; DROP TABLE fact_order"
# 判定：拒绝（原因码 SQL_MULTIPLE_STATEMENTS）
#   - SQL_MULTIPLE_STATEMENTS: SQL 必须只有一条语句，实际解析出 2 条
```

```
cd backend
uv run python -m app.scripts.security --policy "SELECT order_amount FROM fact_order UNION SELECT
customer_id FROM dim_customer"
# 判定：拒绝（原因码 SQL_FORBIDDEN_CONSTRUCT）
#   - SQL_FORBIDDEN_CONSTRUCT：出现集合运算 Union
```

零误杀也要能举出一条真实的放行结果。`simple_aggregate` 的输入是：

```
SELECT SUM(fact_order.order_amount) AS gmv FROM fact_order
```

本机判定是「放行（自动 LIMIT = 1000）」，可执行 SQL 变成：

```
SELECT SUM(fact_order.order_amount) AS gmv FROM fact_order LIMIT 1000
```

这就是 S5 的「改写后执行」：用户看到的 SQL 和执行器跑的 SQL 可以不同，但差异只有注释与 LIMIT，且这个差异是策略显式做出的。

只读账号的验收是另一条命令，来自 `Makefile` 的 `stage5-live`：

```
source scripts/export-docker-env.sh
make stage5-live
```

它做三件事：

```
cd backend && uv run python -m app.scripts.security --provision-ro-account
cd backend && uv run python -m app.scripts.security --verify-ro-account
cd backend && uv run python -m app.scripts.security --demo
```

`--provision-ro-account` 用 `DROP USER IF EXISTS` + `CREATE USER` + `GRANT SELECT ON dw.*` 开通 `dw_reader`，权限集合恰好是策略声明的那一项。`--verify-ro-account` 读 `SHOW GRANTS`，并且**真的去写一次**：执行 `UPDATE ... WHERE 1 = 0`，匹配不到任何行，但数据库仍会先检查 `UPDATE` 权限；如果账号只有 `SELECT`，这一步会被拒绝。`--demo` 是两屏对比：同一段 payload，旧系统 `explain {sql}` 校验通过并执行了第二句的 `DROP`，新系统在连库之前就拒绝。

本工作树没有 `REBUILD_DW_RO_PASSWORD`、也没有启动 MySQL，所以 `make stage5-live` 标 ⑦。策略验过了，账号在线行为没验，两者分开记账。

1.7 常见故障

故障一：`make stage5` 报缺少只读密码。 不会。`make stage5` 是离线门禁，不连库；报缺 `REBUILD_DW_RO_PASSWORD` 的通常是 `stage5-live`，或者你在本地起了后端却没有配只读账号。后端的执行步骤会以 `INTERNAL_ERROR` 收尾，而不是编一份假数据；这是 S6 定义的「诚实的失败」。

故障二：把 `SELECT 1` 当成「策略漏了」。 `SELECT 1` 在注入语料的 `allow` 段里，策略确实放行它。原因是常量投影归 S4 的生成质量校验（`SQL_CONSTANT_PROJECTION`），不归 S5 的执行安全。S5 只回答「这条 SQL 会不会写库、越权、拖垮服务」；`SELECT 1` 不写库、不越权，所以策略放行。真正的问数链路上，S4 的 guard 会先把它拦下；S5 是执行前的最后一道，不是唯一一道。

这条 SQL 值得多停一步：它在 S5 是放行的，在 S4 是拒绝的。两个结论都正确，因为它们回答的问题不同。S4 的 guard 管「这条 SQL 像不像这个问题的答案」，常量投影意味着模型没有真的查数据，只是拿一个常量占位；S5 管「这条 SQL 会不会写库、越权、拖垮服务」，`SELECT 1` 三样都不占。真正的问数链路上，S4 先跑，`SELECT 1` 到不了执行器；S5 的放行只说明「如果它到了这里，也不会造成危害」。把两层当成一层，才会得出「策略漏了」的错觉。

故障三：列白名单误杀 `ORDER BY 销售额`。 `销售额` 不是物理列，是投影别名。`_projection_aliases()` 会把 `exp.Alias` 的名字收集起来，无限定列命中别名时放行。这条规则由 `group_by_with_cn_alias` 与 `order_by_alias` 两条 allow 语料钉住；如果改坏了，零误杀会立刻变红。

故障四：可执行注释漏过。 检查顺序里第 1 步就是 `has_executable_comment()`，不是等 AST 解析。原因是 `/*! ... */` 在 MySQL 里是「注释形式，但服务端执行」；解析器会把它当注释忽略，等你看到 AST 时已经晚了。语料 `executable_comment_union` 与 `executable_comment_drop` 就是为这条准备的。

故障五：执行器超时没生效。 `_prepare_session()` 执行 `SET SESSION max_execution_time = 5000`，失败会显式抛出，不静默降级。客户端还有 `read_timeout=10` 作为兜底。`max_execution_time` 只对 `SELECT` 生效，这也是为什么「只允许单条 SELECT」和「超时」要一起看。

故障六：以为「策略拒绝」等于「数据库没连」。 在 `ReadOnlyExecutor.execute()` 里确实如此：`enforce()` 不通过就直接返回，不进 `_run()`。但如果你绕过执行器，直接拿 `decision.sql`

去连库，那就是另一个调用方的问题。S5 的保证是「执行器这条路径」；新增入口必须复用 `enforce()`。

1.8 证据

- `contracts/sql.policy.yaml`：单语句、根节点、集合运算、可执行注释、写词、危险函数、LIMIT、超时、账号、通道码与原因码（✓ 文件存在，逐项可读）。
- `contracts/schema.contract.yaml`：5 表 / 24 列 / 6 指标，白名单的唯一来源（✓）。
- `backend/app/security/policy.py:enforce`：12 步判定与重新序列化（✓ 文件存在）。
- `backend/app/security/executor.py:ReadOnlyExecutor`：四道闸、错误码映射、会话超时（✓）。
- `fixtures/injection/sql-injection.yaml`：36 条 reject + 11 条 allow（✓ 文件存在；本机 `--check` 输出 36/36 与 11/11）。
- `make stage5` 的 `--check` 输出（✓ 2026-10-29 本工作树）。
- `make stage5-live`：只读账号开通、验证、两屏对比（? 本工作树无 MySQL / 无 `REBUILD_DW_RO_PASSWORD`）。

1.9 检查清单

- [ ] 我能说出 S4 静态校验和 S5 安全策略各自回答什么问题。
- [ ] 我能解释为什么策略拒绝时执行器不连库。
- [ ] 我能背出至少三条注入 SQL 及其原因码。
- [ ] 我知道 `SQL_REJECTED` 是通道码，`SQL_MULTIPLE_STATEMENTS` 是原因码。
- [ ] 我知道白名单来自 `schema.contract.yaml`，不是手写在策略里。
- [ ] 我知道执行的是 `decision.sql`，不是用户输入的原文。
- [ ] 我知道 `make stage5` 与 `make stage5-live` 的依赖不同。
- [ ] 我知道只读账号的密码来自 `REBUILD_DW_RO_PASSWORD`，不在仓库里。

2. S6 · 失败通道

2.1 问题：S5 留下了什么风险

S5 让「拒绝」变得可判定，但它只解决了执行前的那一半。剩下的一半是：**拒绝之后，用户看到什么？** 旧系统在这里有三个更深的缺陷：

第一，后端一次都没发过 `type=error`。模型编出 `SELECT '无法生成SQL…' AS error_message`，这条 SQL 能过 `EXPLAIN`，于是后端把它当成一行正常数据发回来，前端渲染成一张 1 行 1 列的表，全屏没有一处红色。用户以为「查到了 1 条数据」。

第二，空结果被当成失败，或者失败被当成空结果。审计 FP-8 的结论是：`[]` 是合法语义，查询成功但没有数据，不应该标红。旧系统没有 `empty` 这个类别，所以「没有数据」和「查询炸了」只能挤在同一套红框里。

第三，取消被当成完成。旧前端写的是 `status: isAbort ? "done" : "error"`：用户点了停止，界面显示「已停止」，状态却是 `done`——一次被掐断的查询和一次成功返回 20 行的查询长得一样。

三个缺陷有一个共同点：失败没有被建模。问题不在错误码的数量，而在失败的形状不确定——有时是一行数据，有时是一个空数组，有时是一个被改写的状态字符串。形状不确定，界面就只能猜；猜错的代价由用户承担。

所以 S6 要解决的是：如何让每一种失败都有名字、有类别、有界面、有是否可重试的结论，并且保证流一定以 `result` 或 `error` 收尾。多写几个错误码解决不了这件事。

2.2 实体与边界：错误表、通道错误、编排器、校正器、取消开关

S6 的中心是一张表：`contracts/errors.yaml`。它只回答三个问题：这个失败叫什么（`code` + `label`）、用户该看到什么（`class` + `message`）、能不能自动校正一次（`retryable`）。运行时的 `ErrorTable` 会严格加载它：少一个键、多一个键、`class` 不在 `classes` 里、`retryable` 不是布尔值、`max_attempts < 1`，都会抛 `ErrorTableError`。这一层不放过「差不多能用」的配置。

错误表的严格加载值得多看一眼。契约是运行时的事实源，一个拼错的键在开发期被放过，到了线上就变成一次 `INTERNAL_ERROR`；而 `INTERNAL_ERROR` 恰恰是最难排查的那一类，因为它只说明「别处坏了」。加载期就炸，至少能指到是哪一行。

第二个实体是 `ChannelError`。它是所有失败的唯一形状：`code` / `message` / `step` / `detail`。S4 的拒答、S5 的策略拒绝、数据库执行错误、校正失败、取消，全部变成它。`ChannelError.to_event()` 只保留 `contracts/sse.schema.json` 允许的 `detail` 键，避免把内部对象泄漏到线上。

第三个实体是 `AgentRunner`。它把上下文 8 阶段、生成、校验、校正、执行接成一条会发事件的流水线。它的三条不变式写在文件头：每一步都有真事、每一种失败都有名字、流必须以 `result` 或 `error` 收尾。任何未捕获异常都会被 `events()` 兜住，变成 `INTERNAL_ERROR`，绝不静默结束。

第四个实体是校正器。`propose_correction()` 只在错误可重试、有校正器、契约允许时才会动手；最多一次。校正后的 SQL 必须先回到 `validate_sql` 重新过 guard 与 policy，不能直接执行。

第五个实体是取消开关。`CancelRegistry` 维护 `request_id → CancelToken`，生命周期跟着 SSE 流走：流开始登记，结束注销。取消是一**个状态**，不是 `done`，也不是 `error`。

这张图回答：什么情况下会走 correct_sql？走完之后为什么必须重新校验？哪些失败是不可重试的？
读图：1 先看触发者（谁发起）→ 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边

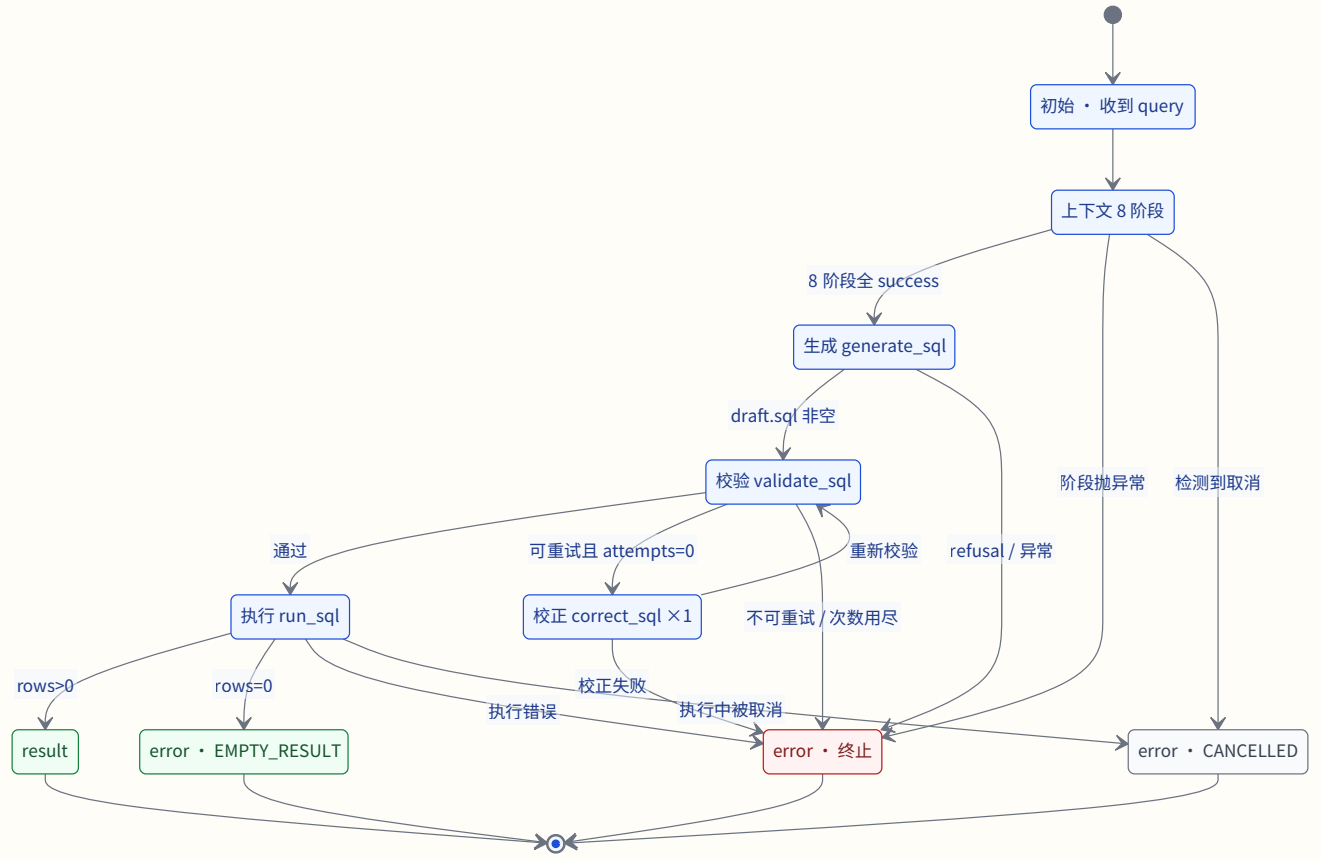


图 12 · S6 校正闭环状态机

图 12 把这条路径画成了状态机：`Gen → Val`，可重试且 `attempts=0` 才进 `Cor`，`Cor → Val` 重新校验；不可重试或次数用尽直接进 `Err`；`Run` 有行进 `Ok`，0 行进 `Empty`，执行错误进 `Err`，执行中被取消进 `Can`。注意 `Empty` 用的是绿色类，不是红色：空结果不是失败。

后文用四个缩写：`Gen` = `generate_sql`，`Val` = `validate_sql`，`Cor` = `correct_sql`，`Run` = `run_sql`；`Ok` / `Empty` / `Err` / `Can` 分别是成功、空结果、失败、取消。`attempts` 数的是已用掉的校正次数，首次校验失败时为 0。

把这台状态机当成一个证明来看：状态有限，正常路径的终止状态恰好四个——`Ok`、`Empty`、`Err`、`Can`，从 `Gen` 出发的每条路径都必须在其中一个停下。`Gen → Val` 之后，校验通过进 `Run`；不通过时，可重试且次数未用尽进 `Cor`，其余进 `Err`。`Cor → Val` 只有回边，没有直达 `Run` 的边，这一条边就保证了校正结果一定重新校验。`Run` 之后按「有行 / 零行 / 执行错误」三分，取消则在步骤边界与执行成功之后各检查一次。最后还有一层兜底：`events()` 把任何未捕获异常收成 `INTERNAL_ERROR`。四类终止状态加上这层兜底，就是「流一定以 `result` 或 `error` 收尾」的完整论证。

2.3 输入：内部码、异常、取消信号

S6 的输入有四类：

1. **S4 的拒答码**：`OUT_OF_RANGE`、`UNSUPPORTED_INTENT` 等。它们不是失败类别，是模型的判断。`channel_code_for_refusal()` 的规则只有一条：在错误码表里就是它自己，不在就是 `LLM_REFUSAL`；原码留在 `detail.reason_code`。
2. **S4 的静态校验违规码**：9 个 `SQL_*` 违规码，包括 `SQL_EMPTY`、`SQL_MULTIPLE_STATEMENTS`、`SQL_NOT_READ_ONLY`、`SQL_UNKNOWN_TABLE`、`SQL_CONSTANT_PROJECTION`、`SQL_MISSING_ORDER_BY`、`SQL_MISSING_LIMIT`、`SQL_UNKNOWN_COLUMN`、`SQL_SYNTAX`。
3. **S5 的执行错误**：`SQL_REJECTED`、`SQL_SYNTAX`、`SQL_UNKNOWN_TABLE`、`SQL_UNKNOWN_COLUMN`、`SQL_EXECUTION_ERROR`、`TIMEOUT`。
4. **取消信号**：`CancelToken.cancelled`，在每个步骤边界检查；客户端断开时 `_stream` 的 `finally` 也会调用 `CANCEL_REGISTRY.cancel()`。

另外还有两个兜底：编排器自身异常 → `INTERNAL_ERROR`；执行器返回了表里没有的码 → `INTERNAL_ERROR`，并把原码放进

`reason_code`。这条规则很重要：**失败必须有名字，不能因为码不认识就悄悄消失**。

四类输入是四种不同的信号，没有统一的错误对象形状。拒答码是模型的判断，违规码是静态检查的结论，执行错误是数据库的回应，取消是用户的动作。把它们压成同一种异常，就会丢掉「谁说的」这个信息；`ChannelError` 保留 `step`，正是为了记住「在哪一步、由谁给出」。

2.4 动作：映射 → 分类 → 校正一次 → 收尾

S6 的动作可以拆成五步。

第一步，映射成通道码。 所有内部码先过 `channel_code_for_refusal()` 或 `_execution_error()`。在表里就保留，不在表里就归到 `LLM_REFUSAL` 或 `INTERNAL_ERROR`。这样下游永远只需要面对 16 个码。

第二步，由 `class` 决定界面。 `failure` 是真失败：红框 + 失败步骤红叉。`empty` 是合法但没有数据：虚线空态卡，不标红，执行步骤保持绿色。`cancelled` 是用户取消：中性卡，正在跑的步骤标「已取消」，既不是 `done` 也不是失败。最容易写反的是把判断换成 `code`：同一个码换个 `class`，界面就该跟着换，这是 S6 里最容易错的一处。

第三步，由 `retryable` 决定要不要校正。 只有校验阶段、`retryable: true` 的错误才进 `correct_sql`，且最多一次（`correction.max_attempts: 1`）。`SQL_REJECTED` 永远不可重试：拿一条被安全策略拒掉的 SQL 去问模型「改一下」，等于给注入开了重试的口子。`TIMEOUT`、`SQL_EXECUTION_ERROR`、`SQL_NOT_READ_ONLY`、`SQL_MULTIPLE_STATEMENTS`、`SQL_EMPTY`、`LLM_REFUSAL`、`CANCELLED`、`INTERNAL_ERROR` 也都不可重试。

第四步，校正后重新校验。 `correction.revalidate: true`，`revalidate_with: [guard, policy]`。校正器给出的候选 SQL 先过 `validate_payload()` 的形状校验，再回到 `validate_sql` 跑 S4 的 guard 与 S5 的 policy。校正不是绕过安全检查的后门；如果校正结果带写操作或 UNION，策略照样拒绝，最后的错误仍然是 `SQL_REJECTED`。

第五步，收尾。 流必须以 `result` 或 `error` 收尾。有行 → `result`；0 行 → `EMPTY_RESULT`（`class: empty`）；执行失败 → 对应错误；取消 → `CANCELLED`（`class: cancelled`）。`AgentRunner._skip_correct()` 还会在流提前结束时补一条 `correct_sql: skipped`，避免条件步骤永远是一块灰的 pending。

图结构：12 个节点怎么落成 LangGraph

前面五步讲的是「发生了什么」，这一节讲「它长成什么形状」。参考实现没有手写 `while` 循环，而是用 `langgraph.graph.StateGraph` 编译成图。原因不是追新：图把「节点顺序」和「回边」变成可枚举的数据结构，测试可以直接拿编译后的节点集合去对契约。

图里只有 12 个节点，和 `contracts/steps.yaml` 逐字一致（`backend/app/agent/graph.py:428`）：

段	节点	边
上下文（8）	`extract_keywords` → … → `add_extra_context`	串联
生成（1）	`generate_sql`	上下文末尾 → 生成
校验（1）	`validate_sql`	生成 → 校验
分支（0）	条件边	通过 → `run_sql`；可重试且未超次 → `correct_sql`；否则 → `END`
校正（1）	`correct_sql`	回边 → `validate_sql`
执行（1）	`run_sql`	→ `END`

三条边决定行为：

1. `validate_sql` 是唯一的分叉点。它不是一个 `if`，而是 `add_conditional_edges` 的三选一：`run_sql`、`correct_sql`、`END`。
2. `correct_sql → validate_sql` 是回边。校正结果必须重新过 guard + policy，不允许「改完直接跑」。
3. `run_sql → END`。执行是最后一站，后面没有节点。

为什么强调「图而不是循环」？因为 SSE 的 12 条 `progress` 必须和图的 12 个节点是同一份事实。手写循环也能跑，但「哪些节点会出现、顺序是什么、条件步骤何时 `skipped`」会散在代码里；图把这些问题收敛成一个可断言的集合：`backend/tests/test_s6_langgraph.py:96` 直接断言编译后的节点数等于 12。

12 个节点怎么压成 10 步。 读者记 12 个节点太重，PRD 又要求「12 节点 / 10 步」。分组写在 `contracts/steps.yaml` 的 `stages` 里，机器可校验（`backend/tests/test_steps_stages.py` 守着不重不漏）：

表 3-2a · 12 节点 → 10 步

步	阶段	节点
1	抽取关键词	`extract_keywords`
2	召回	`recall_column`、`recall_value`、`recall_metric`
3	合并召回	`merge_retrieved_info`
4	过滤表	`filter_table`
5	过滤指标	`filter_metric`
6	补充上下文	`add_extra_context`
7	生成 SQL	`generate_sql`
8	校验 SQL	`validate_sql`
9	校正 SQL	`correct_sql`
10	执行 SQL	`run_sql`

「召回」一步里有三个节点，是因为列、值、指标走的是三个不同存储；它们在 SSE 里仍然各发一条 `progress`，但在教学和验收里算一步。

> 注意：`stage` 是读图分组，不是新事件。SSE 仍然是 **12 条** `progress`；谁要是按 10 条去等事件，会少收 2 条。

例题：被拒绝的 SQL 与被校正的 SQL

一条被拒绝的 SQL 走完整条链。 假设模型生成了一条带 `UNION` 的查询。生成阶段发出 `progress(generate_sql, running)`，成功后在步骤边界检查取消；校验阶段 `validate_sql` 先过 S4 的 guard，再问 S5 的 policy；策略返回 `SQL_FORBIDDEN_CONSTRUCT`，校验把通道码统一成 `SQL_REJECTED`，原因码放进 `detail.reason_code`。编排器查错误表：`class` 是 `failure`，`retryable` 是「否」，于是不进校正，直接发 `error` 事件并结束。前端的步骤条把 `validate_sql` 标红，`correct_sql` 显式发一条 `skipped`，卡片显示「SQL 被安全策略拒绝」。整条链上没有任何一步重试，也没有任何一步把原因码当成通道码。

一条可校正的 SQL 走完一次回边。 模型第一次给出语法错误的 SQL，`validate_sql` 失败，错误码 `SQL_SYNTAX`，`retryable` 为「是」，`attempts` 一直没动，于是进入 `correct_sql`。校正器返回候选 SQL，`propose_correction()` 先做形状校验，通过后把候选交回循环顶部；`validate_sql` 重新跑 guard 与 policy，通过后进入

`run_sql`，有行则发 `result`。步骤条上 `validate_sql` 先红后绿，`correct_sql` 从 running 到 success。回边只允许一次：第二次校验如果仍失败，次数已用尽，直接进 `Err`，不再问模型。

取消的实现有一个必须讲清楚的边界：编排器在**步骤边界**检查取消开关，不能中断已经发给 MySQL 的查询。取消发生在执行期间时，`ReadOnlyExecutor` 仍会把这条 SQL 跑完；编排器在执行返回后丢弃结果并发 `CANCELLED`。所以「取消」的语义是「用户说了停，系统不再推进」，不是「数据库查询被瞬间杀死」。

收尾那一段代码把「取消」写成了控制流，可以对照读：

```
            if execution.ok:
                if self._cancelled():
                    # 取消发生在执行期间：这次问数不算成功——用户已经说了
                    # 「停」。结果照常丢弃，换成一条 CANCELLED。
                    yield from self._skip_correct()
                    yield self._error_event(self._cancel_error())
                    return
```

注意 `if execution.ok` 这一层：取消的检查发生在执行成功之后、结果发出之前。执行失败走的是另一条分支，不会被这条取消逻辑改写。

> **关键问题**：零行算不算失败？答案取决于「查询是否成功」，不取决于「用户是否满意」。成功但零行是 `empty`；查询本身出错才是 `failure`。

常见误解：错误码越多，失败就管得越好

错误码是失败的名字，不是失败的分类。名字回答「发生了什么」，类别回答「界面该怎么表现」，步骤回答「在哪一步发生」，可重试回答「要不要再试一次」。四件事任意一件缺失，下游都只能猜。把码从十几个加到几十个，如果类别仍然是「全都红」，界面还是一句话；反过来，码不多但四件事齐备，用户至少知道该做什么。这一节所有的表与代码，都在补这四个维度，而不是在扩充码表。

2.5 输出：16 个错误码、3 个 class

表 3-3 · `contracts/errors.yaml` 的 16 个错误码（class / retryable 与契约逐项一致）

code	class	retryable	UI 表现
`SQL_REJECTED`	failure	否	

code	class	retryable	UI 表现
			红框 + `validate_sql` 红叉；显示「SQL 被安全策略拒绝」，`detail.reason_code` 给出细粒度原因；不可校正
`SQL_SYNTAX`	failure	是	红框 + `validate_sql` 红叉；若校正器给出候选 SQL，先 `correct_sql` 再重新校验；校正失败仍显示原错误
`SQL_UNKNOWN_COLUMN`	failure	是	红框 + `validate_sql` 红叉；可进入一次校正，校正后必须重新过 guard 与 policy
`EMPTY_RESULT`	empty	否	虚线空态卡，不标红；`run_sql` 保持绿色；文案「查询成功但没有数据」
`LLM_REFUSAL`	failure	否	红框 + `generate_sql` 红叉；`detail.reason_code` 保留 `OUT_OF_RANGE` 等原码
`TIMEOUT`	failure	否	红框 + `run_sql` 红叉；文案「SQL 执行超时，已中止」；不校正
`CANCELLED`	cancelled	否	中性卡；正在跑的步骤标「已取消」；状态是「已取消」，不是「完成」
`SQL_UNKNOWN_TABLE`	failure	是	红框 + `validate_sql` 红叉；可校正一次，重新校验
`SQL_NOT_READ_ONLY`	failure	否	红框 + `validate_sql` 红叉；不可校正，杜绝「改一下就能写」
`SQL_MULTIPLE_STATEMENTS`	failure	否	红框 + `validate_sql` 红叉；不可校正，避免多语句再次出现
`SQL_CONSTANT_PROJECTION`	failure	是	红框 + `validate_sql` 红叉；可校正一次，常用于「模型拿常量占位」
`SQL_MISSING_ORDER_BY`	failure	是	红框 + `validate_sql` 红叉；可校正一次，补排序
`SQL_MISSING_LIMIT`	failure	是	红框 + `validate_sql` 红叉；可校正一次，补 LIMIT
`SQL_EMPTY`	failure	否	红框 + `validate_sql` 红叉；生成结果是空串，校正没有意义
`SQL_EXECUTION_ERROR`	failure	否	红框 + `run_sql` 红叉；数据库执行出错，不校正
`INTERNAL_ERROR`	failure	否	红框；多为配置 / 连接 / 未预期异常；`detail.request_id` 便于排查

表 3-3 的步骤列写的是最常见产生点；`SQL_SYNTAX` / `SQL_UNKNOWN_TABLE` / `SQL_UNKNOWN_COLUMN` 也可能在执行期出现，此时步骤是 `run_sql`，且不再校正。

十六个码不是随手列的：它覆盖生成拒答、静态校验的九类违规、执行期的语法 / 表 / 列 / 超时 / 其他错误、空结果、取消与内部错误。三类 `class` 也不是装饰：`failure` 要红，`empty` 要中性，`cancelled` 要另一种中性。路线图点名的那几个码必须在表里，这是错误表加载期的检查；新增码可以加，但必须同时给出类别、是否可重试与文案，否则表加载不通过。

三个 class 的界面差异可以再压成一张小表：

表 3-4 · **class 决定界面，不是 code 决定界面**

class	步骤条	卡片	状态文案	典型码
`failure`	当前 running 步骤标 `×`	红框，可复制错误	失败	`SQL_REJECTED` 、 `TIMEOUT` 、 `INTERNAL_ERROR`
`empty`	执行步骤保持 `✓`	虚线空态卡，不标红	无数据	`EMPTY_RESULT`
`cancelled`	running 步骤标 `⊘`	中性卡	已取消	`CANCELLED`

把 `表 3-4` 读成一张真值表：三行互斥且穷尽，输入是 `class`，输出是步骤条标记、卡片样式、状态文案三件事。`failure` 的三个输出都是「坏」；`empty` 的三个输出都是「中性偏绿」；`cancelled` 的三个输出都是「中性」。判断依据只有 `class`，`code` 不参与。为什么？因为同一个码可以换类别——`EMPTY_RESULT` 今天是 `empty`，如果业务改口说「零行也算失败」，改的是契约里的 `class`，界面应当跟着变，而不是去改前端的一串 `if code === ...`。这就是把分类放在契约里、把渲染放在组件里的理由。

`empty` 不是失败，这条规则值得单独说清楚，因为它最容易被写反。`[]` 是合法语义：查询成功，库里就是没有这批数据。用户需要知道「没有」，不需要看到红色。把它归成 `failure` 会带来三个错误：步骤条把 `run_sql` 标成红叉，卡片给出一句道歉，用户会去改问题、重试，甚至怀疑权限。归类为 `empty` 之后，界面说「查询成功但没有数据」，步骤保持绿色，重试与否由用户决定。判断依据只有一个：执行成功且行数为零。

`error` 事件的形状必须符合 `contracts/sse.schema.json#/$defs/error`：

```
{
  "type": "error",
  "code": "EMPTY_RESULT",
  "class": "empty",
  "message": "查询执行成功，但没有符合条件的数据。",
  "step": "run_sql",
  "detail": {
    "sql": "SELECT ... LIMIT 1000",
    "rows": 0,
    "request_id": "8f0c..."
  }
}
```

`detail` 有白名单：`reason_code`、`reason`、`sql`、`original_sql`、`corrected_sql`、`attempts`、`rows`、`request_id` 这八个键以外，`ChannelError.to_event()` 会过滤掉。这样前端拿到的永远是契约允许的字段，不会因为后端多塞一个内部对象而崩。

2.6 验收：6 类失败、16 个码、校正只一次、取消不是 done

离线门禁来自 `Makefile` 的 `stage6`：

```
make stage6
```

它跑：

```
cd backend && uv run python -m app.scripts.failures --check
cd backend && uv run python -m app.scripts.failures --table
cd backend && uv run python -m app.scripts.failures --scenarios
cd backend && uv run pytest -q tests/agent
cd frontend && npm run test
cd frontend && npm run build
cd backend && uv run pytest -q
```

2026-10-29 在本工作树执行 `--check` 与 `--table`，输出（✓）：

```
== 1/2 失败场景（每类都必须演出它声称的结局） ==
  happy                期望 result               实际 result
  sql_rejected         期望 SQL_REJECTED         实际 SQL_REJECTED
  sql_syntax           期望 SQL_SYNTAX           实际 SQL_SYNTAX
  sql_unknown_column   期望 SQL_UNKNOWN_COLUMN   实际 SQL_UNKNOWN_COLUMN
  empty_result         期望 EMPTY_RESULT         实际 EMPTY_RESULT
  llm_refusal          期望 LLM_REFUSAL          实际 LLM_REFUSAL
  timeout              期望 TIMEOUT              实际 TIMEOUT
  slow                 期望 result               实际 result
  corrected            期望 result               实际 result
== 2/2 错误码表（路线图点名的 7 个码必须都在） ==
  错误码表：16 个码，路线图要求的 7 个全在
S6 PASS
```

标题的「6 类」指六个失败剧本（含空结果）；输出里的 9 行另有 happy / slow / corrected 三个非失败场景。

`--check` 不只比对错误码，还比对 `class` 与失败步骤。比如 `sql_rejected` 必须是 `SQL_REJECTED` / `failure` / `validate_sql`，`empty_result` 必须是 `EMPTY_RESULT` / `empty` / `run_sql`。如果只比错误码，`empty` 被写成 `failure` 也能过，界面就会把空结果标红——这正是 S6 要消灭的缺陷。

6 类失败的剧本由 `fixtures/failures/scenarios.yaml` 驱动。场景应用 `backend/app/scenarios/app.py` 起的是**同一个** `create_app`，只把模型与数据库换成夹具替身；上下文、提示词、静态校验、安全策略、校正闭环、SSE 序列化全是真代码，所以每个红叉都对应一次真实判定。

浏览器验收是另一条命令：

```
make stage6-ui
```

它自起场景应用（默认 8102）与前端（5275），跑正常路径、6 类失败各一个用例、以及取消。本工作树没有 Chrome / Edge，也没有执行它，所以标 ⑦。`stage6` 与 `stage6-ui` 的关系和 S5 的 `stage5` / `stage5-live` 一样：一个证明后端判定，一个证明界面表现，不能互相代替。

校正闭环的验收要点是三个负例：

1. 校正器返回一条带写操作的 SQL → 重新校验必须给 `SQL_REJECTED`，不能因为「这是校正结果」就放行。
2. 校正器返回一条 UNION → 同样必须拒绝，原因码 `SQL_FORBIDDEN_CONSTRUCT`。
3. 校正器拒答或产物不合契约 → 旧错误仍然是根因，不能把旧错误清掉、换成「校正成功」。

这三条在 `tests/agent/test_orchestrator.py` 里有对应负例。正例则是 `corrected` 场景：模型第一次给出语法错误的 SQL，校正器给出最小修正，重新校验通过，最后 `result` 成功；步骤条上 `validate_sql` 会先红后绿，`correct_sql` 从 running 到 success。

取消的验收要看三件事：状态文案是「已取消」而不是「已完成」；正在跑的步骤标 `⊘`，不是 `×`；结果表不出现。旧系统的 `status: isAbort ? "done" : "error"` 在这里被拆成了 `cancelled` 与 `failure` 两条路径。

2.7 常见故障

故障一：`error.code` 不在表里。 `build_error()` 会直接抛 `ErrorTableError`；`_execution_error()` 则把它降级成 `INTERNAL_ERROR`，并把原码放进 `reason_code`。两种处理看起来不一致，但原则一致：**码必须有名字**。前者是开发期错误，越早炸越好；后者是运行期，不能让一次未知码把整个流吞掉。

故障二：把 `SQL_REJECTED` 设成可重试。 这是最危险的一条。策略拒绝意味着 SQL 已经越过了安全边界；再问模型「改一下」，等于给注入一次重试机会。契约里 `SQL_REJECTED.retryable: false`，`propose_correction()` 也会在 `if not error.retryable` 处直接返回 `attempted=False`。两处都拦，不靠单点。

故障三：校正成功但没有重新校验。 `propose_correction()` 返回的 `sql` 只是**候选**，不是最终 SQL。编排器拿到候选后 `continue` 回到循环顶部，重新跑 `validate_sql`。如果跳过这一步，模型改出来的写操作就会直接执行；这正是旧系统 `correct_sql` 的三个问题之一。

故障四：校正失败后旧错误被清空。
`CorrectionProposal(ok=False)` 会带着原来的 `error` 返回；编排器用 `proposal.error or failure` 收尾。根因没变，错误就不该变。旧系统 `except` 之后 `raise`，但 `state["error"]` 还留着上一轮的旧错误，界面会同时显示「已校正」和「校验失败」；新系统只保留一个根因。

故障五：空结果被标红。 执行成功且 `rows == 0` 时，编排器发的是 `EMPTY_RESULT`，`class` 是 `empty`；前端 `App.tsx` 收到 `event.class === "empty"` 时只把状态设为 `empty`，不调用 `markRunning(..., "error")`。如果这里写错，步骤条会把 `run_sql` 标成红叉，用户会以为查询失败了。

故障六：前端自己发明错误码。 `App.tsx` 在流没有 terminal 事件时，会伪造一个 `STREAM_INCOMPLETE`；网络异常时伪造 `NETWORK_ERROR`。这两个码**不在** `contracts/errors.yaml` 里，它们是前端本地状态，不是后端通道码。`PseudoErrorCard` 也刻意不编造错误码，只显示「这一行不是查询结果」。前端硬编一个契约码，等于又抄了一份契约。

故障七：多 worker 下取消失效。 `CANCEL_REGISTRY` 是进程内状态。`compose.yaml` 的 backend 固定 `--workers 1`，为的是让「取消」打到同一个进程，省资源只是顺带。如果改成多 worker，取消请求可能落到另一个进程，`cancel()` 返回 `False`，用户点了停止但查询继续跑。

这条限制的根源在状态放在哪里。取消需要「同一个请求 id 打到同一个进程」，进程内字典是最直接的实现；要跨进程，就得引入共享存储或消息通道，那是另一个量级的工程。当前的选择是单 worker，把限制写在部署里，而不是假装可以水平扩展。

故障八：把一次校正成功当成「已修复」。 校正是否成功，取决于模型这一次给出的候选 SQL。同一条问题可能第一次 `SQL_UNKNOWN_TABLE`、第二次就成功。2026-11-09 在 5051 上对 8 条问题各跑 3 次：只有 5 次真正触发校正，5 次都成功；但**第一次单跑确实失败过**。所以验收必须写「N 次采样 + 成功率」，不能写「已修复」。可重跑的脚本是 `eval/live_probe.py`。

故障九：以为某个问题一定报某个码。 同一条危险问题可能被三层中的任意一层拦下：模型拒答（`LLM_REFUSAL`）、静态校验（`SQL_*`）、安全策略（`SQL_REJECTED`）。在 5051 上，UNION、

越界年份、城市粒度三条都被模型先拒答，根本没走到策略层。验收要断言「没有被执行 + 属于拒绝类」，而不是某个固定通道码；固定码只在**注入语料**里成立，因为那里绕过了模型。

2.8 证据

- `contracts/errors.yaml`：16 个码、3 个 class、`correction.max_attempts: 1`、`revalidate_with: [guard, policy]`（✓ 文件存在）。
- `backend/app/agent/errors.py:parse_error_table`：严格键校验、class 校验、retryable 布尔校验、路线图 7 码校验（✓）。
- `backend/app/agent/correct.py:propose_correction`：不可重试直接返回、最多一次、产物形状校验（✓）。
- `backend/app/agent/orchestrator.py:AgentRunner`：校验 → 校正 → 重新校验 → 执行；取消在步骤边界；流必收尾（✓）。
- `backend/app/agent/cancel.py:CancelRegistry`：`request_id → CancelToken`，流结束注销（✓）。
- `backend/app/scenarios/app.py`：生产/场景共用 `create_app`，只换模型与数据库（✓）。
- `make stage6` 的 `--check` / `--table` 输出（✓ 2026-10-29 本工作树）。
- `make stage6-ui`：6 类失败 + 取消的浏览器用例（? 本工作树无 Chrome / Edge）。

2.9 检查清单

- [ ] 我能说出 `failure` / `empty` / `cancelled` 三种界面差在哪里。
- [ ] 我能背出 16 个错误码，并说出哪些是可重试的。
- [ ] 我知道 `SQL_REJECTED` 永远不可重试，以及为什么。
- [ ] 我知道校正最多一次，且校正后必须重新过 guard 与 policy。
- [ ] 我知道执行阶段的失败**不**进入校正。
- [ ] 我知道空结果不是失败，`run_sql` 保持绿色。
- [ ] 我知道取消是 `cancelled`，不是 `done`，也不是 `failure`。
- [ ] 我知道取消只在步骤边界生效，不能中断已经发出的数据库查询。

3. S7 · 前端收口

3.1 问题：S6 留下了什么风险

S6 让后端有了名字，但名字要到用户眼前才算数。S6 结束时，后端能发 `progress` / `result` / `error`，错误带 `code` / `class` / `step`；但前端仍可能把它压扁成三种老问题：

第一，**前端硬编码步骤**。旧前端写死 12 个节点，`correct_sql` 永远灰着；后端新增一个步骤，前端不认，界面就少一块。S6 的 `skipped` 事件如果没有对应的渲染，条件步骤会永远停在灰色 pending。

第二，**未知事件被当成失败**。旧 `App.tsx` 的兜底分支把任何未知 `type` 当成 `error`；后端将来加一个 `type:"meta"`，用户就会看到一次莫名的红框，而且 `message` 还是 `undefined`。契约长了一条腿，不该变成一次报错。

第三，**类型被悄悄改写**。旧系统 `json.dumps(..., default=str)` 把 `Decimal` 降级成字符串，同一行里出现 `{"销量": "10", "销售额": 62190.0}`。屏幕上看不出差别，复制出去才是 `"10"`，下游做数值计算会踩坑。审计把这条记成 B4。

第三个问题最隐蔽：类型被改写之后，界面完全正常。用户看不出 `"10"` 与 `10` 的差别，只有把结果复制到表格里做求和，才会发现一列数字里混着字符串。这类缺陷不制造红色，也不制造空态，它制造的是「看起来对」。

所以 S7 的问题：**如何让后端的契约直接变成前端的类型、步骤、错误文案和界面状态，而不是靠人再抄一遍？** 换句话说，S6 解决的是「失败有没有名字」，S7 解决的是「名字有没有被正确地翻译成界面」。

3.2 实体与边界：契约、生成物、组件、解析器

S7 的实体分三层。

事实源层：`contracts/sse.schema.json`、`contracts/steps.yaml`、`contracts/errors.yaml`。它们是唯一事实源；前端不硬编码事件字段、步骤列表、错误码含义。

生成物层：`frontend/src/generated/` 下三个文件——`sse.schema.json`、`sse.ts`、`contract.json`。它们由 `backend/app/scripts/contracts_gen.py` 从契约生成，文件头明确写着「不要手改」。`sse.ts` 生成判别联合 `AgentEvent = ProgressEvent | ResultEvent | ErrorEvent`，以及从联合派生的别名

`ProgressStatus`、`ErrorClass`、`ColumnType`、`CellValue`。派生而不是重抄，是为了让「一个事实源」在类型层也成立。

运行时层：`frontend/src/lib/api.ts` 的 `SseParser` 与 `parseAgentEvent`，`frontend/src/lib/json-schema.ts` 的通用校验器，`frontend/src/App.tsx` 的事件 switch，`StepRail` / `ErrorCard` / `PseudoErrorCard` / `ResultTable` 四个组件。`/api/steps` 与 `/api/errors` 在启动时拉取，前端不写死步骤与错误文案。

界面栈：参考实现的两个前端都是 React 19。重建版（`frontend/`）用 Vite 6 + Tailwind 4（`@import "tailwindcss"` 与 `@tailwindcss/vite`）；线上原版（`deliverable/v2/original-frontend/`）用 Tailwind 3.4。版本号不同，但 PRD 要的是「React 19 + Tailwind」，不是某个小版本。教材只依赖三件事：聊天式提问、结果表格、错误卡；样式实现可以换，契约与组件边界不能换。

三层之间是单向依赖：事实源生成生成物，生成物约束运行时，运行时不再回头改事实源。手改生成物等于在运行时层伪造了一个事实源，下一次生成就会覆盖它，而 `--check` 会在覆盖之前先报错。正确顺序永远是「改契约 → 重新生成 → 再跑门禁」。

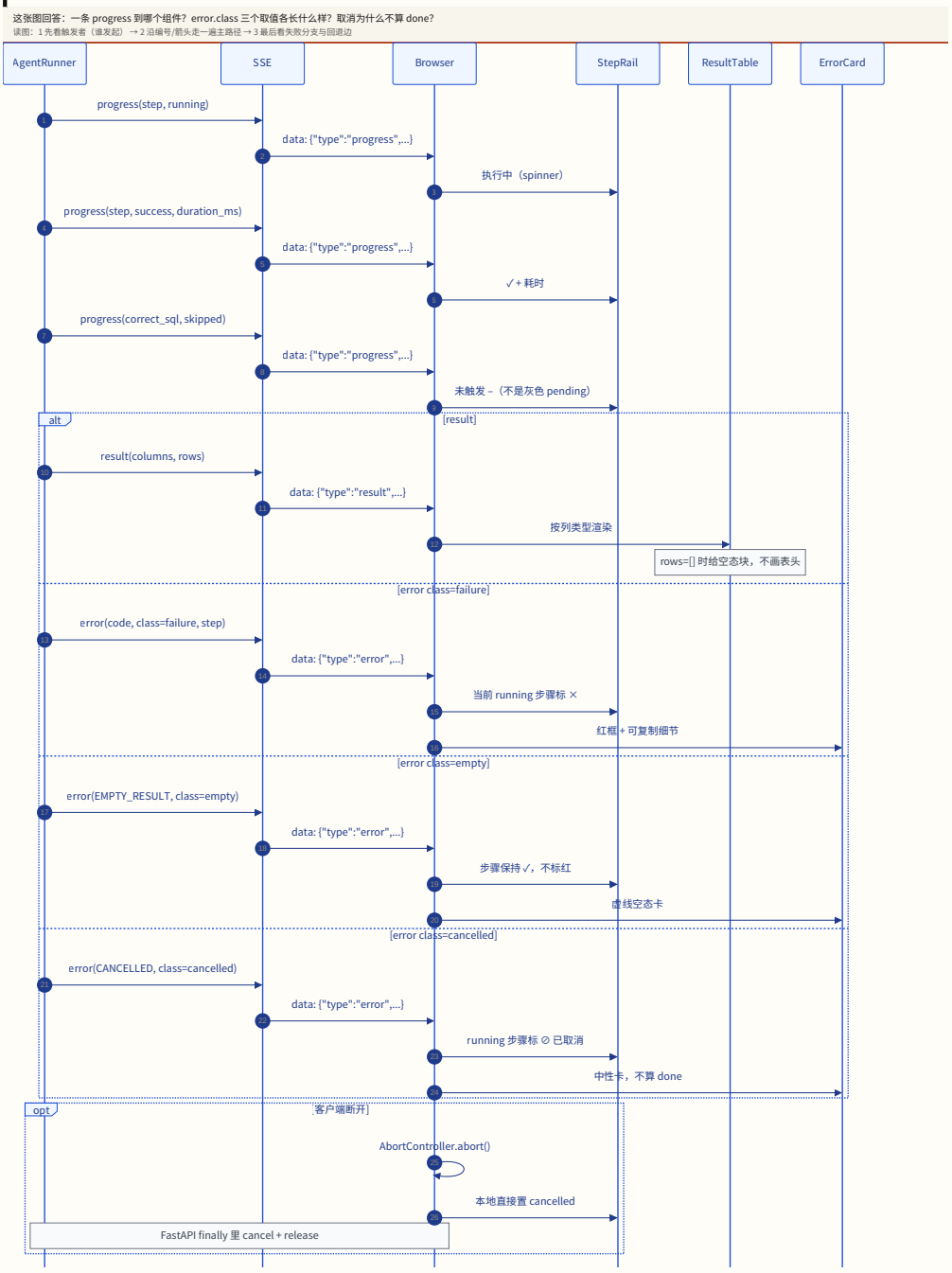


图 13 · S7 SSE 事件序列与前端消费

图 13 把「事件 → 组件」画得很清楚：`progress` 进 `StepRail`，`result` 进 `ResultTable`，`error` 按 `class` 分到红框 / 空态 / 中性卡。`correct_sql` 的 `skipped` 也在图里：它是「未触发 –」，不是灰色 pending。

这张图回答：契约里到底有哪些对象？每个对象带哪些字段？ValueDomain 的 kind 有哪几种？ColumnMetric 为什么是派生表？
读图：1 先看实体与 grain（一行代表什么）→ 2 再看主键/外键与关系 → 3 最后看约束、值域与口径

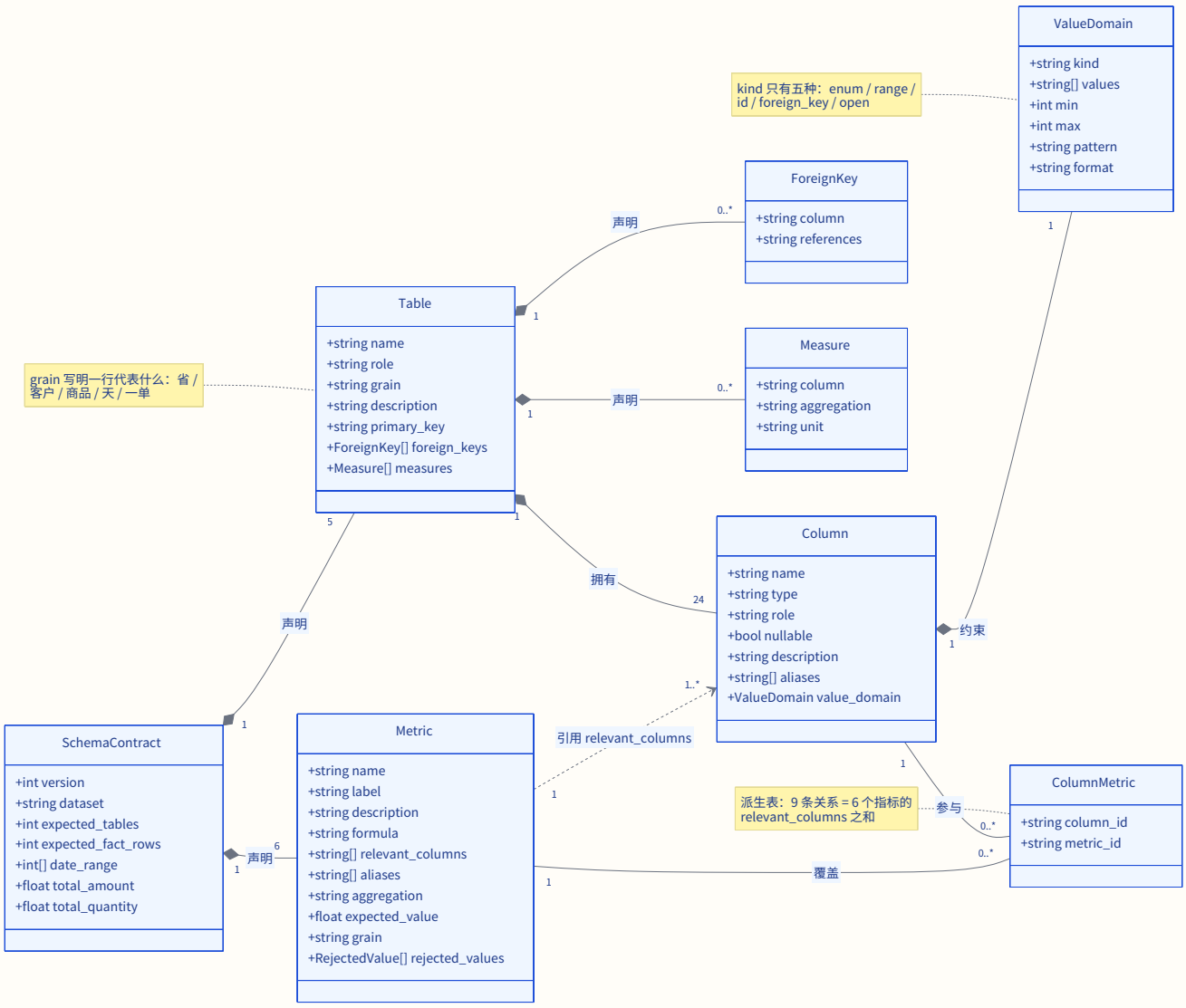


图 15 · 契约模型（表/列/指标/值域）

图 15 解释了 S7 生成物的底气：契约是有结构、有计数、有类型约束的模型，不只是给人看的文档。`SchemaContract` 声明 5 张表、6 个指标；`Table` 拥有 24 个 `Column`；每个 `Column` 有 `ValueDomain`。S7 的 `sse.schema.json` 和它一样，是可以被机器校验的结构，不是一段散文。

3.3 输入：SSE 字节流 + 契约 + 生成物

S7 的输入是网络上的字节流，不是后端内存里的事件对象。这一点决定了前端必须自己校验：代理可能改写、后端可能回滚到旧版本、前端可能连上别的服务。旧系统就是在这里崩的——它信任了「后端会发对」，然后把未知类型当失败。

SSE 的帧以空行分隔，帧内每行是 `field: value`；前端只取 `data:` 行拼成 JSON，event 行和以 `:` 开头的心跳行不参与。

一个合法的 SSE 帧长这样：

```
data: {"type":"progress","step":"run_sql","status":"running","duration_ms":null}
```

前端按空行切帧，接受 `\n\n` 与 `\r\n\r\n`；多条 `data:` 行按规范用 `\n` 拼接；跨 chunk 的半帧留在 buffer；流结束时 flush 尾帧。这四点看起来是解析细节，实际上每一点都对应一种丢帧方式：只认一种换行、跨 chunk 丢帧、多行截断、尾帧丢失。`SseParser` 把这四点都修了。

把解析器想成一个只有两个字段的状态机：`buffer` 与「当前帧是否完整」。输入是任意切分的字节块，输出是若干个完整帧。四条规则对应四个转移：

- **换行**：帧分隔符接受 `\n\n` 与 `\r\n\r\n` 两种，一条正则同时覆盖。只认一种换行的实现，在另一种代理后面会把整条流当成一帧。
- **跨 chunk**：每次 `feed()` 把新块拼到 `buffer` 尾部，按分隔符切分，最后一段可能是不完整的帧，留在 `buffer` 里等下一次。切完就丢 buffer 的实现会丢帧。
- **多 data 行**：一帧里可以有多条 `data:` 行，规范要求用换行拼接。只取第一行的实现会截断 JSON。
- **尾帧**：流结束时 buffer 里可能还剩最后一帧，没有空行结尾。`flush()` 负责把它交出去；不 flush 就会丢掉最后一条事件，而最后一条往往正是 `result` 或 `error`。

真实实现只有两个方法：

```
  feed(chunk: string): void {
    this.buffer += chunk;
    const frames = this.buffer.split(/\r?\n\r?\n/);
    this.buffer = frames.pop() ?? "";
    for (const frame of frames) this.dispatch(frame);
  }

  flush(): void {
    const rest = this.buffer.trim();
    this.buffer = "";
    if (rest) this.dispatch(rest);
  }
```

`feed()` 里的 `pop()` 就是「半帧留在 buffer」：切出来的最后一段永远不派发，等下一次拼接。`flush()` 是尾帧的唯一出口，调用方在流结束时必须显式调用它。

解析出来以后，`parseAgentEvent()` 先看 `type` 是否在 `EVENT_TYPES` 里。不在，返回 `unknown_type`，调用 `onUnknownType` 记一笔，**忽略**，不报错。在，则用 `sse.schema.json` 做运行时校验；字段不合法返回 `invalid`，抛 `PROTOCOL_ERROR`。三种结局，不是两种：合法、未知类型、契约破坏。这是 F3 的落点。

3.4 动作：校验 → 状态迁移 → 渲染

`App.tsx` 的 `ask()` 是一个显式状态机。开始请求时先把状态设为 `running`，清空 `states`、`result`、`error`；收到事件后按 `type` 分派：

- `progress`：先确认 `event.step` 在 `/api/steps` 返回的步骤列表里；不在就抛 `UNKNOWN_STEP`。在就更新 `states[step]`，`StepRail` 据此渲染 running / success / error / skipped。
- `result`：`terminal = true`，设置结果，状态 `done`。
- `error`：`terminal = true`，设置错误；`class === "empty"` → 状态 `empty`，不动步骤；`class === "cancelled"` → 状态 `cancelled`，把 running 步骤标 `cancelled`；其余 → 状态 `error`，把 running 步骤标 `error`，并把 `event.step` 也标红。

流结束后如果 `terminal` 仍是 `false`，前端伪造 `STREAM_INCOMPLETE`，状态 `error`。这是契约要求的兜底：静默结束等于失败，不能当成成功。捕获到 `AbortError` 时，状态是 `cancelled`，不是 `done`；这是旧系统 `isAbort ? "done" : "error"` 的直接修正。

例题：一条 progress 事件从字节到 DOM

拿一条 `progress` 走一遍，从字节到像素：

1. 网络层交来一段字符串，可能是半个帧，也可能是一帧半。`feed()` 把它接到 `buffer` 后面。
2. 切分后，完整帧进入 `dispatch()`；半帧留在 `buffer`。
3. `dispatch()` 只取 `data:` 开头的行，去掉前缀与一个可选空格，用换行拼接。
4. `JSON.parse` 得到对象；失败抛 `PROTOCOL_ERROR`，这次问数结束。
5. `parseAgentEvent()` 先看 `type`。不在 `EVENT_TYPES` 里，交给 `onUnknownType` 记一笔并忽略。

6. 在，则用 `sse.schema.json` 校验字段；不合法抛 `PROTOCOL_ERROR`，错误信息带路径。
7. 合法事件回到 `ask()` 的 switch：`progress` 先查步骤白名单，不在抛 `UNKNOWN_STEP`，在就更新 `states[step]`。
8. `StepRail` 按状态渲染：running 是 spinner，success 是 √ 加耗时，skipped 是「未触发 –」。

这条路径上有三处「停下来」：JSON 不合法、字段不合契约、步骤不在白名单。三处都抛错，因为契约已经被破坏，继续渲染只会把错误藏得更深。未知事件类型是唯一的例外，它选择继续。

状态迁移可以写成表：

表 3-5　· 前端运行状态迁移

当前状态	触发	下一状态	步骤条变化
`idle`	提交问题	`running`	全部 pending，随后按 `progress` 推进
`running`	`progress(status=running)`	`running`	该步骤 spinner
`running`	`progress(status=success)`	`running`	该步骤 √ + 耗时
`running`	`progress(status=skipped)`	`running`	该步骤「未触发 –」
`running`	`result`	`done`	已完成步骤保持绿色
`running`	`error(class=failure)`	`error`	当前 running 步骤 ×，`event.step` 也 ×
`running`	`error(class=empty)`	`empty`	执行步骤保持 √，不标红
`running`	`error(class=cancelled)`	`cancelled`	当前 running 步骤 ⊘
`running`	用户点停止 / `AbortError`	`cancelled`	当前 running 步骤 ⊘
`running`	流结束但没有 terminal	`error`	当前 running 步骤 ×（`STREAM_INCOMPLETE`）
任意	新的提交	`running`	清空上一次的状态

`StepRail` 的 `STATUS_TEXT` 有六个值：`pending` 等待、`running` 执行中、`success` 完成、`error` 失败、`skipped` 未触发、`cancelled` 已取消。六个值对应六种标记：数字、spinner、√、×、–、⊘。`skipped` 不是 `pending`：pending 是「还没轮到」，skipped 是「这一步这次不会跑」；`correct_sql` 在没有可重试错误时必须显式 `skipped`，否则它会永远灰着。

六个状态值不是装饰，它们把「没轮到」「正在跑」「跑完了」「没触发」「失败」「取消」分成互不相同的六件事。最容易混淆的是 `pending` 与 `skipped`：前者是时间上的「还没到」，后者是逻辑上的

「这次不会到」。条件步骤如果没有显式的 `skipped`，就会永远停在 `pending`，用户无法区分「系统卡住了」与「这一步本来就不需要」。

`ErrorCard` 的三种长相由 `class` 决定，不是由 `code` 决定：

- `failure`：红框，标题来自 `/api/errors` 的 `label`（`SQL_REJECTED` → 「SQL 被安全策略拒绝」），显示错误码、失败步骤中文名、原因码、消息；有 `original_sql` / `corrected_sql` 时给可展开的 SQL 细节；有 `attempts > 0` 时显示「已校正 N 次」。整段文本可一键复制。
- `empty`：虚线空态卡，标题「查询成功但没有数据」，不标红；步骤条保持绿色。
- `cancelled`：中性卡，标题「用户取消」，状态「已取消」；不算 done，也不算失败。

`PseudoErrorCard` 是第四种，但它不是错误码驱动的：它是前端止损。判据刻意保守——单行 + 单列 + 列名像错误文案（`error|错误|失败|异常|message|状态|reason|无法|不支持`）+ 值是文本，才判定为伪数据行。宁可漏判，也不要把一张真的 1 行 1 列表误判成失败。它不编造错误码，只显示「这一行不是查询结果，是失败信息」。

3.5 输出：三种 SSE 事件与类型保真

`progress` 的样例：

```
data: {"type":"progress","step":"run_sql","status":"running","duration_ms":null}
```

`result` 的样例：

```
data: {"type":"result","data":{"columns":[{"name":"地区","type":"string"},{"name":"销售
额","type":"number"}],"rows":[["华北",128400.0]]}}
```

`error` 的样例（`class: empty`）：

```
data: {"type":"error","code":"EMPTY_RESULT","class":"empty","message":"查询执行成功，但没有符合条件的数
据。","step":"run_sql","detail":{"sql":"SELECT ... LIMIT 1000","rows":0,"request_id":"8f0c..."}}
```

三条事件都受 `contracts/sse.schema.json` 约束：`progress.duration_ms` 在 running / skipped 时必须为 `null`，在 success / error 时必须是数字；`result.data.rows` 是二维数组，不是对象数组；`error.detail` 只允许八个键。前端收到的如果不符合，`parseAgentEvent()` 会给带路径的错误，例如 `$.duration_ms 缺少必填字段`，而不是一句「解析失败」。

`result.data.rows` 是二维数组而不是对象数组，这个选择让「列顺序」由 `columns` 单独声明，行只负责值。对象数组会把列名重复写进每一行，列顺序还要靠键的插入顺序猜；二维数组把「有哪些列」与「每行有哪些值」分开，校验器可以分别检查列数、类型与值。

类型保真的规则写在 `backend/app/agent/values.py`：

表 3-6 · 单元格值的归一化

数据库类型	归一化后	JSON 结果	列类型
`DECIMAL("10")`	`int`	`10`	`number`
`DECIMAL("10.5")`	`float`	`10.5`	`number`
`DATE` / `DATETIME`	ISO 8601 字符串	`"2025-03-01"`	`date`
未知类型	抛 `TypeError`	变成 `INTERNAL_ERROR`	—

`Decimal("10")` 走 `int` 而不是 `10.0`：`SUM(INT)` 在 MySQL 里是 DECIMAL，但用户心里的「10 件」应该显示成 `10`。`Decimal("10")` 走 `int` 是刻意的取舍；反过来，真正带小数的金额走 `float`，会引入浮点误差，这是 JSON 数字类型的固有限制。归一化模块选择「整数走整数、非整数走浮点」，并在列类型上统一标成 `number`，让前端按数字渲染。

`_sse()` 也刻意不传 `default=str`，万一还有漏网的类型，就让它炸成 `INTERNAL_ERROR`，不要悄悄把数字改成字符串。`default=str` 会把任何没归一化的类型悄悄变成字符串，屏幕上看起来正常，复制出去才发现 `"10"`。归一化模块遇到未知类型直接抛 `TypeError`，调用方把它变成 `INTERNAL_ERROR`，用户看到一次诚实的失败，而不是一份被改过类型的数据。

前端 `ResultTable` 再按契约列类型渲染：`number` 走 `Intl.NumberFormat`，整列 NULL 加「无数据」标记，数字样式的字符串加类型不匹配标记，复制输出 JSON 而不是 TSV。

3.6 验收：生成物新鲜、夹具逐字对账、13 条 F 缺陷

离线门禁来自 `Makefile` 的 `stage7`：

```
make stage7
```

它跑：

```
cd backend && uv run python -m app.scripts.contracts_gen --check
cd backend && uv run python -m app.scripts.record_sse --check
cd backend && uv run pytest -q tests/frontend
cd frontend && npm run test
cd frontend && npm run build
cd backend && uv run pytest -q
```

2026-10-29 在本工作树执行前两条，输出（✓）：

```
S7 CONTRACT GEN PASS（3 个文件与契约一致）
S7 SSE RECORD PASS（14 个夹具与真实输出一致）
```

`contracts_gen --check` 检查 `frontend/src/generated/` 三个文件是否与 `contracts/sse.schema.json` 一致。手改生成物会立刻变红：`--check` 不重新生成覆盖，而是生成到内存、逐字比对，不一致就报错。`record_sse --check` 重新跑一遍场景应用，把后端吐出的字节流与 `fixtures/sse/*.json` 逐字比对，只归一化 `duration_ms` 与 `request_id` 两处。14 条夹具里 10 条是录制的真实 SSE，4 条是手写的（`pseudo_row` / `unknown_event` / `empty_rows` / `null_column`），因为新后端已经产生不了伪 SQL 行、未知事件、空数组和全 NULL 列。所以 `--check` 的「一致」指与夹具逐字一致，不指这 4 条由后端重新生成。

`contracts_gen --check` 与「重新生成再覆盖」的区别也值得说清楚：前者生成到内存、逐字比对，不一致就报错；后者会掩盖漂移，让手改生成物的错误看起来像一次正常更新。门禁要的是发现漂移，不是消除证据。

前端契约测试 `pytest -q tests/frontend` 计划 45 条，覆盖生成器 / 夹具 / 类型保真；`npm run test` 计划 40 条，覆盖 SSE 解析与结果表语义；`npm run build` 包含 `tsc --noEmit`。这三条本工作树没有执行，标 ⚠（文档声称 45 / 40 / 零错误）。

浏览器验收是：

```
make stage7-ui
```

它自起回放服务器（默认 8103）与隔离前端（5276），跑 14 条断言，覆盖 F1–F13（F11 拆成 5xx / 422 两例）。端口冲突时：

```
API_PORT=8203 WEB_PORT=5376 make stage7-ui
```

本工作树没有 Chrome / Edge，标 ⓘ。回放而不是真后端的原因很实际：有些界面行为真系统产生不了——旧系统的伪 SQL 行、契约之外的事件、`data:[]` 空数组。回放服务器按生产协议提供 `/api/health`、`/api/steps`、`/api/errors`、`/api/query`、`/api/cancel`，问题写成「场景:<夹具 id> <真实问题>」；`场景:http_500` / `场景:http_422` 直接返回对应 HTTP 状态，用来验收 F11。

表 3-7 · 几条代表性的 F 缺陷与修法

编号	旧缺陷	新行为	证据
F1	取消被算成「完成」	状态「已取消」，且不出现结果表	`App.tsx:stop` / `isAbortError`
F3	未知事件类型被当成错误	忽略 + 提示条，界面正常出结果	`lib/api.ts:parseAgentEvent`
F4	空数组导致表格消失	空态块，文案「没有数据」，不画表头	`ResultTable`
F5	伪 SQL 行被当成功	`PseudoErrorCard` 止损卡，不画表	`lib/result.ts:looksLikePseudoError`
F9	条件步骤永远灰着	`skipped` → 「未触发 –」	`StepRail`
F11	4xx/5xx 一律同一句话	422「问题格式不对」、5xx「服务异常」	`lib/api.ts:httpError`
F13	复制把 `"10"` 原样复制	数字就是数字，复制输出 JSON	`values.py` + `buildCopyPayload`

F13 是这条链的终点：复制出去的 `"10"` 不是显示问题，是数据问题。修法不在剪贴板，而在源头——归一化保证数字是数字，列类型保证前端按数字渲染，复制输出 JSON 保证下游拿到的是类型而不是字符串。三处任何一处退回旧写法，缺陷都会复发。

3.7 常见故障

故障一：改了契约但忘了重新生成。 症状是 `make stage7` 在 `contracts_gen --check` 处变红，提示运行 `cd backend && uv run python -m app.scripts.contracts_gen`。不要手改 `frontend/src/generated/`；手改的下一秒就会被 `--check` 抓住。改契约 → 重新生成 → 再跑 `make stage7`，这是唯一顺序。

故障二：SSE 夹具对不上。 `record_sse --check` 会逐字比对，只归一化 `duration_ms` 与 `request_id`。如果后端改了事件顺序、字段名、默认文案，或者前端场景前缀解析变了，夹具就会红。正确做法

是先判断哪边是事实源：契约变了就重新录制；代码坏了就修代码。改夹具让它过只是把证据擦掉——夹具是证据，不是橡皮擦。

故障三：未知步骤导致整次失败。 `App.tsx` 对 `progress.step` 做白名单校验，不在 `/api/steps` 里就抛 `UNKNOWN_STEP`。这是刻意的：步骤 id 是契约的一部分，后端发了一个前端不认识的步骤，说明契约不一致，应该报错而不是悄悄忽略。注意这和「未知事件类型」不同：事件类型未知可以忽略（契约长了一条腿），步骤 id 未知是契约内部矛盾。

两种处理的差别在于「谁应该被修」。未知事件类型是后端向前扩展，前端必须容忍；未知步骤 id 是两端版本不一致，容忍只会把错误藏起来。一条流里如果步骤 id 出错，后面的 `states` 更新会落在不存在的键上，界面看起来「什么都没发生」，这比一次明确的报错更难查。

故障四：`duration_ms` 在 running 时不是 null。 `sse.schema.json` 用 `allOf` + `if/then/else` 约束：success / error 必须是数字，running / skipped 必须是 null。前端校验器支持 `if/then/else`，所以这条会在运行时被抓。后端 `_progress()` 默认 `duration_ms=None`，success / error 才传数字；改事件构造时不要顺手给 running 也塞一个 0。

故障五：流没有 terminal 事件。 前端会伪造 `STREAM_INCOMPLETE`，状态 `error`。这个码不在契约里，是前端本地状态；它不应该出现在后端事件里，也不应该被当成契约码去 `/api/errors` 查文案。它存在的意义是「静默结束等于失败」，而不是新增一种业务失败。

故障六：Decimal 又变成字符串。 检查 `_sse()` 是否被加上了 `default=str`，检查 `to_result_payload()` 是否走了 `normalize_cell()`，检查 `ResultTable` 是否对 `number` 列做了类型校验。三处任何一处退回旧写法，F13 就会复发；前端会在数字列上标出「数字样式的字符串」，让问题可见，而不是安安静静地显示。

故障七：空结果被画成空表。 `EMPTY_RESULT` 的 `class` 是 `empty`，前端状态是 `empty`；`ResultTable` 对 `rows.length === 0` 走空态块，不画表头。后端把空结果发成 `result` 且 `rows: []` 时前端也有兜底（F4 的 `empty_rows` 夹具），但正确路径仍然是 `EMPTY_RESULT` 事件。

3.8 证据

- `contracts/sse.schema.json`：3 种事件、`progress` 的 `duration_ms` 条件约束、`result` 的二维 rows、`error` 的 3 个 class（✓）。

- `contracts/steps.yaml`：12 步，`correct_sql` 标 `conditional: true`（✓）。
- `contracts/errors.yaml`：16 码 / 3 class（✓）。
- `frontend/src/generated/sse.ts`、`sse.schema.json`、`contract.json`：机器生成物（✓ 文件存在；`contracts_gen --check` 本机 PASS）。
- `frontend/src/lib/api.ts`：`SseParser`、`parseAgentEvent`、`httpError`（✓）。
- `frontend/src/lib/json-schema.ts`：通用校验器，支持 `$ref` / `oneOf` / `allOf` / `if-then-else`（✓）。
- `frontend/src/App.tsx`：事件 switch、状态迁移、取消、`STREAM_INCOMPLETE`（✓）。
- `frontend/src/components/StepRail.tsx`、`ErrorCard.tsx`、`PseudoErrorCard.tsx`、`ResultTable.tsx`（✓）。
- `fixtures/sse/` 14 条夹具（✓ 文件存在；`record_sse --check` 本机 PASS）。
- `make stage7-ui` 的 14 条断言（? 本工作树无 Chrome / Edge）。

3.9 检查清单

- [ ] 我能说出 `frontend/src/generated/` 三个文件分别是什么，以及为什么不能手改。
- [ ] 我能背出 SSE 的三种事件，并各写一条合法样例。
- [ ] 我知道未知事件类型应该忽略，未知步骤 id 应该报错，为什么不同。
- [ ] 我知道 `skipped` 和 `pending` 的区别，以及 `correct_sql` 为什么必须显式 skipped。
- [ ] 我知道 `ErrorCard` 的三种长相由 `class` 决定，不是由 `code` 决定。
- [ ] 我知道 `DECIMAL("10")` 为什么是 `10` 而不是 `"10"` 或 `10.0`。
- [ ] 我知道 `_sse()` 为什么不传 `default=str`。
- [ ] 我知道 `STREAM_INCOMPLETE` / `NETWORK_ERROR` 是前端本地码，不在契约里。

4. S8 · 生产化（边界与下一步）

4.1 问题：S7 留下了什么风险

S7 结束时，系统在开发机上「能跑、能失败、能展示」。但「能跑」和「敢用」之间还隔着一层：换一台机器、换一个网络、换一个用户，系统还成不成立?

S7 留下的风险至少有六个:

1. **部署没有端到端证据**。`compose.yaml` 写得很完整，但本工作树没有 Docker socket 权限，`make docker-up` 与 `make docker-smoke` 都没跑过。拓扑是「设计上应该成立」，不是「本机已经成立」。
2. **没有身份与传输安全**。`/api/query` 没有鉴权，前端默认 `0.0.0.0:8080`，没有 TLS。它适合教学与内网演示，不适合直接暴露到公网。
3. **取消是进程内状态**。`CANCEL_REGISTRY` 只在一个进程里有效，backend 固定单 worker。这是正确的取舍，但也意味着它不是一个可水平扩展的服务。
4. **没有可观测性**。有 `/api/health`、`request_id`、Docker healthcheck、nginx access log，但没有 metrics、tracing、结构化日志聚合、告警。
5. **没有容量基线**。`default_limit: 1000`、`max_execution_time_ms: 5000`、LLM 超时 30 秒是保护阈值，不是容量结论。P50 / P95、并发数、连接池、慢查询分布都没有测。
6. **测试总数只有文档声称**。README 写「782 passed / 31 skipped」，`docs/SETUP-FROM-SCRATCH.md` §3 另一处写「665 passed / 31 skipped」；本工作树没有跑全量 pytest。这个数字只能标 ⓘ，不能当已验证事实。

这六条风险有一个共同形状：它们在开发机上都不显形。端口冲突、密码轮换、健康检查、SSE 缓冲，只有换一台机器、换一个网络、换一个用户才会暴露。S8 的工作因此是把「本机成立」改写成「条件成立」——每条结论都带上它依赖的条件。

所以 S8 要做的是把「本机可运行」变成「可交付、可观测、可恢复」，并把没证明的部分标出来。这不是再加一个功能。这三个词各自对应一类证据：可交付的证据是「另一台机器起得来」，可观测的证据是「出了问题能定位到哪一环」，可恢复的证据是「卷坏了、密码换了，能按顺序救回来」。S8 目前只完成了第一类的设计部分，后两类只有清单。

4.2 实体与边界：六服务、镜像、端口、卷

`compose.yaml` 定义六个服务，项目名 `shopkeeper-rebuild`：

服务	作用	关键依赖	健康检查
`mysql`	数仓 `dw` + Meta 库	无	`mysqladmin ping`
`elasticsearch`	取值索引	无	集群状态 green / yellow
`qdrant`	向量索引	无	`GET /readyz` 返回 200
`init`	建库、开只读账号、校验/构建 S2	三存储 healthy	无，退出码即结果
`backend`	FastAPI + uvicorn，单 worker	`init` completed	`GET /api/health`
`frontend`	nginx + 静态资源	`backend` healthy	`GET /healthz`

这张图回答：哪六个服务？谁依赖谁？数据落在哪？init 为什么和另外五个不一样？
读图：1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点

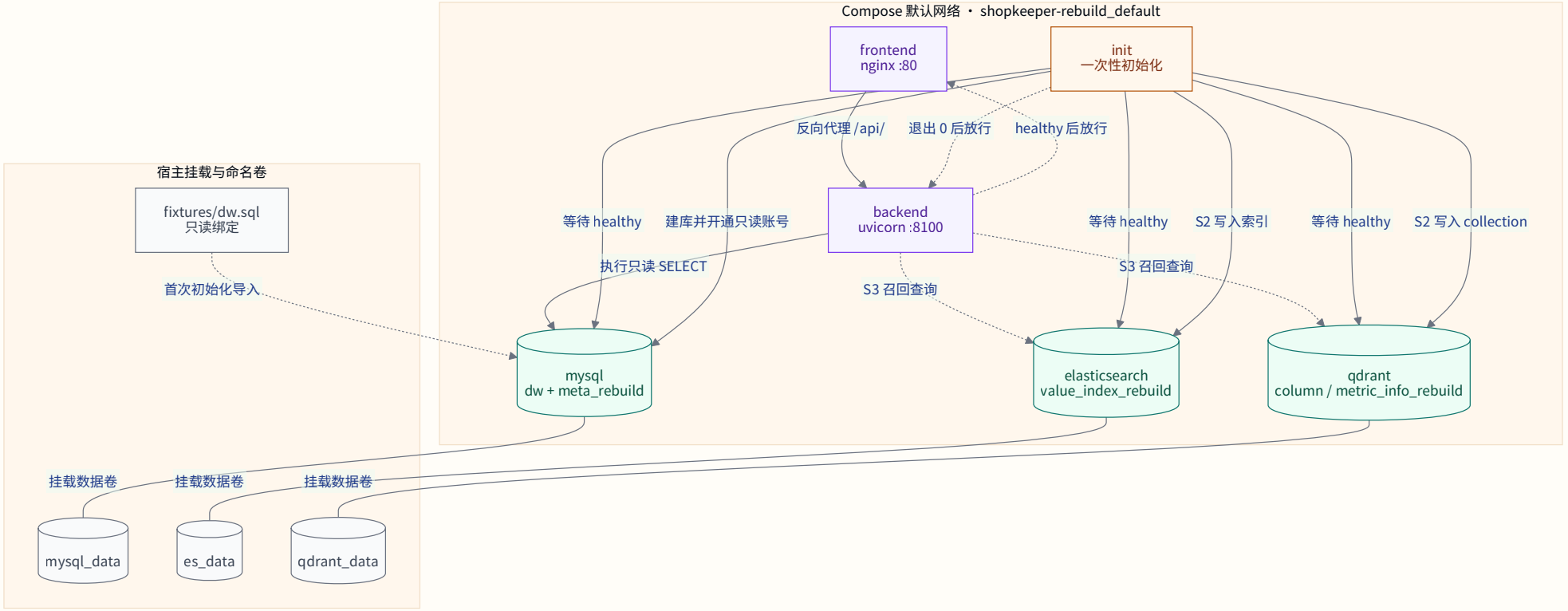


图 25 · Docker Compose 拓扑

图 25 的重点是三条依赖边，六个盒子只是背景：三存储 healthy → `init` 开始；`init` 退出 0 → `backend` 开始；`backend` healthy → `frontend` 开始。`depends_on` 写的是条件，不是「启动顺序建议」：`service_healthy` 会一直等健康检查变绿，`service_completed_successfully` 会等 `init` 以 0 退出。所以「栈起来了但页面 502」通常意味着这条链在某一环停住了，nginx 本身多半没问题。

把这三条边连起来，启动链是一张有向无环图：三存储是三个源点，`init` 是汇合点，`backend` 依赖 `init` 的退出码，`frontend` 依赖 `backend` 的健康状态。图上有两种不同性质的边：`service_healthy` 等的是「持续为真」的状态，`service_completed_successfully` 等的是「一次性事件的结果」。这两者不能互换。

`service_started` 为什么不够？它只说明容器进程起来了，不说明里面的服务能接受连接。MySQL 启动后还要初始化数据目录、建库、建账号；这段时间里端口可能已经监听，但查询会失败。如果 `init` 只等 `service_started`，它会在数据库还没准备好的时候尝试建表、开账号，结果要么失败退出，要么写下一半状态；而 `backend` 又依赖 `init` 的退出码，整条链会把一次「等得不够久」放大成「栈起不来」。健康检查等的是「服务真的可用」，不是「进程真的存在」。

`service_completed_successfully` 同理：它等的是 `init` 以 0 退出，而不是「`init` 启动过」。`init` 是一次性任务，没有长期健康状态可等；它的结果就是退出码。把条件改成 `service_started`，`backend` 可能在 `init` 还在建库时就启动，连上一个半成品。

镜像边界也值得说清楚。后端镜像是多阶段构建：`uv` 阶段装依赖，`python:3.12-slim` 运行阶段只带 `.venv`、`backend/app`、`contracts`、`fixtures`，不含测试；非 root，uid 10001；`uvicorn` 单 worker。前端镜像是 `node:20-alpine` 构建 → `nginx:1.27-alpine` 托管 `dist`；`/api/` 关闭 `proxy_buffering`，否则 SSE 会被 nginx 攒起来一次性发。`init` 与 `backend` 用同一个镜像、同一份源码，但环境变量不同：`init` 拿构建凭据与 root 密码，`backend` 只拿 `REBUILD_DW_RO_PASSWORD`。

共用镜像让「初始化用的代码」与「运行时用的代码」不可能版本不一致，也省了一次构建；代价是镜像里同时带着构建依赖与运行依赖，在一次性任务面前可以接受。

nginx 那一侧只有一行是关键：`/api/` 关闭 `proxy_buffering`。开着缓冲时，nginx 会把后端的 SSE 响应攒够一批再发，浏览器看到的是「一次性到达」；流式语义在代理这一层被悄悄取消，而后端与前端都以为一切正常。`gzip off` 同理：压缩需要攒数据，攒数据就破坏实时性。

4.3 输入：环境变量、镜像、端口、卷

S8 的输入主要是四类：

1. **环境变量**：`.env` 里的 `MYSQL_ROOT_PASSWORD`、`MYSQL_PASSWORD`、`REBUILD_DW_RO_PASSWORD`、镜像名、宿主机端口、可选的 LLM 三元组。`.env` 不入库；`.env.example` 里是 `change-me-*` 占位值。
2. **镜像**：MySQL 8.0、Elasticsearch 8.15.3、Qdrant v1.12.4、Python 3.12-slim、Node 20-alpine、nginx 1.27-alpine。Qdrant 必须 ⩾ v1.5.0，因为健康检查打的是 `/readyz`。
3. **端口**：前端 `0.0.0.0:8080 → 80`；后端、MySQL、ES、Qdrant 都只绑 `127.0.0.1`，分别是 `8100`、`3306`、`9200`、`6333`。只有前端是唯一对外入口。

4. **卷**：`mysql_data`、`es_data`、`qdrant_data` 三个命名卷；`fixtures/dw.sql` 以只读方式挂到 MySQL 初始化目录，首次启动导入。

端口收敛也遵循同一条原则：只有 `frontend` 绑 `0.0.0.0`，其余全部绑 `127.0.0.1`。后端、MySQL、ES、Qdrant 都需要被容器网络里的服务访问，但不需要被宿主机以外的机器访问。把「谁能访问」写成端口绑定，比写在文档里可靠。

还有一条容易忽略的边界：构建凭据与应用凭据分离。`mysql` 与 `init` 拿 `MYSQL_PASSWORD`（应用账号 `didilili`，在 `dw` 上有 ALL PRIVILEGES）和 root 密码；`backend` 只拿 `REBUILD_DW_RO_PASSWORD`，也就是 `dw_reader` 的密码。`didilili` 的 ALL PRIVILEGES 正是 S5 要消除的越权入口，不能因为「容器里方便」就把它给 backend。

构建凭据与运行凭据的分离，是 S5 的账号边界在部署层的投影。`init` 需要 root 与 `didilili`：它要建库、开只读账号、写元知识。`backend` 只需要 `REBUILD_DW_RO_PASSWORD`：它执行的每一条 SQL 都该走 `dw_reader`。如果为了「容器里方便」把 `MYSQL_PASSWORD` 也给 `backend`，S5 在代码层收窄的权限，会在环境变量层被重新放宽；一次策略漏判就又有了写库的能力。凭据的最小化不是运维偏好，是安全边界的一部分。

4.4 动作：生成 .env → 构建 → 等健康 → 验收

首次部署的命令来自 `README.md` 与 `docs/DOCKER-DEPLOY.md`：

```
make docker-env       # 生成 .env（随机密码；已存在则跳过）
make docker-up        # docker compose up -d --build
curl http://127.0.0.1:8080/api/health
make docker-smoke     # 8 步验收，输出 DOCKER SMOKE PASS
```

启动链是：

1. MySQL / Elasticsearch / Qdrant 并行启动，各自跑健康检查。
2. 三存储全部 healthy 后，`init` 开始。
3. `init` 等待三存储（每个最多 180 秒），开通 `dw_reader`，验证它真的只有 SELECT，对账 S2 元知识（`--verify`；失败则 `--fresh` 再 `--verify`）。
4. `init` 退出 0，`backend` 才启动。
5. `backend` 的 `/api/health` 通过后，`frontend` 才启动。

换一台机器时，按依赖顺序检查，而不是按报错顺序：

1. `.env` 是否存在、密码是否与卷匹配。`make docker-env` 只在缺失时生成；卷还在而密码换了，栈会一直不健康。
2. `docker compose ps -a` 看 `init` 的退出码。非零就先看 `docker compose logs init`，不要先动 `backend`。
3. 三存储是否 healthy。Qdrant 的健康检查打 `/readyz`，老镜像会一直 404。
4. `backend` 的 `/api/health` 是否返回 `ok`。这一步通过，才说明 `init` 的产物可用。
5. 最后才看前端：健康检查与一条真实查询。页面 502 先查上游某一环，nginx 本身通常没问题。

`make docker-smoke` 检查五服务运行、12 个步骤、经 8080 的真实查询、只读账号、S2 对账，并再次运行 `init` 证明「跳过重建」。真实查询那一步看的是 SSE **是不是边生成边发**：用裸 socket 记录每个分片的到达时刻，首尾分片几乎同时到达就说明代理把响应攒起来了。判定分三档：首尾时间差 ≥ 50 毫秒 → 流式；整个响应 < 200 毫秒 → 太快，分辨不出来，重试几次；其余 → 攒完了一次性到达，失败，并提示查 `docker/nginx.conf` 的 `proxy_buffering off`。这条判据防的是验收脚本最要命的失效：报了 PASS，可它验的东西根本没验。它是一条启发式：50 / 200 毫秒是经验阈值，不是统计结论；它能抓住「一次性到达」，不能证明「每一帧都单独到达」。

分片个数为什么不能当证据？因为一个攒起来一次性发送的响应，照样会在传输层被拆成好几个分片；数分片只能证明「网络把它切开了」，不能证明「后端边生成边发」。所以判据看的是到达时刻的分布：首尾几乎同时到达，说明中间被攒过；首尾拉开足够间隔，才说明每一段都是现发的。整个响应太快时也分辨不出来，脚本选择重试而不是宣布通过，这是同一原则的另一面：判据不确定时，不给绿灯。

4.5 输出：健康、日志、验收结论

S8 的输出是四类可核对的证据，不是「页面能打开」：

- **健康检查**：`curl http://127.0.0.1:8100/api/health` 返回 `{"status":"ok","stage":"S7"}`（`stage` 是后端声明的已完成阶段）；Compose 的 backend healthcheck 用 Python 标准库 `urllib.request.urlopen` 打同一个接口；frontend 用 `wget -qO- http://127.0.0.1/healthz`。
- **启动状态**：`docker compose ps -a` 能看到 `init` 是 `Exited (0)`，backend / frontend 是 `healthy`。

- **验收结论**：`make docker-smoke` 输出 `DOCKER SMOKE PASS`；`init` 第二次运行显示 `NOOP: 0 writes`。
- **日志**：`make docker-logs` 跟踪全部日志，`docker compose logs -f backend` 只看后端；nginx 的 `/api/` 关闭了 `proxy_buffering`，SSE 才能实时到达。

但本工作树没有这些输出。Docker socket 无权限，`make docker-env` / `make docker-up` / `make docker-smoke` 都没有执行。所以 S8 的部署结论只能是「按文件推导成立」，标 ?；不能写成「已验证」。

4.6 验收：哪些已验证、哪些没有

表 3-8 · S8 相关的证据状态

结论	状态	依据 / 缺口
`compose.yaml` 定义 6 个服务、3 个卷、1 个默认网络	✓	文件可逐项核对
只有 frontend 默认监听 `0.0.0.0`，其余绑 `127.0.0.1`	✓	`compose.yaml` 的 `ports` 段
backend 非 root（uid 10001）、单 worker	✓	`docker/backend.Dockerfile`、`compose.yaml` 的 `--workers 1`
nginx `/api/` 关闭缓冲、读超时 3600s	✓	`docker/nginx.conf`
Docker 全栈本机跑通	?	本工作树无 Docker socket 权限，未执行 `make docker-up` / `make docker-smoke`
真实 MySQL 上 `dw_reader` 写不了	?	无 `REBUILD_DW_RO_PASSWORD`、未执行 `make stage5-live`
真实 LLM 生成	?	无端点 / 无 Key
浏览器 UI 验收	?	本工作树无 Chrome / Edge
后端 782 passed / 31 skipped	!	`README.md` 声称；`docs/SETUP-FROM-SCRATCH.md` §3 另有 665 的版本；本工作树未跑全量
S8 性能基线（P50 / P95、并发、容量）	?	`README.md` 进度表写「S8 性能 待做」

这张表是 S8 最重要的产出：哪些结论能拿去做验收，哪些只能当设计意图。评审时该问的是「没跑通的那部分，凭什么说它成立」，而不是「你跑通了吗」。

这张表的用法是「先看状态，再看结论」。标为未验证的条目不是「设计错了」，是「本工作树没有条件验证」；标为推断的条目不是

「假的」，是「有文档声称、没有本机输出」。所以「按文件推导成立」只能算一个待验证的命题，不能算一条验收结论。

4.7 常见故障

故障一：容器后端和 `make dev-backend` 抢 8100。 两者都映射 `127.0.0.1:8100`，后起的那个报 `address already in use`。要么只用 Docker，要么只用本机开发；要并存就把 `.env` 的 `BACKEND_HOST_PORT` 改成 `8110`。前端不受影响，因为 nginx 在容器网络里仍按 `backend:8100` 找后端。

故障二：改了 `.env` 但密码没生效。 `docker compose restart` 不会重新读取环境变量；必须 `docker compose up -d`。换 MySQL 密码更麻烦：MySQL 只在**首次**初始化 `mysql_data` 时用 `MYSQL_ROOT_PASSWORD` 建账号；卷还在的话，容器里的密码仍是旧的，新密码不生效，健康检查会一直失败。正确顺序是 `make docker-reset` → `python scripts/generate_docker_env.py --rotate-secrets` → `source scripts/export-docker-env.sh` → `make docker-up`。

故障三：`make docker-env` 覆盖了正在用的 `.env`。 默认不覆盖；`--force` 保留已有密码与配置，`--rotate-secrets` 才换密码。不要手动删 `.env` 再生成：数据卷里的账号还是旧密码，栈会起不来。

故障二与故障三指向同一件事：`.env` 是输入，不是缓存。`docker compose restart` 不会重新读取它；改密码而不删卷，数据库里的账号还是旧的。把「配置变更」当成「重启一下」，是这一类故障的共同根因。

故障四：Qdrant 永远不 healthy。 健康检查打 `/readyz`，这个端点 v1.5.0 才有；老镜像上它一直 404，`init` 就不会启动。默认 `QDRANT_IMAGE` 是 v1.12.4，满足要求；如果你在 `.env` 里换了老版本，先换回来。

故障五：`init` 失败导致 backend 不启动。 `init` 非零退出会阻止 backend（`service_completed_successfully`）。先看 `docker compose logs init`：常见原因是三存储没 healthy、`REBUILD_DW_RO_PASSWORD` 缺失、S2 对账失败。不要用「把 depends_on 删掉」来绕过；那只会让 backend 带着坏数据启动。

故障六：页面 502。 先看 `docker compose ps -a` 的 backend 是否 healthy，再看 `docker compose logs backend`；backend 没起来查 `init`，backend healthy 但 502 查 nginx 反代。不要一上来就改 nginx 配置。

故障七：SSE 看起来「一次性到达」。 查 `docker/nginx.conf` 的 `/api/` location 是否 `proxy_buffering off`、`proxy_cache off`、`gzip off`，以及 `proxy_read_timeout` 是否足够长。`make docker-smoke` 的裸 socket 判据就是为这条准备的；如果它说「攒完了一次性到达」，不要用「分片个数」反驳——攒起来一次发的响应照样会被拆成好几个分片。

故障八：在 backend 容器里跑构建 CLI。 backend 只有 `dw_reader`，读不了 `meta_rebuild`；构建类 CLI 要挂在 `init` 服务下，并加 `--no-deps`，前提是三存储已经在跑：

```
docker compose run --rm --no-deps init python -m app.build_meta_knowledge --verify
```

4.8 证据

- `compose.yaml`：6 服务、健康检查、`depends_on` 条件、端口绑定、卷（✓ 文件存在）。
- `docker/backend.Dockerfile`：多阶段、非 root uid 10001、只带运行所需文件（✓）。
- `docker/frontend.Dockerfile`、`docker/nginx.conf`：nginx 托管、`/api/` 反代、SSE 关闭缓冲（✓）。
- `docs/DOCKER-DEPLOY.md`：启动顺序、密钥轮换、`docker-smoke` 判据（✓ 文档存在）。
- `README.md`：Docker 表、端口、`make docker-*` 命令（✓ 文档存在）。
- `design/inventory/facts.md` §7、§9：服务清单与「Docker Compose 全栈未跑」的记录（✓）。
- `make docker-up` / `make docker-smoke` 的实际输出（? 本工作树未执行）。
- `README.md` 的 782 passed / 31 skipped（! 文档声称，未在本工作树复跑）。

4.9 检查清单

- ☐ 我能说出六个服务、三个卷、唯一对外端口。
- ☐ 我知道 `init` 为什么是 `service_completed_successfully`，不是 `service_started`。
- ☐ 我知道构建凭据与应用凭据为什么要分开。
- ☐ 我知道改 `.env` 为什么必须 `up -d`，`restart` 为什么不行。

- [ ] 我知道换 MySQL 密码为什么必须先删卷。
- [ ] 我知道 SSE 在 nginx 上要关哪些开关。
- [ ] 我知道 Docker 全栈在本工作树是 ?，不是 ✓。
- [ ] 我知道 782 是 README 声称，标 !，不能写成已验证。

5. 走到这里，系统能做什么、还不能做什么

把 S5–S8 放在一起看，系统已经从「能生成 SQL」走到「能拒绝、能解释、能展示、能装进容器」。但「能」和「敢」之间还有一段距离。下面这张表把话说死：哪些是本次真的跑过的，哪些是代码推导，哪些是环境缺口。

表 3-9　· 能力与证据状态

能力	状态	证据 / 缺口
36 条注入语料 100% 拒绝，原因码精确匹配	✓	2026-10-29 `security --check`：36/36
11 条正常查询零误杀	✓	同一次输出：11/11
S4 基线 SQL 通过策略	✓	同一次输出：8/8
16 个错误码 / 3 个 class / 7 个路线图通道码	✓	2026-10-29 `failures --table` / `--check`
6 类失败场景逐条对上码 / 类 / 步骤	✓	同一次 `failures --check`：9 个场景全 PASS
前端生成物与契约一致	✓	2026-10-29 `contracts_gen --check`：3 个文件一致
14 条 SSE 夹具逐字对账	✓	2026-10-29 `record_sse --check`：14 条一致
只读账号 `dw_reader` 真的写不了	?	无 MySQL / 无 `REBUILD_DW_RO_PASSWORD`，`stage5-live` 未跑
浏览器里 6 类失败 + 取消各有各的样子	?	无 Chrome / Edge，`stage6-ui` 未跑
浏览器里 13 条 F 缺陷闭环	?	无 Chrome / Edge，`stage7-ui` 未跑
真实 LLM 生成并过策略	?	无端点 / 无 Key
Docker 全栈本机跑通	?	无 Docker socket 权限，`docker-up` / `docker-smoke` 未跑
后端 782 passed / 31 skipped	!	README 声称；SETUP §3 另有 665 的数字；本工作树未跑全量
前端 40 passed、`tsc` 零错误	!	README / SETUP 声称；本工作树未跑 `npm run test` / `npm run build`
S8 性能基线（P50 / P95、并发、容量）	?	README 进度表写「待做」

这张表直接回答两个问题。

系统能做什么？ 它能在离线环境证明「恶意 SQL 进不来、正常 SQL 不被误杀、失败有名字、前端契约不漂移」。这四件事都有可复现的命令，不依赖真实模型或数据库。

系统还不能做什么？ 它还没有证明「换一台机器、用真实 MySQL、接真实 LLM、在浏览器里点一遍」也成立。证据链缺了四段：`stage5-live`、`stage6-ui`、`stage7-ui`、`docker-smoke`。下一步是补证据，不是加功能，顺序就是上面这四段；没有真实执行，性能数字没有意义。

能力表与证据表要一起读。能力回答「系统会做什么」，证据回答「这件事被证明到什么程度」。一条能力可以在表里，但证据是空的；一条证据可以很硬，但只覆盖一条窄路径。把两列并排看，才不会把「代码里有」当成「线上成立」。

倒读排障顺序

正向读，四节是「设计 → 证明」；排障时反过来读更快：从用户看到的现象出发，往上游找最近一个该成立却没成立的不变量。

1. 页面显示成功、但表格只有一行像错误文案的数据 → 先查 S7 的 `PseudoErrorCard` 是否触发，再查后端是否把失败发成了 `result`。
2. 页面红框、码是 `SQL_REJECTED` → 看 `detail.reason_code`，再回到 S5 的对应步骤。
3. 页面一直转圈、最后报 `STREAM_INCOMPLETE` → 流没有收尾，查 S6 的终止状态与代理缓冲。
4. 页面 502 → 查 S8 的启动链：`init` 退出码、`backend` 健康、nginx 反代，顺序不能反。
5. 同一条查询在另一台机器上失败 → 查 `.env` 与卷，再查镜像与端口绑定。

倒读的价值在于：它把「四道交接」变成四条定位路径，每条路径的终点都是一个可执行的命令，而不是一句「再看看」。

5.1 本部分小结

- S5 像闸机：刷卡不过，门不开，连库都不连。36/36 注入全拒，11/11 正常查询零误杀。
- S6 像病历：每种失败都有码、有类别、有步骤。16 个错误码 / 3 个 class；`SQL_REJECTED` 永远不可重试。
- S7 像收据：后端说 `10`，前端不能写成 `"10"`。14 条 SSE 夹具逐字对账；13 条 F 缺陷都有浏览器用例，但本工作树没跑（?）。

- S8 像搬家：本机收拾得再好，换一台机器能不能起，是另一回事。这一步目前是 ⓘ：只有清单，没有整机实跑。

回到本章开头那个问题：一条能生成 SQL 的流水线，怎样证明它「敢用」？这一部分的回答是四句话：恶意 SQL 在连库前被判定（S5），失败有名字、有类别、有位置（S6），契约直接约束界面而不是靠人再抄一遍（S7），交付边界被写清楚、没证明的部分被标出来（S8）。四件事合起来，才是「敢用」的证据链；缺任何一环，链上就有一段只能靠相信。

证据等级划出了这条链的边界。标为已验证的条目说明本工作树跑过，标为推断的说明只有文档声称，标为未验证的说明环境不具备。读到这里，你应该能对着表 3-9 说出：哪些结论可以直接拿去做验收，哪些只能当设计意图，下一步该补哪条命令。

---

6. 复习题

十四道题按难度分三档：易题考「能不能复述一条规则」，中等题考「能不能解释一个选择」，难题考「能不能推演一个反事实」。每题后面给一条自检线索，只指方向，不给完整答案。

1. 【易】S4 的 `validate_sql` 已经做了 AST 静态校验，为什么还需要 S5 的 `enforce()`？两层各自回答什么问题？ 自检线索：一层管「像不像答案」，一层管「能不能碰库」；输入不同，失败后果也不同。
2. 【中】为什么 `ReadOnlyExecutor.execute()` 在策略拒绝时连数据库连接都不打开？如果先连库再判定，会多出什么风险？ 自检线索：判定与连接谁先谁后，决定「拒绝」是否等价于「没发生过」。
3. 【中】`SQL_REJECTED` 是通道码，`SQL_MULTIPLE_STATEMENTS` 是原因码。为什么要这样分层？如果反过来，下游会遇到什么问题？ 自检线索：通道码是下游分支的全集，原因码是诊断信息；想想每种攻击各占一个分支的代价。
4. 【易】为什么 `UNION` 被归到 `SQL_FORBIDDEN_CONSTRUCT`，而不是 `SQL_NOT_READ_ONLY`？这两条原因码在语义上差在哪里？ 自检线索：`Union` 的根节点本身是只读的，问题在它能带出什么。
5. 【中】策略返回的 `decision.sql` 和用户输入的原始 SQL 可能不一样（去注释、补 LIMIT）。为什么执行器必须执行前者？这和旧系统 D6 的漏洞有什么关系？ 自检线索：D6 的漏洞是「校验的文本」与「执行的文本」分叉；重新序列化把两者钉成一个对象。

6. 【中】16 个错误码里哪些是可重试的？为什么 `SQL_REJECTED` 永远不可重试？如果把它设成可重试，攻击面会怎么变化？ 自检线索：可重试意味着「再问一次模型」，而被策略拒绝的 SQL 已经越过边界。

7. 【中】校正闭环为什么最多只重试一次？校正后的 SQL 为什么必须重新过 guard 与 policy？如果只过 guard 不过 policy，会漏掉什么？ 自检线索：一次是成本，重新校验是边界；只过 guard 不过 policy，漏掉的是执行安全。

8. 【易】`EMPTY_RESULT` 的 `class` 是 `empty`，不是 `failure`。这个分类对前端步骤条、错误卡片、用户心理分别有什么影响？ 自检线索：分别看步骤条、卡片、用户心理三处的变化。

9. 【难】取消为什么是 `cancelled`，不是 `done`，也不是 `failure`？编排器在哪个粒度检查取消？为什么不能中断已经发给 MySQL 的查询？ 自检线索：取消的语义是「不再推进」，不是「杀死数据库里的查询」；再想步骤边界在哪。

10. 【易】`frontend/src/generated/` 三个文件为什么不能手改？`contracts_gen --check` 与「重新生成再覆盖」有什么区别？ 自检线索：生成物是派生物；`--check` 做的是比对，不是覆盖。

11. 【难】前端遇到未知事件类型和未知步骤 id 的处理为什么不同？这两种「未知」分别代表什么契约状态？ 自检线索：一个是「契约多了一条腿」，一个是「契约自相矛盾」；容忍与报错分别对应哪种。

12. 【中】`DECIMAL("10")` 归一化成 `int`，`DECIMAL("10.5")` 归一化成 `float`。为什么不用统一 `float`？ `_sse()` 为什么不传 `default=str`？ 自检线索：`int` 与 `float` 的选择服务的是「用户看到什么」，`default=str` 服务的是「不要报错」。

13. 【难】Docker Compose 里 `init` 为什么用 `service_completed_successfully`，而不是 `service_started`？如果改成后者，backend 可能在什么状态下启动？ 自检线索：一个是「进程起来了」，一个是「初始化成功退出」；`init` 是一次性任务。

14. 【难】为什么本工作树把 Docker 全栈标 ⓘ、把 782 tests 标 ⓘ？这两个标记分别缺什么证据？如果你要补，第一步跑哪条命令？ 自检线索：一个缺环境，一个缺复跑；补证据的第一步是让缺的那条命令真的跑起来。

难题不要求你同意本章的取舍，只要求你指出取舍的代价。比如第 6 题与第 9 题都可以反问：如果业务要求「取消立刻释放数据库连接」，当前设计要改哪一层？能问出这个问题，说明你已经把四道交接看成了四条可以单独修改的边界，而不是一个笼统的「系统」。

04

第四部分 ·

设计图集：三十二个角度

先读图 32。三十二张图都有 SVG，但有图不等于有证据；图 32 是证据台账：哪条结论跑过、哪条只是推导、哪条完全没验，先看它，再回头看前面的图。

这一部分按六个角度排图。每组开头先给一句「这组最容易误读什么」，再进具体图；那句话是给评审者的，先知道会读错在哪，再看图有没有把这个错挡住。

同一套系统，从三十二个角度看，会得到三十二个不同的问题。图集不求画全，每个角度只承担一段可检查的论证：看完这张图，读者能回答一个具体问题，并知道证据与边界在哪里。图在这里既不是装饰，也不替代源码，它把设计意图和可核对的事实接起来。

0. 怎么用这套图

每张图只服务一个主问题。图前的结论应可被反驳；图后三段说明它回答什么、由哪些元素构成、证据与边界在哪。图里找不到对应元素的话不该写；指向不了文件或命令的图只能算草图。按 `STYLEGUIDE.md` 的三类读者来读：新手看「谁把什么交给谁」，实现者顺图找文件，评审者先看证据再看结论。

本工作树的图状态不一致，且会随其他任务推进变化。截至定稿，图 01–32 均已有 SVG；但图 20–24 的条目仍写「未生成」，是旧状态。对未渲染或未生成的图，条目只写它按 `WBS.md` 应承担的论证责任，不描述看不见的画面，证据按实际状态标 ⓘ；图源落地后须回填对账。

图号 01–32 全图集唯一，不按目录重排；引用统一写「图 NN · 标题」。图片路径相对本章文件，解析后落在 `design/diagrams/**`。每组开头一句总纲，组内按依赖顺序而非重要性排列。

1. 三十二个角度索引

> 完整的图页与读图地图见独立文档 `design/design-doc.pdf`。本节只保留索引，避免与设计文档重复。

图	标题	回答什么	读图 1→2→3
01	C4-L1 系统上下文	DSWS 的系统边界在哪？哪些是外部依赖？生产问数主链路到底读哪几个外部系统？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
02	C4-L2 容器图	一次请求跨哪些容器或进程？S2 构建与 init 在容器拓扑里处于什么位置？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
03	backend/app 模块依赖	谁 import 谁？哪些依赖只发生在构建时，哪些发生在每次请求？安全层和生成层各自依赖什么？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
04	前端文件树	前端有哪些文件、谁调用谁？SSE 解析与契约类型分别住在哪？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
05	阶段演进 S0→S8	为什么按 S0–S8 顺序重建？每个阶段到底在验收哪一种「还不确定」？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
06	构建时 vs 查询时	S2 退出之后，查询时每个阶段各自直接读什么？有没有常驻构建进程？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
07	端到端请求生命周期	一次问数里谁把什么交给谁？失败、空结果、取消分别在哪一步分叉？为什么这条流一定会结束？	1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
08	S2 构建流水线时序	谁在读契约？谁在写三个存储？写序为什么是这个顺序？四个开关各改变了什么？	1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
09	S3 召回流水线时序	三存储各自被问了几次？候选怎么合并？哪些字段会被丢掉？	1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
10	S4 生成 SQL 流水线	上下文怎么进提示词？生成器有两种，怎么选？为什么形状校验必须在静态校验之前？	1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边

图	标题	回答什么	读图 1→2→3
11	S5 安全校验与执行	策略在哪两个地方各查一次？被拒绝时会不会连库？超时是怎么产生的？	1 先看触发者（谁发起）→ 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
12	S6 校正闭环状态机	什么情况下会走 `correct_sql`？走完之后为什么必须重新校验？哪些失败是不可重试的？	1 先看触发者（谁发起）→ 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
13	S7 SSE 事件序列	一条 `progress` 到哪个组件？`error.class` 三个取值各长什么样？取消为什么不算 done？	1 先看触发者（谁发起）→ 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
14	数仓 ER 图（5 表）	每张表一行代表什么？主键是什么？事实表如何关联维度？哪些列是真实存在、哪些是常见误问（`city`、`price`）？	1 先看实体与 grain（一行代表什么）→ 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
15	契约模型（表 / 列 / 指标 / 值域 / 列—指标）	契约里到底有哪些对象？每个对象带哪些字段？`ValueDomain` 的 `kind` 有哪几种？`ColumnMetric` 为什么是派生表？	1 先看实体与 grain（一行代表什么）→ 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
16	一份契约如何落到三存储（region_name / order_amount）	`region_name` 和 `order_amount` 分别变成什么？为什么一个进 ES、一个不进？Qdrant 的 98 个 point 是怎么来的？Meta 的 `examples` 能不能当全集？	1 先看实体与 grain（一行代表什么）→ 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
17	S2 数据 lineage（契约 + 冻结数仓 → 三存储 + manifest）	数据从哪来？在哪一步校验？三存储各写多少？`5/24/6/9/76/98/22` 分别是什么？manifest 为什么是构建成功的唯一凭据？	1 先看实体与 grain（一行代表什么）→ 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
18	Meta 表结构（5 张元数据表）	每张 Meta 表的字段和主键是什么？`column_info` 怎么指向表？`column_metric` 的复合主键是什么？manifest 记录哪些计数？	1 先看实体与 grain（一行代表什么）→ 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
19	值域分类决策树（哪些列写 ES）	`enum` 和 `range` 分别怎么处理？`open` 文本为什么有的进 ES 有的不进？`sync_values` 到底记什么？Meta 的 `examples` 和 ES 的完整值域是什么关系？	1 先看实体与 grain（一行代表什么）→ 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
20	信任边界与攻击面	一次问数跨过哪些信任边界？谁持有数据库凭据？LLM 能拿到什么、拿不到什么？旧系统会不会被新系统写入？	1 先看信任边界在哪 → 2 再按顺序看拦截点 → 3 最后看被拒绝的路径与出口
21	五层纵深防御	一条 SQL 从模型出来到数据库要过几关？每关挡住什么？为什么说「策略被绕过也还有数据库权限兜底」？失败最终怎么到 UI？	1 先看信任边界在哪 → 2 再按顺序看拦截点 → 3 最后看被拒绝的路径与出口
22	SQL 策略判定流程（六关 + 收尾）	每一关拒绝什么？拒绝后会不会连库？为什么执行的不是模型返回的原文？guard 和 policy 的先后顺序是什么？	1 先看信任边界在哪 → 2 再按顺序看拦截点 → 3 最后看被拒绝的路径与出口
23	只读账号权限模型（dw_reader 与应用账号分离）	`dw_reader` 是谁、怎么创建、怎么验证？执行器和构建任务用的是不是同一个账号？旧 `GRANT ALL` 为什么危险？应用账号与只读账号怎么分离？	1 先看信任边界在哪 → 2 再按顺序看拦截点 → 3 最后看被拒绝的路径与出口
24	错误码分类（16 码 / 3 class / retryable）	16 个错误码分别属于哪个 class？哪些能自动校正？空结果和用户取消为什么不算失败？失败最后怎么变成 SSE 事件？	1 先看信任边界在哪 → 2 再按顺序看拦截点 → 3 最后看被拒绝的路径与出口

图	标题	回答什么	读图 1→2→3
25	Docker Compose 拓扑	哪六个服务？谁依赖谁？数据落在哪？`init` 为什么和另外五个不一样？	1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点
26	宿主端口与容器端口暴露面	谁能从局域网进来？哪些端口只能本机访问？容器之间走什么端口？	1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点
27	启动顺序与失败停点	谁在等谁？每级等多久？失败会停在哪一步？`init` 内部又做了什么？	1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点
28	旧系统与 rebuild 的隔离边界	旧系统的 `meta` / `value_index` / `5173` / `3306` 与 rebuild 的 `*_rebuild` / `5273` / `8100` 在哪一层分开？哪些东西其实是共用的？	1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点
29	测试金字塔与三存储依赖	五层测试各自覆盖什么？哪一层开始需要 MySQL / ES / Qdrant？哪些是「本机就能重复」的门禁？	1 先看层级（单元/集成/端到端）→ 2 再看阶段门禁顺序 → 3 最后看失败注入覆盖了哪些码
30	S0–S7 阶段门禁	每个阶段跑什么？「通过」的判据是什么？哪两个阶段需要先起三存储？	1 先看层级（单元/集成/端到端）→ 2 再看阶段门禁顺序 → 3 最后看失败注入覆盖了哪些码
31	失败注入矩阵	失败从哪里注入？落到哪个错误码？属于 `failure` / `empty` / `cancelled` 哪一类？哪些会触发校正闭环？界面上长什么样？	1 先看层级（单元/集成/端到端）→ 2 再看阶段门禁顺序 → 3 最后看失败注入覆盖了哪些码
32	证据台账：已验证 / 推断 / 未验证	图里的结论从哪来？哪些能在本机复现？哪些只是引用他机记录？最终汇到哪里？	1 先看层级（单元/集成/端到端）→ 2 再看阶段门禁顺序 → 3 最后看失败注入覆盖了哪些码

05

第五部分 ·

部署、测试与故障排查

在本设计工作树里跑 Docker，得到的不是容器列表，而是一行 `permission denied ... /var/run/docker.sock`。命令没有错，Compose 文件也没有错；当前账号没有这个 socket 的访问权限。

这一部分的顺序不能反：先跑起来，再证明没坏，最后才轮到「坏了先看哪一眼」。环境没通，翻业务日志是白翻。

这一部分回答的是「代码写完之后，怎么把它跑起来、怎么证明它没坏、坏了先看哪里」。前面的章节按阶段讲每个能力怎么长出来；这一章换一个视角：把系统当成一台要交付的机器，从启动路径、容器拓扑、凭据注入、启动顺序、隔离边界、测试门禁、失败注入，一直讲到上线前的检查清单。

阅读时请记住一条纪律：本章所有命令都能在仓库的 `Makefile`、`docs/DOCKER-DEPLOY.md`、`docs/SETUP-FROM-SCRATCH.md` 或 `scripts/` 下找到出处；所有计数以 `design/inventory/facts.md` 为准。凡是本设计工作树没有实际跑过的，一律标 ⍰，不把推断写成已验证。

1. 两条启动路径：本地开发 vs Docker

DSWS 有两条互不替代的启动路径。它们跑的是同一份代码，但依赖的边界不同：本地开发把 Python、Node、三存储拆开装；Docker 把六个服务装进同一个 Compose 项目。新手最常见的错误，是把两条路径的端口混着用，尤其是 `8100`。

表 5-1 · 两条启动路径

维度	本地开发	Docker Compose
用途	改代码、跑单阶段验收、看日志	交付一套可复现的整机环境
安装	`make install`（uv + npm）	只需 Docker Engine 与 Compose v2
启动后端	`make dev-backend`	`make docker-up` 中的 `backend` 服务

维度	本地开发	Docker Compose
启动前端	`make dev-frontend`	`make docker-up` 中的 `frontend` 服务
三存储	单独 `docker compose up -d mysql elasticsearch qdrant`	同一条 `make docker-up` 一并拉起
后端端口	`127.0.0.1:8100`	宿主机 `127.0.0.1:8100` → 容器 `8100`
前端端口	`5273`（Vite，刻意避开旧栈 `5173`）	宿主机 `8080` → 容器 `80`
访问入口	`http://127.0.0.1:5273`	`http://127.0.0.1:8080`
前置	Python 3.12、Node、uv、网络	Docker、8 GB 内存、首次构建需网络
适用场景	开发、调试、逐阶段验收	演示、交接、无本机运行时

两条路径的第一个冲突点就是 `8100`：容器后端也映射到宿主机 `8100`，和 `make dev-backend` 完全相同。两者同时启动，后起的那个会报 `address already in use`。如果确实要并存，在 `.env` 里把 `BACKEND_HOST_PORT` 改成 `8110`；前端不受影响，因为 nginx 在容器网络里仍然按 `backend:8100` 找后端。

本地路径还有一条隐含前置：S2、S3 需要三存储，S5 的只读账号与两屏对比需要 MySQL。S0、S1、S4、S5 离线门禁、S6、S7 都不需要三存储。这一点在测试章节会再展开，因为它决定了「一台干净机器上先能跑到哪一步」。

```
# 本地：只起三存储，给 S2 / S3 用
make docker-env
docker compose up -d mysql elasticsearch qdrant
source scripts/export-docker-env.sh
```

`make docker-env` 只在 `.env` 不存在时生成随机密码；`source scripts/export-docker-env.sh` 把 Compose 的密码和宿主机端口导出成 `REBUILD_*`，成功时不打印任何内容。宿主 CLI 不会自动读 `.env`，漏掉这一句，S2 会在连库之前就报缺凭据。

---

2. Docker Compose 拓扑

`compose.yaml` 定义六个服务，项目名固定为 `shopkeeper-rebuild`。三个存储是长驻服务，`init` 是一次性任务，`backend` 与 `frontend` 是应用服务。拓扑见图 25（`design/diagrams/05-deployment/25-compose-topology.mmd`）。

这张图回答：哪六个服务？谁依赖谁？数据落在哪？init 为什么和另外五个不一样？

读图：1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点

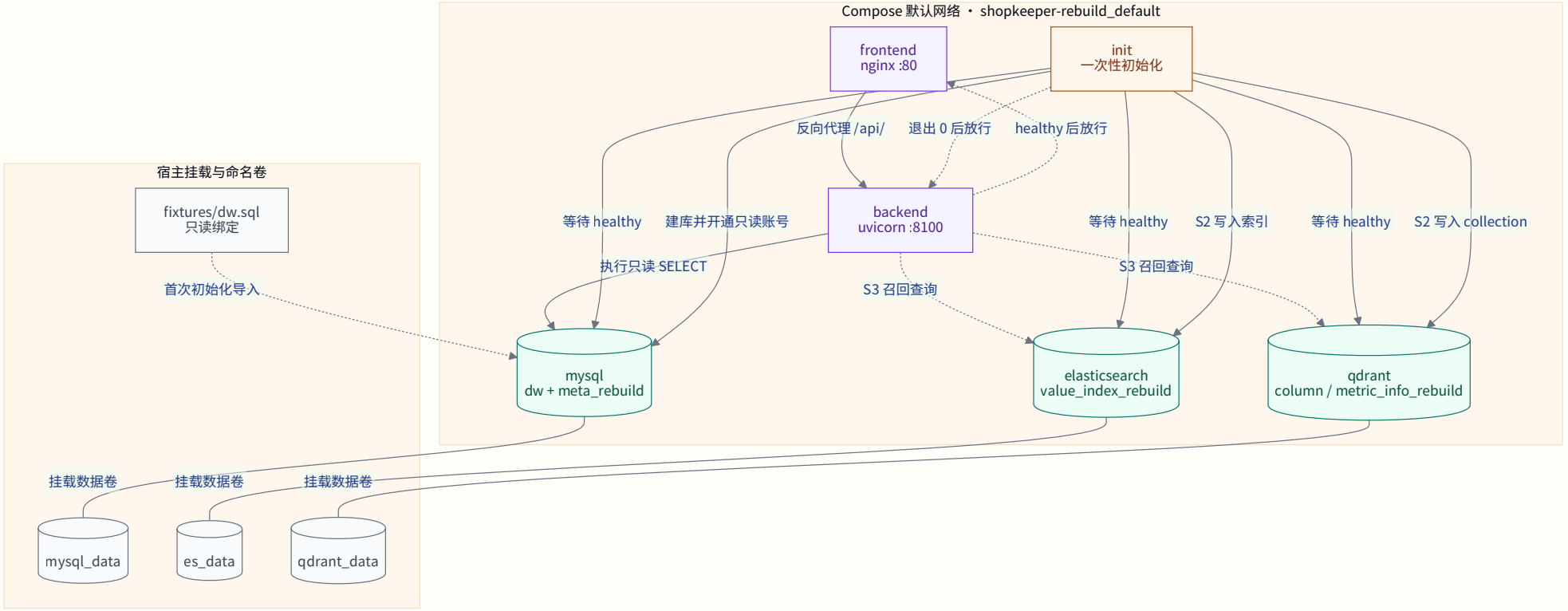


图 25 · Docker Compose 拓扑

表 5-2 · 六个服务

服务	镜像来源	作用	关键依赖	健康检查
`mysql`	`MYSQL_IMAGE`	数仓 `dw` + Meta 库	无	`mysqladmin ping`
`elasticsearch`	`ELASTICSEARCH_IMAGE`	取值索引	无	集群状态 green/yellow
`qdrant`	`QDRANT_IMAGE`	向量索引	无	`GET /readyz` 返回 200

服务	镜像来源	作用	关键依赖	健康检查
`init`	后端镜像	建库、开只读账号、校验/构建 S2	三存储 healthy	无，退出码即结果
`backend`	后端镜像	FastAPI + uvicorn，单 worker	`init` completed	`GET /api/health`
`frontend`	nginx 镜像	静态资源 + `/api/` 反向代理	`backend` healthy	`GET /healthz`

三个存储都只绑定宿主机 `127.0.0.1`，默认端口分别是 `3306`、`9200`、`6333`。只有 `frontend` 默认监听所有网卡（`FRONTEND_BIND=0.0.0.0`），宿主机端口 `8080`。这是刻意的暴露面设计：应用入口可以给局域网，数据库和搜索引擎不对外。端口全景见图 26（`design/diagrams/05-deployment/26-port-exposure.mmd`）。

这张图回答：谁能从局域网进来？哪些端口只能本机访问？容器之间走什么端口？

读图：1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点

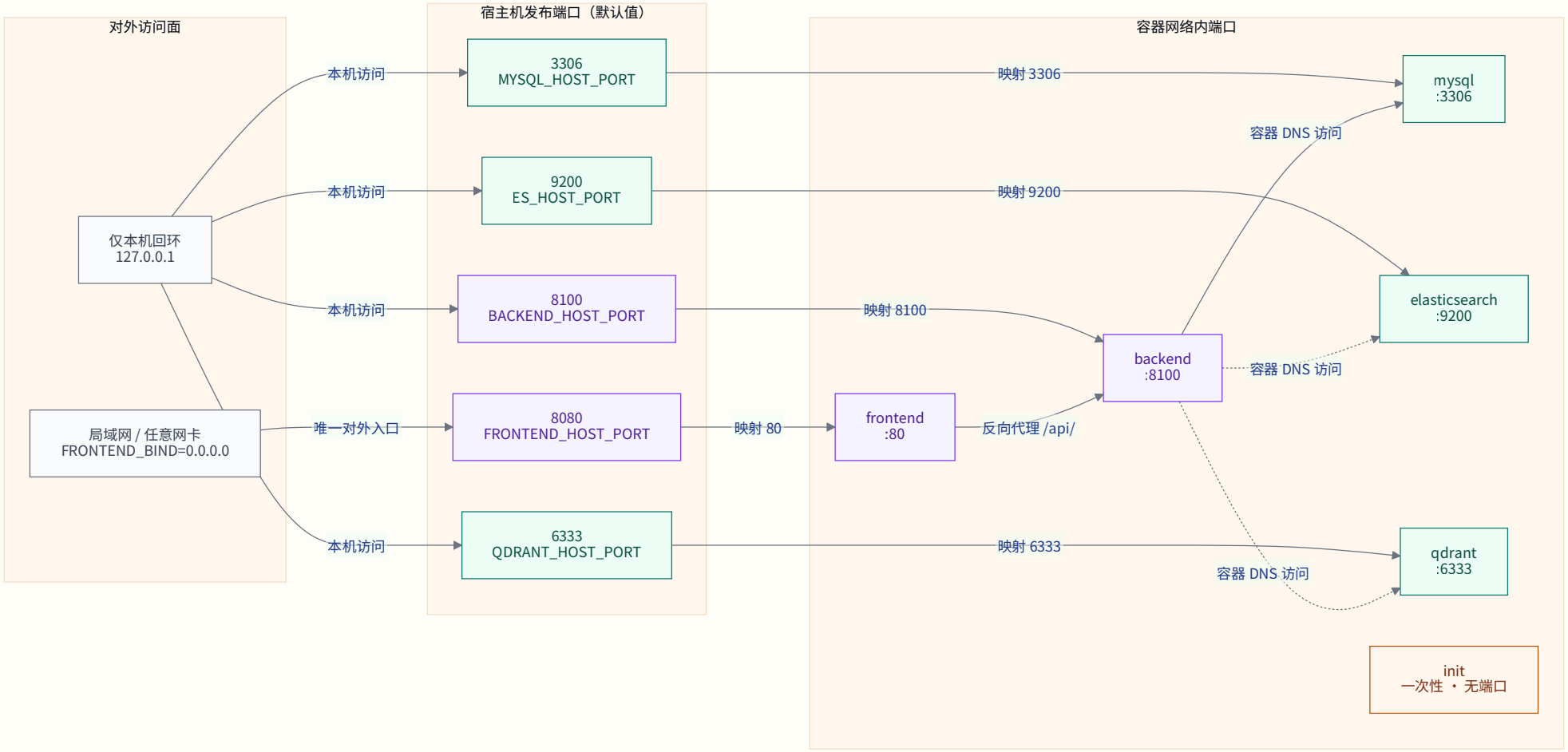


图 26 · 端口与网络暴露面

卷有三个，都是命名卷，Compose 会按项目名加前缀：

卷名	挂载点	存什么
`mysql_data`	`/var/lib/mysql`	`dw` 与 `meta_rebuild`，含首次导入的 `fixtures/dw.sql`

卷名	挂载点	存什么
`es_data`	`/usr/share/elasticsearch/data`	`value_index_rebuild`
`qdrant_data`	`/qdrant/storage`	`column_info_rebuild` 、 `metric_info_rebuild`

实际卷名是 `shopkeeper-rebuild_mysql_data` 、 `shopkeeper-rebuild_es_data` 、 `shopkeeper-rebuild_qdrant_data` 。 `make docker-down` 只删容器和网络，保留这三个卷；只有 `make docker-reset` 会先列出卷名、要求输入 `yes` ，再执行 `docker compose --project-name <项目名> down -v` 。

网络只有 Compose 默认创建的一个 bridge 网络。服务之间用服务名互相访问： `mysql:3306` 、 `elasticsearch:9200` 、 `qdrant:6333` 、 `backend:8100` 。 `init` 的 `REBUILD_ES_URL` 就是 `http://elasticsearch:9200` ， `backend` 的 `REBUILD_QDRANT_URL` 就是 `http://qdrant:6333` 。这些名字不能改成 `localhost` ，容器里的 `localhost` 指向容器自己。

健康检查的细节值得单独看，因为它们是启动顺序的唯一依据：

- MySQL： `mysqladmin ping -h 127.0.0.1 -uroot -p$$MYSQL_ROOT_PASSWORD --silent` ，间隔 5 秒，重试 60 次，启动宽限 30 秒。
- Elasticsearch： `curl` 集群健康接口并 `grep` 出 `green` 或 `yellow` ，间隔 10 秒，重试 30 次。
- Qdrant：镜像里没有 `curl` / `wget` ，健康检查用 `bash` 的 `/dev/tcp` 手工发 `GET /readyz` ，再 `grep 200` 。必须显式用 `bash` ，因为镜像里的 `/bin/sh` 是 dash，没有 `/dev/tcp` 。
- backend：用 Python 标准库 `urllib.request.urlopen('http://127.0.0.1:8100/api/health')` ，间隔 10 秒，重试 12 次。
- frontend： `wget -qO- http://127.0.0.1/healthz` 。

Qdrant 的健康检查还有一个版本门槛： `/readyz` 是 v1.5.0 才有的端点。老镜像上它一直 404，容器永远不 healthy， `init` 就不会启动。默认镜像 `qdrant:v1.12.4` 满足要求。

`backend` 固定 `--workers 1` 是为了取消能用：注册表 `CANCEL_REGISTRY` 是进程内状态，多 worker 会让「取消」打到另一个进程上，取消按钮就失效了。

镜像构建也有一套边界。后端镜像用多阶段构建： `uv` 阶段装依赖，运行阶段只带 `.venv` 与 `app/` ，再复制 `contracts/` 与 `fixtures/` 。前端镜像在 Node 阶段跑 `npm run build` ，运行阶段只把 `dist/` 交给 nginx。这解释了一个常见困惑：改了 `docker/nginx.conf` ，必须

重建 frontend 镜像才会生效；只 `restart` 容器不会重新拷贝配置。反过来，改了后端源码但构建命中了缓存，`docker compose up -d --build` 会重建 backend 与 init，这是正常行为，不是「没重启」。

还有一个容易误判的点：`init` 与 `backend` 用的是同一个镜像、同一份源码，但环境变量不同。`init` 拿得到 `MYSQL_PASSWORD` 与 root 密码，`backend` 只有 `dw_reader`。如果某个构建类命令在 `backend` 容器里报「表不存在」或「Access denied」，先别怀疑代码——很可能是把只读容器当构建容器用了。构建类 CLI 一律挂在 `init` 服务下，并加 `--no-deps`，前提是三存储已经在跑。

3. 环境变量与凭据

环境变量分三类：Compose 插值用的变量（`MYSQL_*`、镜像名、宿主机端口）、后端读取的 `REBUILD_*`、以及可选的 LLM 三元组。表 5-3 只列会影响启动或运行的关键项。

表 5-3 · 关键环境变量

变量	谁读	缺了会怎样
`MYSQL_ROOT_PASSWORD`	MySQL 容器、`init`	Compose 的 `${VAR:?}` 直接拒绝启动
`MYSQL_PASSWORD`	MySQL 容器、`init`	同上
`REBUILD_MYSQL_PASSWORD`	`init`、宿主 CLI	连库前报缺凭据，不会拿错口令去试
`REBUILD_MYSQL_ROOT_PASSWORD`	`init` 建库/授权	`init` 在授权步骤失败
`REBUILD_DW_RO_PASSWORD`	`init`、`backend`、`make stage5-live`	只读账号开不了；查询以 `INTERNAL_ERROR` 收尾
`REBUILD_MYSQL_HOST` / `PORT` / `USER`	后端、CLI	默认 `127.0.0.1:3306` / `didilili`
`REBUILD_META_DB`	S2 / S3 / 后端	默认 `meta_rebuild`
`REBUILD_ES_URL` / `REBUILD_ES_INDEX`	S2 / S3	默认 `http://127.0.0.1:9200` / `value_index_rebuild`
`REBUILD_QDRANT_URL`	S2 / S3	默认 `http://127.0.0.1:6333`
`REBUILD_COLUMN_COLLECTION` / `METRIC_COLLECTION`	S2 / S3	默认 `column_info_rebuild` / `metric_info_rebuild`
`REBUILD_VECTOR_SIZE`	S2 / S3	默认 `256`，与旧系统的 `1024` 不同
`REBUILD_LLM_BASE_URL` / `API_KEY` / `MODEL`	`backend`、CLI	三个都空则走确定性生成器，不会报错

变量	谁读	缺了会怎样
`REBUILD_LLM_TIMEOUT`	`backend`	默认 `30` 秒
`FRONTEND_BIND` / `FRONTEND_HOST_PORT`	Compose	默认 `0.0.0.0` / `8080`
`BACKEND_HOST_PORT`	Compose	默认 `8100`，与本地后端冲突时改这里
`MYSQL_HOST_PORT` / `ES_HOST_PORT` / `QDRANT_HOST_PORT`	Compose	默认 `3306` / `9200` / `6333`

凭据纪律有三条，任何一条破了都不是「小问题」：

第一，**密码不进仓库**。`.env` 被 `.gitignore` 忽略；仓库里只有 `.env.example`，里面是 `change-me-*` 占位值。`docker-smoke.sh` 在动手之前会扫 `.env`，只要命中 `change-me` 就直接判不通过——因为 Compose 的 `${VAR:?}` 只检查「非空」，占位密码照样能让整个栈起来、验收照常通过，但那个密码是公开的。

第二，**构建凭据与应用凭据分离**。`mysql` 与 `init` 拿 `MYSQL_PASSWORD`（应用账号 `didilili`，在 `dw` 上有 ALL PRIVILEGES）和 root 密码；`backend` 只拿 `REBUILD_DW_RO_PASSWORD`，也就是 `dw_reader` 的密码。构建类 CLI 要挂在 `init` 服务下跑，不能把构建凭据给 `backend`。`didilili` 的 ALL PRIVILEGES 正是 S5 要消除的越权入口。

第三，**宿主 CLI 与容器必须读到同一个字符串**。`source scripts/export-docker-env.sh` 以 `.env` 为事实源导出 `REBUILD_*`；`MYSQL_PASSWORD` / `MYSQL_ROOT_PASSWORD` 刻意**不导出**，因为 Compose 只看得到已 export 的变量，导出后旧快照会一直挡住 `.env` 里的新值。改了 `.env` 必须重新 `source` 一次。容器那边不用管，Compose 每次都重新读 `.env`。

轮换只读密码的顺序是固定的：改 `.env` 的 `REBUILD_DW_RO_PASSWORD`，跑 `docker compose run --rm --no-deps init python -m app.scripts.security --provision-ro-account`，再 `docker compose up -d --force-recreate backend`。轮换 MySQL 密码则必须先 `make docker-reset`，因为 MySQL 只在数据卷首次初始化时用 root 密码建账号；卷还在，新密码不生效。

4. 启动顺序与失败定位

启动顺序由 Compose 的 `depends_on` 条件串成一条链，不是「谁先写完谁先起」。图 27（`design/diagrams/05-deployment/27-startup-order.mmd`）描述了整条链，这里用文字再钉一遍：

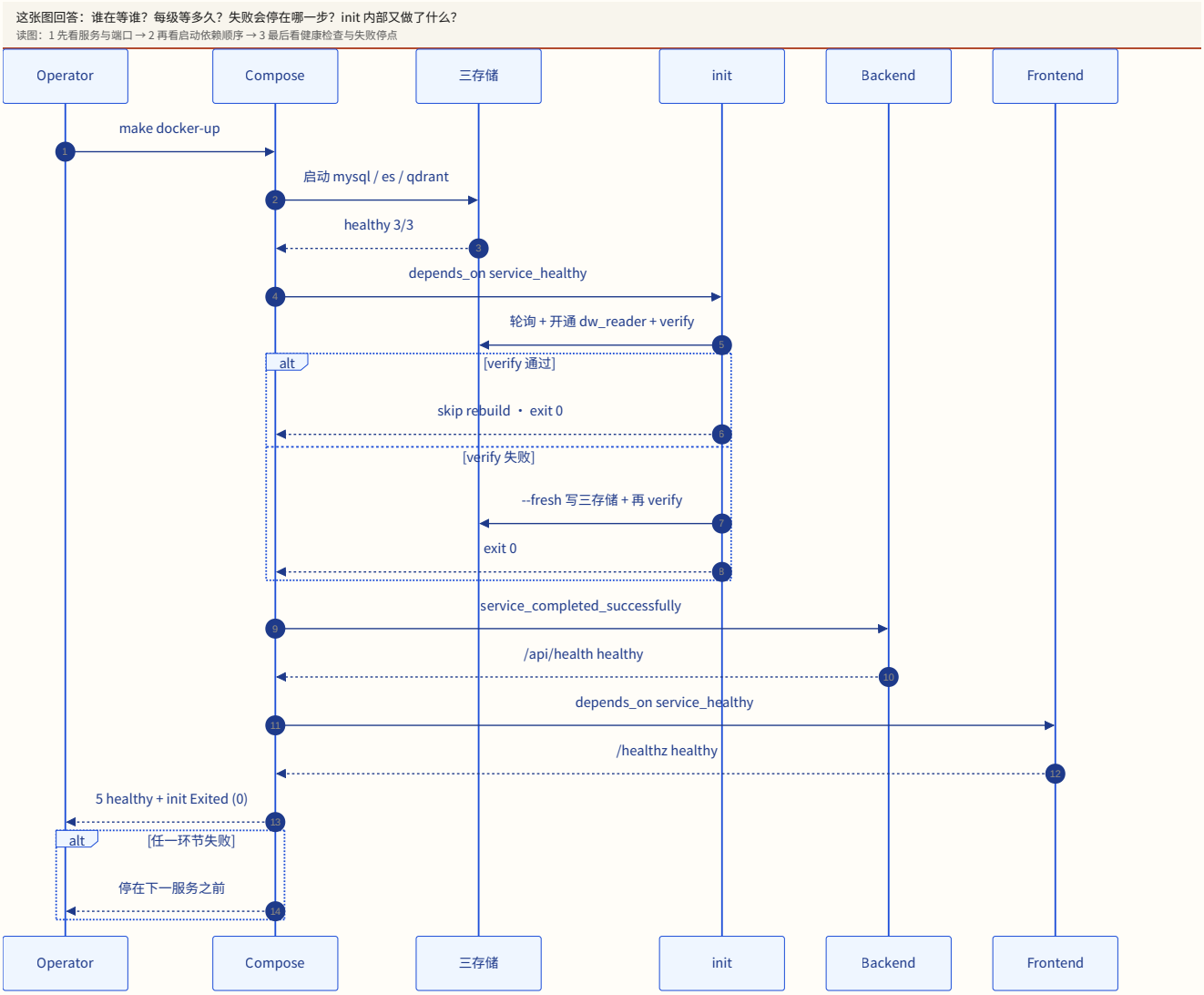


图 27 · 启动顺序与失败停点

1. `docker compose up -d --build` 先构建后端与前端镜像。
2. MySQL、Elasticsearch、Qdrant 并行启动，各自跑健康检查。
3. 三个存储全部 healthy 后，`init` 才开始。
4. `init` 依次做四件事：等待三个存储（每个最多 180 秒）、开通 `dw_reader`、验证它真的只有 SELECT、对账 S2 元知识（`--verify`；失败则 `--fresh` 再 `--verify`）。
5. `init` 退出码为 0，`backend` 才启动。

6. `backend` 的 `/api/health` 通过后，`frontend` 才启动。

`depends_on` 写的是条件，不是「启动顺序建议」：`service_healthy` 会一直等健康检查变绿，`service_completed_successfully` 会等 `init` 以 0 退出。所以「栈起来了但页面 502」通常不是 nginx 的问题，而是这条链在某一环停住了。

失败会停在哪里，可以从 `docker compose ps -a` 一眼看出来：

停在哪	现象	先看什么
三存储	某个服务 `starting` 或 `unhealthy`	`docker compose logs mysql` / `elasticsearch` / `qdrant`
`init`	`exited 1`，backend 不存在	`docker compose logs init`
`backend`	`running` 但 `unhealthy`	`docker compose logs backend`，再 `curl /api/health`
`frontend`	`running` 但 `unhealthy`	`docker compose logs frontend`，`nginx -T`

`init` 是最容易被忽略的一环，因为它退出后就不在 `docker compose ps` 的默认视图里。诊断时一定要加 `-a`：`docker compose ps -a` 能看到 `init` 的 `ExitCode`。`docker-smoke.sh` 的失败处理也特意先打 `ps -a` 和 `logs --tail=60 init backend frontend`，就是怕「最可能的故障现场」消失。

`/api/health` 返回 `{"status":"ok","stage":"S7"}` 只证明 FastAPI 进程活着，不证明数据库、ES、Qdrant 真的通了。真正的数据面验证在 `make docker-smoke` 的后四步：12 个步骤、经 8080 的真实查询、只读账号、S2 对账。把健康检查当成就绪证明，是运维里最常见的假阳性。

先看哪一眼：`docker compose ps -a` 的状态列。`running` 只说明进程在，`healthy` 才说明探针过；`init` 的退出码是 0，backend 才有资格启动。

---

5. 隔离：旧系统 / rebuild

这套系统在旧系统旁边重建，没有原地改造。隔离是硬边界，不是建议：`--fresh` 会删东西，靠它才不至于删错。图 28（`design/diagrams/05-deployment/28-legacy-isolation.mmd`）画出了两套系统的边界。

这张图回答：旧系统的 meta / value_index / 5173 / 3306 与 rebuild 的 *_rebuild / 5273 / 8100 在哪一层分开？哪些东西其实是共用的？
读图：1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点

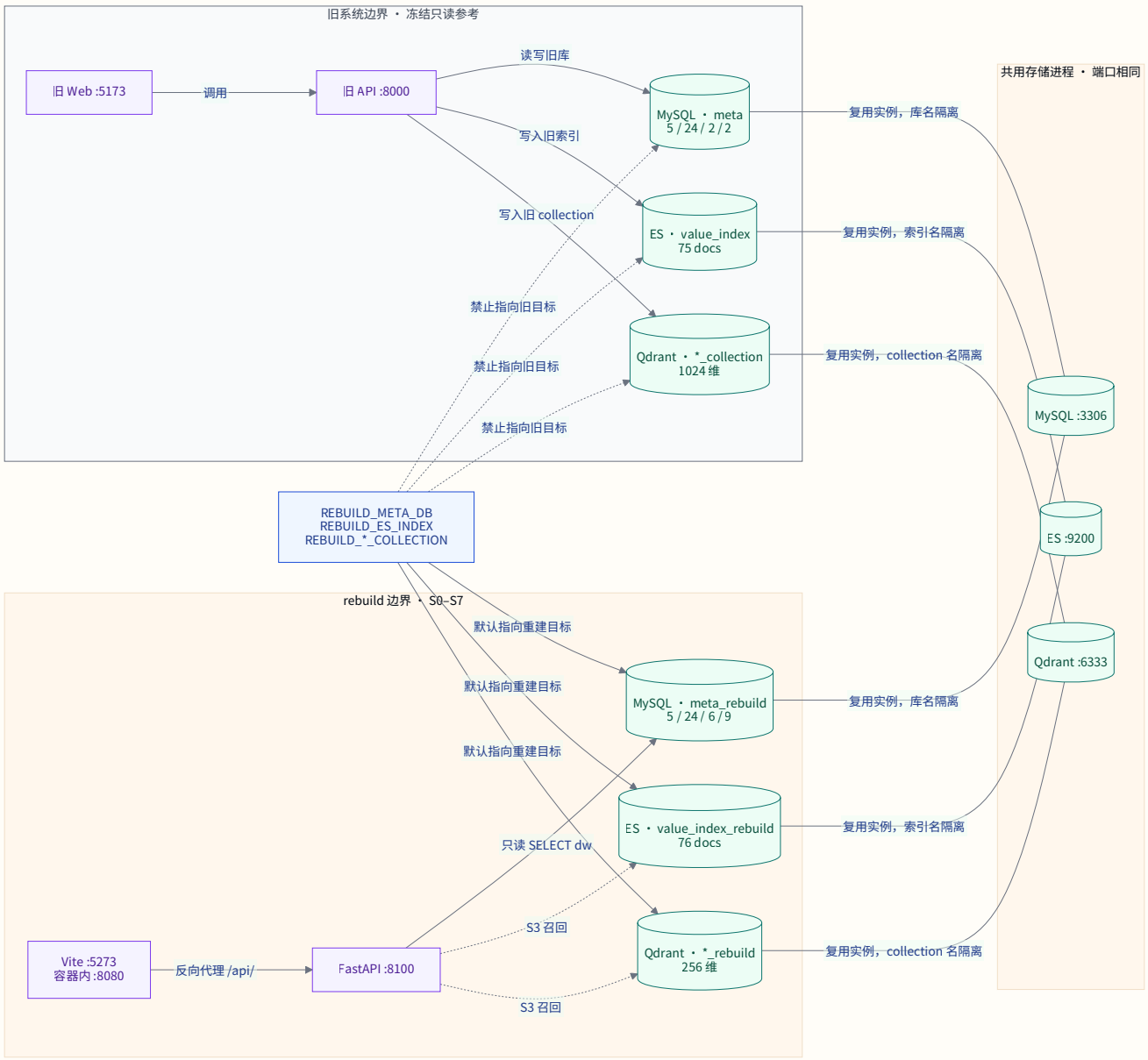


图 28 · 旧系统与 rebuild 隔离

表 5-4 · **旧系统与 rebuild 的隔离**

维度	旧系统	rebuild
MySQL 库	`meta`	`meta_rebuild`
ES 索引	`value_index`	`value_index_rebuild`
Qdrant collection	`column_info_collection` / `metric_info_collection`	`column_info_rebuild` / `metric_info_rebuild`
向量维度	1024	256
前端端口	`5173`	`5273`（本地）/ `8080`（Docker）
后端端口	`8000`	`8100`

维度	旧系统	rebuild
配置来源	`fixtures/legacy/meta_config.yaml`（负向夹具）	`contracts/schema.contract.yaml`（事实源）
数据	旧库既有数据	`fixtures/dw.sql` 导入的 5 表 / 115 行
角色	只读参考，不作为实现	唯一构建目标

MySQL 这一层要注意：两套系统可能共用同一个 `3306` 实例，隔离靠的是**库名**而不是端口。所以「端口不同就是隔离」是错的；要核对的是 `--dry-run` 打出来的 `targets:`。同理，ES 和 Qdrant 也可能共用实例，隔离靠索引名与 collection 名。

`--fresh` 会 `DROP DATABASE`、删索引、删 collection。代码没有强制「目标名必须以 `_rebuild` 结尾」，所以默认值安全不等于任何配置都安全。绝对不要这样写：

```
# 危险：把目标指到旧系统
export REBUILD_META_DB=meta
export REBUILD_ES_INDEX=value_index
```

第一次跑 S2 前后各做一次隔离检查：旧系统计数不变，rebuild 计数符合预期。本机已验证的计数是 MySQL `5 / 24 / 6 / 9`、ES `76`、Qdrant `98` / `22`；旧系统是 `5 / 24 / 2 / 2`、`75`、`98`（1024 维）/ `8`（1024 维）。Qdrant 维度不同不是坏了，是两套独立 collection。

关于 `down -v`：任何时候都不要在没有确认的情况下执行。`make docker-reset` 会先按 Compose 的项目名优先级（环境变量 `COMPOSE_PROJECT_NAME` > `.env` 同名键 > `compose.yaml` 的 `name:`）解析出项目名，逐个列出真正要删的卷，要求输入 `yes`，然后把项目名显式交给 Compose。卷名猜错，删的就是别的东西。

还有一条容易被忽略的隔离：`fixtures/dw.sql` 通过 MySQL 的 `docker-entrypoint-initdb.d` 挂载导入，只在数据卷为空的那一次执行。改了 SQL 却不生效，不是脚本没挂上，而是卷里已经有数据。重新导入必须先 `make docker-reset`。

---

6. 测试金字塔与阶段门禁

DSWS 的测试按阶段分，不按「单元 / 集成」分，每个阶段回答一个具体的不确定性。图 29（`design/diagrams/06-testing/29-test-`

`pyramid.mmd`）是金字塔，图 30（`design/diagrams/06-testing/30-stage-gates.dot`）是 S0–S7 的门禁链。

这张图回答：五层测试各自覆盖什么？哪一层开始需要 MySQL / ES / Qdrant？哪些是「本机就能重复」的门禁？
读图：1 先看层级（单元/集成/端到端）→ 2 再看阶段门禁顺序 → 3 最后看失败注入覆盖了哪些码

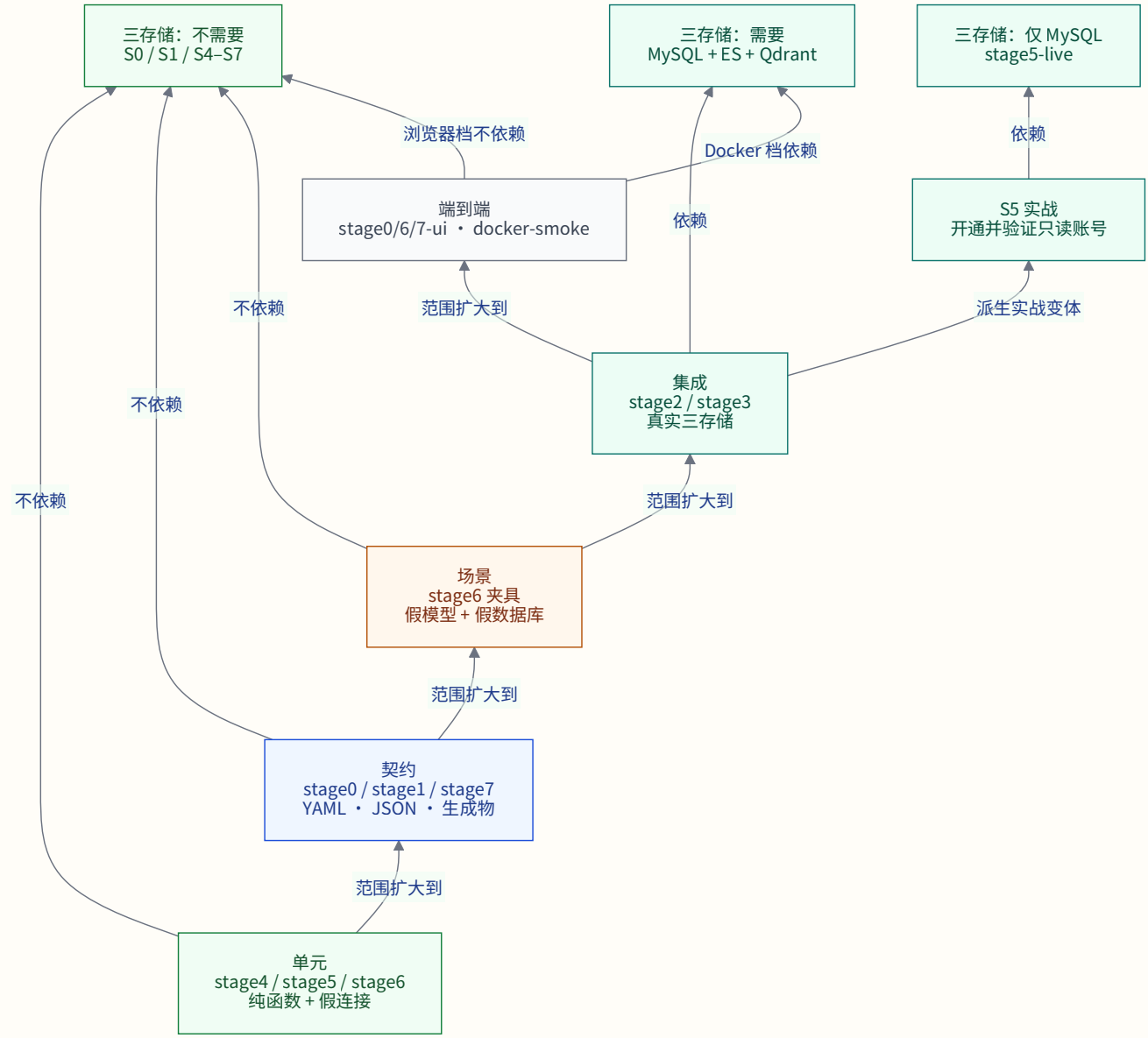


图 29 · 测试金字塔与三存储依赖

表 5-5 · 各阶段门禁与存储依赖

阶段	命令	需要三存储	核心验收
S0	`make stage0`	否	SSE 契约、12 步骤目录、前端解析与构建
S1	`make stage1`	否	5 表 / 24 列 / 6 指标；旧系统负向夹具必须红
S2	`make stage2`	是	三存储构建、manifest、计数对账、`NOOP: 0 writes`
S3	`make stage3`	是	四路召回 + 合并 + 过滤；20 题黄金集

阶段	命令	需要三存储	核心验收
S4	`make stage4`	否	10 题基线 + sqlglot AST 静态校验
S5	`make stage5`	否	注入语料 100% 拒绝 + 零误杀
S5 实战	`make stage5-live`	是（只要 MySQL）	只读账号开通 / 验证 + 两屏对比
S6	`make stage6`	否	16 错误码 + 6 类失败 + 校正闭环 + 取消
S7	`make stage7`	否	生成物新鲜度 + SSE 夹具对账 + 前端契约

这张图回答：每个阶段跑什么？「通过」的判据是什么？哪两个阶段需要先起三存储？

读图：1 先看层级（单元/集成/端到端） → 2 再看阶段门禁顺序 → 3 最后看失败注入覆盖了哪些码

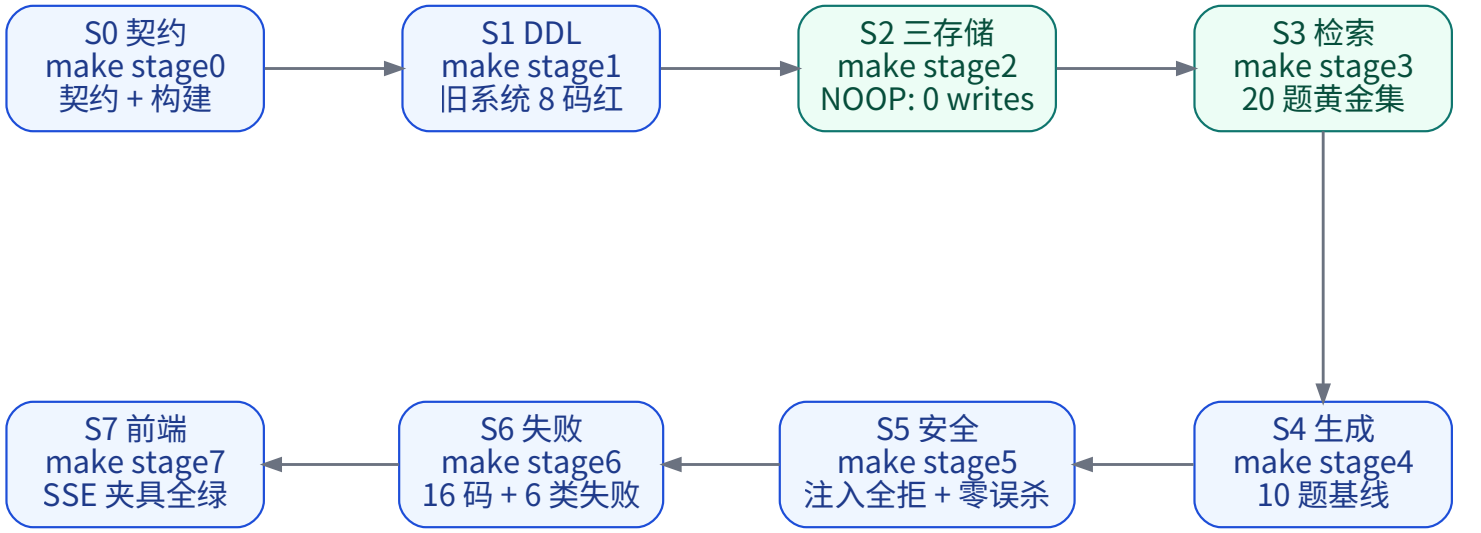


图 30 · S0–S7 阶段门禁链

这张表最重要的信息是「哪些阶段不需要三存储」。S0、S1、S4、S5、S6、S7 都能在一台只有 Python 与 Node 的机器上跑完；S2、S3 才需要三存储，S5 实战只需要 MySQL。这意味着新人可以先在干净环境里把六个阶段跑绿，再决定要不要动 Docker。它也意味着 CI 的分层可以很便宜：把不需要存储的阶段放在最前面。

从金字塔的角度看，底层是纯函数与契约测试：`tests/generate/test_guard.py` 只吃契约与 SQL 文本，`tests/security/test_policy.py` 只依赖 sqlglot，`tests/agent/test_errors.py` 只读 `contracts/errors.yaml`。这些测试没有网络、没有数据库，跑得最快，也最能定位。中间层是契约对齐：`test_s0_contract.py` 验 SSE 与步骤目录，`test_s1_schema_contract.py` 验 5 表 / 24 列 / 115 行，`tests/frontend/test_contract_gen.py` 验生成物与契约一致。顶层才是集成与场景：S2 写三存储、S3 跑 20 题黄金集、S6 跑 6 类失败、S0/S6/S7 的浏览器验收。

这个分层只为一件事：失败时知道该往哪一层查。纯函数测试红了，问题在逻辑；契约对齐红了，问题在事实源或生成物；集成测试红了，问题在存储或环境；浏览器用例红了，问题在 UI 状态机或回放夹具。如果所有测试都混在一起，一个红灯只会告诉你「有东西坏了」，不会告诉你「坏在哪一层」。

测试规模方面，仓库有 45 个后端测试文件（`find backend/tests -name 'test_*.py' | wc -l`），前端有 2 个测试文件（`frontend/src/lib/result.test.ts`、`frontend/src/lib/sse.test.ts`）。README 与 `docs/DOCKER-HANDOFF.md` 记录的全量后端结果是 **782 passed / 31 skipped**；`docs/SETUP-FROM-SCRATCH.md` 的阶段表记录的是更早一轮的分阶段数字（S0 19、S1 59、S4 108、S5 171、S6 107、S7 契约 45）。两处数字是不同时间点的快照，并不矛盾。

证据等级必须说清楚：`DOCKER-HANDOFF.md` 里的 782 是那次实测记录，标 ✓（针对那次环境）；**本设计工作树没有跑全量后端测试**，所以本章引用 782 时标 !。不要把「README 声称」升级成「本机已验证」。

阶段门禁另有两条经验，单独说：

第一，**门禁必须能红**。S1 的 `--expect-legacy-red` 要求旧系统夹具必须报错；S5 的注入语料要求 100% 拒绝；S6 的 6 类失败要求结局与夹具逐项吻合。一个永远通过的门禁，比没有门禁更危险。

第二，**验收脚本不能用 `assert`**。`python -O` 会把 `assert` 剥掉，脚本于是永远「通过」。`scripts/smoke_checks.py` 全部用显式返回码与中文结论；`backend/tests/test_smoke_checks.py` 还用 AST 扫描 `ast.Assert`，确保这条纪律本身不是空转。

---

7. 失败注入矩阵

S6 要做的是让每一种失败都有名字，不是让系统不报错。图 31（`design/diagrams/06-testing/31-failure-injection.mmd`）是失败注入矩阵，表 5-6 把它落成可查的对应关系。

这张图回答：失败从哪里注入？落到哪个错误码？属于 failure / empty / cancelled 哪一类？哪些会触发校正闭环？界面上长什么样？

读图：1 先看层级（单元/集成/端到端）→ 2 再看阶段门禁顺序 → 3 最后看失败注入覆盖了哪些码

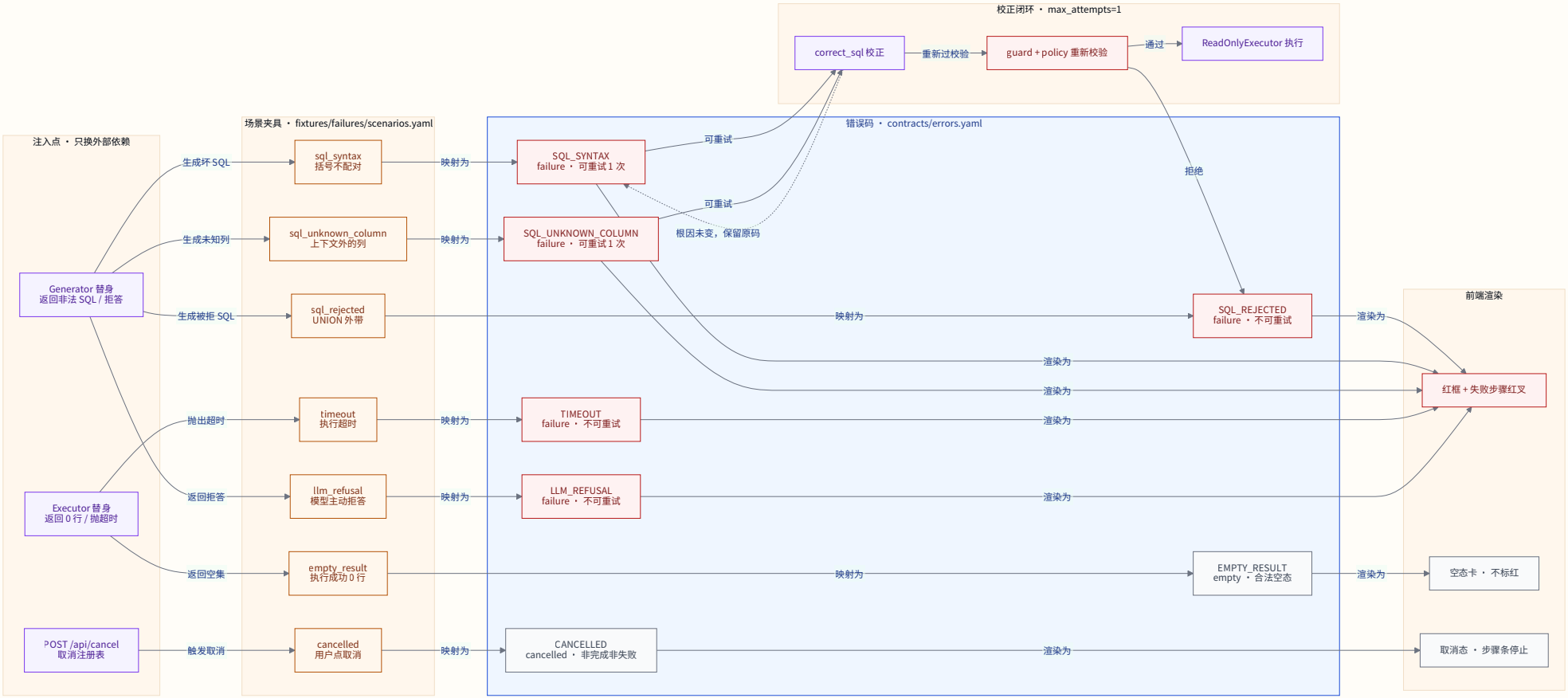


图 31 · 失败注入矩阵

表 5-6 · **失败与取消 → 错误码 → UI → 测试**

失败	错误码	class	用户看到什么	主要测试文件
SQL 被策略拒绝	`SQL_REJECTED`	failure	红框 + 失败步骤红叉	`backend/tests/security/test_policy.py` 、 `test_injection_corpus.py`
SQL 语法错误	`SQL_SYNTAX`	failure	红框，提示语法不合法	`backend/tests/generate/test_guard.py` 、 `agent/test_scenarios.py`

失败	错误码	class	用户看到什么	主要测试文件
引用不存在的列	`SQL_UNKNOWN_COLUMN`	failure	红框，指出列不在 schema	`backend/tests/generate/test_guard.py` 、`agent/test_scenarios.py`
查询成功但无数据	`EMPTY_RESULT`	empty	空态卡，**不标红**	`backend/tests/agent/test_scenarios.py` 、`frontend/src/lib/result.test.ts`
执行超时	`TIMEOUT`	failure	红框，提示已中止	`backend/tests/security/test_executor.py` 、`agent/test_scenarios.py`
用户取消	`CANCELLED`	cancelled	取消态，不算失败也不算完成	`backend/tests/agent/test_api.py` 、`test_orchestrator.py`
模型拒答	`LLM_REFUSAL`	failure	红框，提示现有数据无法回答	`backend/tests/agent/test_errors.py` 、`test_scenarios.py`

这张表要看的是 `class` 的三分法，不是那六个码：`failure` 是真失败，红框；`empty` 是合法但没有数据，空态卡，不标红；`cancelled` 是用户自己停的，既不是 done 也不是失败。把空结果画成红框，是旧系统审计里的一类静默失败；取消被当成完成，是另一类。

`retryable` 决定要不要走 `correct_sql` 校正闭环。安全策略的拒绝（`SQL_REJECTED`）永远不可重试：拿被拒的 SQL 去问模型「改一下」，等于给注入开重试的口子。校正最多 1 次，校正后的 SQL 必须重新过 `guard` 与 `policy` 两道关；校正器返回写操作或 UNION 都会被挡下，执行器一次都不会被调用。

失败场景的注入点是两个外部依赖：模型与数据库。`fixtures/failures/scenarios.yaml` 用「场景: <真实问题>」的写法驱动场景应用，上下文构建、提示词渲染、AST 校验、安全策略、校正闭环、SSE 序列化全是真代码。所以屏幕上的每个红叉都对应一次真实判定，不是摆拍。六类失败在浏览器里必须是**六种互不相同**的界面状态；`make stage6-ui` 就是验这一点。

---

8. 故障排查表

排障的顺序应该是「先分层，再定位，最后才改」。先看是环境问题、启动问题、数据问题还是应用问题；不要一上来就改代码。表 5-7 按症状组织。

表 5-7 · 常见故障

症状	先查什么	命令	修复	证据
镜像拉取失败	镜像源是否可达	`docker compose config --quiet`	改 `.env` 的 `*_IMAGE` 后重跑	`docs/DOCKER-DEPLOY.md` §6
`init` 退出 1	init 日志	`docker compose logs init`	修凭据/存储后 `docker compose run --rm --no-deps init`	同上
`dw_reader` Access denied	init 是否跑完	`docker compose logs init`	重新执行 init	同上
容器连库正常、宿主 CLI 却 Access denied	shell 与 `.env` 是否一致	`source scripts/export-docker-env.sh`	`unset` 旧变量后重新 source	`DOCKER-DEPLOY.md` §4
ES 反复退出（Linux）	`vm.max_map_count`	`sudo sysctl -w vm.max_map_count=262144`	调大后重启	`DOCKER-DEPLOY.md` §6
8080 被占用	宿主机端口冲突	`docker compose port frontend 80`	改 `FRONTEND_HOST_PORT`	同上
8100 被占用	容器后端与 `make dev-backend` 冲突	`docker compose ps`	改 `BACKEND_HOST_PORT` 或停一个	`DOCKER-DEPLOY.md` §2.1
改了 `fixtures/dw.sql` 数据没变	卷里已有数据	`docker compose logs mysql`	`make docker-reset` 后重新 up	`DOCKER-DEPLOY.md` §5
改了密码栈起不来	MySQL 只在空卷初始化密码	`docker compose ps -a`	`make docker-reset` 后重新 up	`DOCKER-DEPLOY.md` §4
npm / pip 拉包失败	默认源不可达	`docker compose logs backend frontend`	改 `NPM_REGISTRY` / `UV_INDEX_URL`	`DOCKER-DEPLOY.md` §6
页面能开但 SSE 不流式	nginx 是否关缓冲	`docker compose exec -T frontend nginx -T`	确认 `proxy_buffering off`，重建 frontend	`DOCKER-DEPLOY.md` §2
`S2 FAIL: unknown analyzer [ik_max_word]`	官方 ES 没有 IK	`docker compose logs init`	确认 S2 回退 `standard`	`DOCKER-DEPLOY.md` §6
查询一直 `INTERNAL_ERROR`	init / 只读账号	`docker compose ps -a`、`logs init`	重新 init 或配 `REBUILD_DW_RO_PASSWORD`	同上
`make stage3` 报 `No rule to make target`	checkout 太旧	`git log --oneline -1`	更新到含 S3 的提交	`SETUP-FROM-SCRATCH.md` §6

症状	先查什么	命令	修复	证据
`make stage7` 报生成物过期	手改了 `frontend/src/generated/`	`cd backend && uv run python -m app.scripts.contracts_gen --check`	重新生成，不手改	`SETUP-FROM-SCRATCH.md` §6
`make stage7` 报 SSE 夹具不一致	后端输出变了	`cd backend && uv run python -m app.scripts.record_sse --check`	重录并人工 review diff	同上
`make stage6-ui` 找不到 Chrome/Edge	浏览器未装	`echo $CHROME_PATH`	装 Chrome/Edge 或设 `CHROME_PATH`	`SETUP-FROM-SCRATCH.md` §6
UI 验收端口被占用	8102/5275 或 8103/5276 冲突	`netstat -ano | findstr :5276`	`API_PORT=... WEB_PORT=... make stageN-ui`	同上

排障时有一条反直觉的原则：没有证据不等于通过。`docker-smoke.sh` 在「SSE 响应太快、分辨不出是否流式」时，不会放行，而是要求拿出 nginx 的完整生效配置来定性；拿不到配置就判不通过。诊断也一样：日志里没写错误，不等于没有错误；要主动去查那个能证伪的指标。

推荐的顺序是四步。第一步，分清「环境」与「部署」：`.env` 缺了、解释器太老、curl 没装，属于环境，不该去翻业务日志。第二步，看 `docker compose ps -a` 的状态列，确定停在哪一环。第三步，只看那一环的日志，不要 `docker compose logs` 全量刷屏；`init` 的日志通常最短也最有用。第四步，拿一个正向证据和一个反向证据：正向证据是 `/api/health` 或 `/api/steps` 返回预期结构，反向证据是故意查一个越界问题，确认它按契约报错，而不是静默返回空。

还有一个细节：`docker compose port frontend 80` 是查询前端实际发布端口的唯一权威。不要自己 `sed` `.env` 去猜端口——用户在 shell 里 export 过 `FRONTEND_HOST_PORT` 时，`.env` 里的值和 Compose 实际用的值可能不是同一个，脚本会去戳一个没人监听的端口，最后报「360 秒内 /api/health 未就绪」，真正的原因（端口不对）一个字都不提。

先看哪一眼：页面 502 先看 `docker compose ps -a`。`init` 没退出 0 就先查 init；`init` 正常、backend 和 frontend 都 healthy，才轮到 nginx 反代。

9. 本机边界与未验证清单

本设计工作树的 Docker socket 没有权限，下面这些事项没有在本机端到端跑过，一律标 ?。

表 5-8 · 本机边界

项	状态	说明
`docker compose up` 全栈	?	当前 shell 无 Docker socket 权限，未从零起过
`--fresh` 写三存储	?	计数来自 `--dry-run` 与代码公式，不是实测写入
Docker 全栈 smoke	?	交接文档记录过 PASS，但本工作树未复跑
`make stage5-live` / `dw_reader` 实测	?	无运行中的 MySQL，只读账号未在本机开通
真实 LLM 生成 / 校正	?	无端点、无 Key；默认确定性生成器
浏览器验收（stage0-ui / 6-ui / 7-ui）	?	本机未装 Chrome/Edge，未跑
真实召回接入 `/api/query`	?	当前上下文由契约离线裁剪，S8 才收口
`--dry-run` 计数 5/24/6/9/76/98/22	✓	2026-10-27 本机跑过
旧系统计数未变	✓	2026-10-25/26 记录，见 `facts.md` §9
后端 782 passed	!	README 与交接文档声称；本工作树未跑

另有三条是当前不支持，跟验证无关：backend 固定单 worker，取消注册表是进程内状态，不能多副本；ES 单节点、关闭鉴权，只适合单机，不要暴露到公网；`fixtures/dw.sql` 是当前数据源，替换真实数据不在本部署范围内。S8 之前，不要承诺「前端能真实问数」——`/api/query` 跑的是真流水线，但上下文是契约离线裁剪的，真实召回与真实数据库还没接上。

10. 上线前检查清单

下面 20 项按「环境 → 启动 → 数据 → 安全 → 验收 → 运维」排列。每一项都能在仓库里找到对应的命令或文件，不写「确认一切正常」这种无法执行的条目。

☐ 1. `docker compose version` 为 v2，Docker Engine 可用。

- [ ] 2. 宿主机有 GNU Make、Python 3.8+、Bash、curl；Windows 用 Git Bash。
- [ ] 3. 内存至少 8 GB，ES 堆固定 512 MB。
- [ ] 4. `.env` 已由 `make docker-env` 生成，且不含 `change-me-*` 占位密码。
- [ ] 5. `.env` 未入库（`git status` 里看不到它）。
- [ ] 6. `QDRANT_IMAGE` 为 v1.5.0 或更新，`/readyz` 可用。
- [ ] 7. 8080 / 8100 / 3306 / 9200 / 6333 无冲突；8100 未同时被 `make dev-backend` 占用。
- [ ] 8. 只在本机使用时，`FRONTEND_BIND=127.0.0.1`；要给局域网才保留 `0.0.0.0`。
- [ ] 9. `make docker-up` 后 `docker compose ps -a` 显示五服务 healthy、`init` exited 0。
- [ ] 10. `curl -fsS http://127.0.0.1:8080/api/health` 返回 `status=ok`、`stage=S7`。
- [ ] 11. `/api/steps` 返回 12 个步骤，`correct_sql` 带 `conditional: true`。
- [ ] 12. `make docker-smoke` 输出 `DOCKER SMOKE PASS`。
- [ ] 13. smoke 的第 7 步确认 `dw_reader` 只有 SELECT，写操作被拒。
- [ ] 14. smoke 的第 8 步确认 S2 元知识对账通过。
- [ ] 15. 再跑一次 `docker compose run --rm --no-deps init`，输出包含 `meta: verify passed; skip rebuild`。
- [ ] 16. nginx 生效配置里，处理 `/api/` 的 location 是 `proxy_buffering off`。
- [ ] 17. 旧系统计数未被改动（`meta` / `value_index` / 旧 collection）。
- [ ] 18. 备份策略已确认：`make docker-down` 保留卷；`make docker-reset` 会删卷。
- [ ] 19. 日志入口已确认：`make docker-logs`、`docker compose logs -f backend`。
- [ ] 20. 已知边界已写进交接：单 worker、单节点 ES、真实 LLM 与真实召回未接。

这张清单里没有「跑一遍 782 个测试」这一项，因为全量测试属于代码门禁，不属于部署门禁；部署门禁要回答的是「这台机器上的这一套栈，是不是真的能服务」。两者不能互相替代：测试全绿但 `.env` 里是公开密码，仍然不能上线；smoke 全过但没跑过全量测试，也不能证明代码没回归。

11. 排障小结

- 启动顺序是条件链，不是建议；`init` 退出码为 0 之前，backend 不会起。
- 健康检查只证明进程活着；数据面要跑 `make docker-smoke`。
- 本工作树的 Docker 全栈是 ⍰；命令能写，不等于这台机器能跑。

06 附录

附录最容易犯的错，是跳过论证直接查表抄列名。`dim_region` 没有 `city` 列，`dim_product` 没有 `price` 列；记反了，SQL 看起来合理，校验时才会报未知列。

这份附录只做三件事：索引、速查、对账。数字都指回正文或 `design/inventory/facts.md`，✓ / ! / ? 保留原状态。

附录把散落在各章的可查项集中成八类速查表与索引：图、步骤、错误码、数据模型、命令、术语、文件、证据。正文里不必背，动手时回来查。

所有条目都以仓库当前基线为准：`eea5e75`（2026-10-29）。计数与 `design/inventory/facts.md` 一致；路径是相对仓库根目录的路径。

A. 32 张图索引

图分六个角度：架构（01–06）、运行时（07–13）、数据（14–19）、安全（20–24）、部署（25–28）、测试（29–32）。源文件统一放在 `design/diagrams/<角度>/` 下，扩展名 `.mmd`（Mermaid）或 `.dot`（Graphviz）；同名 `.md` 是说明。本索引只列路径，不嵌图；源文件列省略公共前缀 `design/diagrams/`。

表 A-1 · 图 01–16

编号	标题	角度	源文件	一句话
01	C4-L1 系统上下文	架构	`01-architecture/01-c4-context.mmd`	用户、浏览器、DSWS、三存储、可选 LLM 与旧系统的边界
02	C4-L2 容器图	架构	`01-architecture/02-c4-container.mmd`	前端、FastAPI、AgentRunner、构建 CLI 与三存储的容器关系
03	后端模块依赖	架构	`01-architecture/03-backend-modules.dot`	`backend/app/` 的 import 与调用方向
04	前端组件树	架构	`01-architecture/04-frontend-tree.mmd`	`App.tsx` 到 Composer / StepRail / ResultTable / ErrorCard

编号	标题	角度	源文件	一句话
05	阶段演进 S0–S8	架构	`01-architecture/05-stage-evolution.mmd`	每个阶段新增的能力、不确定性与可停点
06	构建时 vs 查询时	架构	`01-architecture/06-build-vs-query.mmd`	S2 退出后，查询时只读三存储
07	端到端请求生命周期	运行时	`02-runtime/07-end-to-end-request.mmd`	浏览器到 MySQL 再以 SSE 返回的完整链路
08	S2 构建流水线时序	运行时	`02-runtime/08-s2-build-sequence.mmd`	plan → Meta → ES → Qdrant → manifest → exit
09	S3 召回流水线时序	运行时	`02-runtime/09-s3-recall-sequence.mmd`	关键词 → 值召回 → 列/指标召回 → 合并 → 过滤
10	S4 生成 SQL 流水线	运行时	`02-runtime/10-s4-generate-sequence.mmd`	上下文八阶段 → 生成 → 输出校验 → 静态 guard
11	S5 安全校验与执行	运行时	`02-runtime/11-s5-security-sequence.mmd`	策略判定 → 只读执行器 → 结果或错误
12	S6 校正闭环状态机	运行时	`02-runtime/12-s6-correction-state.mmd`	校验 → 可重试则校正一次 → 再校验 → 执行
13	S7 SSE 事件序列	运行时	`02-runtime/13-s7-sse-sequence.mmd`	progress / result / error 到三个 UI 组件
14	数仓 ER 图	数据	`03-data/14-er-diagram.mmd`	5 表、主键、逻辑外键与粒度
15	契约模型	数据	`03-data/15-contract-model.mmd`	Table / Column / Metric / ValueDomain / ColumnMetric
16	三存储知识布局	数据	`03-data/16-three-store-layout.mmd`	一份契约怎么翻译成 Meta 行、ES 文档、Qdrant 点

表 A-2 · 图 17–32

编号	标题	角度	源文件	一句话
17	S2 数据 lineage	数据	`03-data/17-s2-lineage.mmd`	契约 + 数据 → BuildPlan → 三存储 + manifest
18	Meta 表结构	数据	`03-data/18-meta-schema.mmd`	`table_info` / `column_info` / `metric_info` / `column_metric` / `build_manifest`
19	值域分类决策树	数据	`03-data/19-value-domain-decision.mmd`	enum 写 ES，range 不写；`sync_values` 的边界
20	信任边界与攻击面	安全	`04-security/20-trust-boundary.mmd`	浏览器、前端、FastAPI、LLM、三存储、旧系统的信任线

编号	标题	角度	源文件	一句话
21	五层防御纵深	安全	`04-security/21-defense-in-depth.mmd`	白名单 → guard → policy → 只读执行器 → 错误通道
22	SQL 策略判定流程	安全	`04-security/22-sql-policy-flow.mmd`	单语句 → 只读 → 表/列白名单 → 常量 → 排序/限量
23	只读账号权限模型	安全	`04-security/23-readonly-grants.mmd`	`dw_reader` 只有 SELECT，应用账号与它分离
24	错误码分类与 UI 映射	安全	`04-security/24-error-taxonomy.mmd`	16 个码按 failure / empty / cancelled 分类
25	Docker Compose 拓扑	部署	`05-deployment/25-compose-topology.mmd`	六个服务、三个卷、一个默认网络
26	端口与网络暴露面	部署	`05-deployment/26-port-exposure.mmd`	哪些端口只绑 `127.0.0.1`，哪个对外
27	启动顺序与健康检查	部署	`05-deployment/27-startup-order.mmd`	三存储 → init → backend → frontend
28	旧系统 / rebuild 隔离	部署	`05-deployment/28-legacy-isolation.mmd`	`*_rebuild` 命名、5273 / 5173、8100
29	测试金字塔与阶段门禁	测试	`06-testing/29-test-pyramid.mmd`	单元 / 契约 / 集成 / 场景 / 端到端的分层
30	S0–S7 阶段门禁	测试	`06-testing/30-stage-gates.dot`	每阶段的硬命令与「通过」判据
31	失败注入矩阵	测试	`06-testing/31-failure-injection.mmd`	6 类失败 → 错误码 → UI 状态
32	证据台账	测试	`06-testing/32-evidence-ledger.mmd`	✓ / ! / ? 三类证据的来源与去向

使用索引时注意两件事：编号在全书唯一，不按目录重新计数；某张图若被合并，合并说明写在对应 `.md` 里，索引不删除编号。

读图有一个顺序建议：先看 01 / 02 建立「谁是谁」的地图，再看 07 建立「一次请求经过谁」的时间感，然后按你当前要改的部分跳到对应角度。改数据模型看 14–19，改安全策略看 20–24，部署排障看 25–28，写测试看 29–32。不要从第一张图顺序读到第三十二张——图集是索引，不是小说。

按角度清点，六个角度合计 32 张，既不重复编号，也不把「合并图」算成两张：

表 A-3 · 按角度清点

角度	编号范围	张数	主要回答
架构	01–06	6	系统边界、容器、模块、组件、阶段、构建/查询分界
运行时	07–13	7	一次请求与各阶段流水线在时间上经过谁
数据	14–19	6	表、契约、三存储布局、lineage、Meta、值域
安全	20–24	5	信任边界、纵深、策略、只读账号、错误分类
部署	25–28	4	Compose 拓扑、端口暴露、启动顺序、旧系统隔离
测试	29–32	4	金字塔、阶段门禁、失败注入、证据台账

图号和章节不是一一对应：第 1–2 部分按阶段讲能力，会同时引用架构图与运行时图；第 3 部分讲数据与安全；第 5 部分集中使用 25–31。找不到某张图时，先看它的 `.md` 说明里的「这张图回答」，再回到正文对应小节，不要只凭文件名猜内容。

每张图都配一份 `.md`，里面有一句话结论、这张图回答什么问题、图例、关键元素、证据文件与已知边界。只看 SVG 容易把「推断」当成「事实」；`.md` 里的证据表才是判断依据。

B. 12 个步骤

步骤目录的唯一事实源是 `contracts/steps.yaml`，前端启动时从 `GET /api/steps` 拉取，不硬编码。12 个步骤里有且只有一个条件步骤：`correct_sql`。

表 B-1 · 步骤目录

#	阶段	id	label	条件执行	证据 / 说明
1	1	`extract_keywords`	抽取关键词	否	`contracts/steps.yaml`
2	2	`recall_column`	召回字段信息	否	同上
3	2	`recall_value`	召回字段取值	否	同上
4	2	`recall_metric`	召回指标信息	否	同上
5	3	`merge_retrieved_info`	合并召回信息	否	同上
6	4	`filter_table`	过滤表信息	否	同上

#	阶段	id	label	条件执行	证据 / 说明
7	5	`filter_metric`	过滤指标信息	否	同上
8	6	`add_extra_context`	添加额外上下文	否	同上
9	7	`generate_sql`	生成SQL	否	同上
10	8	`validate_sql`	校验SQL	否	同上
11	9	`correct_sql`	校正SQL	是	`conditional: true`；未触发时后端显式发 `skipped`
12	10	`run_sql`	执行SQL	否	同上

`contracts/steps.yaml` 里同时有 `stages`（10 段）与每个节点的 `stage` 字段；`backend/tests/test_steps_stages.py` 守着「12 个节点不重不漏地落进 10 段」。这是读图分组，不是新事件——SSE 仍然每个节点一条 `progress`。

`correct_sql` 的「条件执行」不是「可能永远灰着」。旧前端硬编码 12 个节点，导致校正节点永远 pending；新契约要求后端在未触发时显式发 `status: "skipped"`，前端据此显示「未触发」，而不是一个永远转圈的灰块。

每个步骤在 SSE 里由一条 `progress` 事件推进，状态只有四种合法值；`duration_ms` 的取值跟着状态走：

表 B-2 · 步骤状态与耗时

status	含义	duration_ms	前端表现
`running`	正在执行	必须为 `null`	转圈或高亮当前节点
`success`	成功完成	必须是数字	打勾，显示耗时
`error`	该步失败	必须是数字	红叉，错误卡指向该步
`skipped`	条件未触发	必须为 `null`	显示「未触发」，不转圈、不算失败

这张小表解释了一个常见误解：`skipped` 不是 `success` 的别名，也不是失败。它表示「这个条件步骤这次不需要执行」，所以既不能打勾，也不能标红。前端如果把它当成功，用户会以为校正跑过了；当失败，用户会以为系统坏了。两种情况都是协议误读。

按职责把这 12 步分成四段，查问题时更容易定位：

- 第 1–4 步是「取料」：抽关键词，再从三个存储分别取列、取值、指标。取料阶段失败通常不是代码逻辑，而是 S2 没构建好或索引名不对。

- 第 5–8 步是「整理」：合并四路召回，过滤掉与问题无关的表与指标，再补上时间范围、粒度之类的额外上下文。这一段决定模型看到的上下文是不是干净。
- 第 9–11 步是「生成与校验」：生成 SQL、静态校验、必要时校正一次。`validate_sql` 失败会带错误码，`correct_sql` 只在 `retryable` 为真时出现。
- 第 12 步是「执行」：只读执行器跑单条 SELECT，结果以 `result` 事件返回，或按错误码以 `error` 收尾。

注意 S4 只到第 11 步，不执行 SQL；S5 才把第 12 步接上。所以「S4 通过」不等于「SQL 能跑」，它只证明生成了合法、只读、在白名单内的 SQL。

C. 16 个错误码

错误码表来自 `contracts/errors.yaml`。三个 `class` 决定 UI：`failure` 红框、`empty` 空态不标红、`cancelled` 取消态。`retryable` 决定是否进入校正闭环；`SQL_REJECTED` 永远不可重试。

表 C-1 · 错误码全表（16/16）

code	label	class	retryable	用户看到什么
`SQL_REJECTED`	SQL 被安全策略拒绝	failure	否	红框，提示未通过安全策略，已拒绝执行
`SQL_SYNTAX`	SQL 语法错误	failure	是	红框，提示 SQL 无法解析
`SQL_UNKNOWN_COLUMN`	引用了不存在的列	failure	是	红框，提示列不在 schema 里
`EMPTY_RESULT`	查询成功但没有数据	empty	否	空态卡，**不标红**
`LLM_REFUSAL`	模型明确拒答	failure	否	红框，提示现有数据无法回答
`TIMEOUT`	执行超时	failure	否	红框，提示已中止
`CANCELLED`	用户取消	cancelled	否	取消态，不算完成也不算失败
`SQL_UNKNOWN_TABLE`	引用了不存在的表	failure	是	红框，提示表不在白名单
`SQL_NOT_READ_ONLY`	不是只读查询	failure	否	红框，提示只允许 SELECT
`SQL_MULTIPLE_STATEMENTS`	不是单条语句	failure	否	红框，提示只能有一条语句
`SQL_CONSTANT_PROJECTION`	投影了常量	failure	是	红框，提示 SELECT 列表不能有字面量
`SQL_MISSING_ORDER_BY`	缺 ORDER BY	failure	是	红框，提示排名意图必须排序

code	label	class	retryable	用户看到什么
`SQL_MISSING_LIMIT`	缺 LIMIT	failure	是	红框，提示限定名次必须限量
`SQL_EMPTY`	SQL 为空	failure	否	红框，提示没有可执行 SQL
`SQL_EXECUTION_ERROR`	数据库执行出错	failure	否	红框，提示数据库执行失败
`INTERNAL_ERROR`	系统内部错误	failure	否	红框，兜底文案，不泄露内部细节

分类统计：`failure` 14 个，`empty` 1 个（`EMPTY_RESULT`），`cancelled` 1 个（`CANCELLED`）。可重试的是 `SQL_SYNTAX`、`SQL_UNKNOWN_COLUMN`、`SQL_UNKNOWN_TABLE`、`SQL_CONSTANT_PROJECTION`、`SQL_MISSING_ORDER_BY`、`SQL_MISSING_LIMIT` 这 6 个，其余 10 个都不可重试。校正策略是 `max_attempts: 1`，校正后重新过 `guard` 与 `policy`。

读这张表要和代码里的映射规则一起读。生成器或校验层可能产生表里没有的内部码（例如 `OUT_OF_RANGE`、`UNSUPPORTED_INTENT`、`INVALID_OUTPUT`），进入通道时统一映射成 `LLM_REFUSAL`，原码保留在 `detail.reason_code`。这样既不丢名字，也不需要在两张表之间手工同步。

`class` 的三个取值是 UI 分支，不是文案分类：前端拿到 `error` 事件后，先看 `class` 再决定渲染哪个组件。`failure` 走红框与失败步骤红叉；`empty` 走空态卡，绝不能标红；`cancelled` 走取消态，步骤条停止推进。把 `empty` 当 `failure` 是审计 FP-8 记录的旧缺陷；把 `cancelled` 当 `done` 是另一条旧缺陷。

`retryable` 也不是「重试一下也许能好」。它只表示这条失败是否允许进入 `correct_sql`；`SQL_REJECTED` 永远为假，因为拿被策略拒绝的 SQL 去问模型「改一下」，等于给注入开重试的口子。`TIMEOUT` 与 `SQL_EXECUTION_ERROR` 同样不可重试：重试只会再超时或再报错一次。

D. 5 表 24 列 6 指标

数据契约在 `contracts/schema.contract.yaml`。5 张表、24 列、6 个指标、9 条列—指标关系；数据覆盖 2025 Q1，115 行事实。

表 D-1 · 五张表

表名	粒度	列数	关键列
dim_region	一行一个 region_id（省级）	4	region_id、province、region_name、country
dim_customer	一行一个 customer_id	4	customer_id、customer_name、gender、member_level
dim_product	一行一个 product_id	4	product_id、product_name、category、brand
dim_date	一行一个 date_id（一天）	5	date_id、year、quarter、month、day
fact_order	一行一个 order_id（一单）	7	order_id、四个外键、order_quantity、order_amount

合计 4 + 4 + 4 + 5 + 7 = 24 列。两个容易记错的点：dim_region 没有 city 列，粒度只到省；dim_product 没有 price 列，单价只能由金额与件数推导。

表 D-2 · 六个指标

指标	公式	粒度	期望值
GMV	SUM(fact_order.order_amount)	订单	279159.5
AOV	SUM(order_amount) / COUNT(DISTINCT order_id)	订单	2427.47
订单量	COUNT(DISTINCT fact_order.order_id)	订单	115
销量	SUM(fact_order.order_quantity)	订单	622
平均件单价	SUM(order_amount) / SUM(order_quantity)	件	448.81
平均每单件数	SUM(order_quantity) / COUNT(DISTINCT order_id)	订单	5.41

AOV 的 rejected_values 明确排除了 448.81（平均件单价）与 5.41（平均每单件数）。这三个数长得很像，旧系统把它们混成一个「客单价」，是 S1 负向夹具要抓的核心错误之一。

逻辑外键四条：fact_order.customer_id → dim_customer.customer_id、product_id → dim_product.product_id、date_id → dim_date.date_id、region_id → dim_region.region_id。物理 DDL 不一定有外键约束，契约表达的是查询时的连接语义。

9 条列—指标关系记录在 column_metric 表里，回答的是「这个问题该看哪一列」。例如 销售额 指向 fact_order.order_amount，订单数 指向 fact_order.order_id，件单价 同时指向前两者。列级别名与指标级别名共用一套空间，所以「销售额」只放在指标 GMV

上，不放在 `order_amount` 列上——否则同一个词会召回两条互相冲突的元知识。

指标与粒度的关系还要单独记一笔：`GMV`、`订单量`、`AOV`、`平均每单件数` 的粒度是「订单」，`销量` 与 `平均件单价` 的粒度是「件」。粒度写错，SQL 的 `GROUP BY` 就会错。`fact_order` 一行一单，`order_quantity` 是单内件数，不是订单数；这是全章最容易被误读的一列。

E. 命令速查

表 E-1 · 阶段门禁

命令	作用	需要三存储
`make install`	安装后端与前端依赖	否
`make stage0`	契约 + 前端协议 + 构建	否
`make stage0-ui`	真实浏览器冒烟（临时 8101/5274）	否
`make stage1`	契约 ↔ DDL/数据；旧系统必须红	否
`make stage2`	dry-run → `--fresh` → `--verify` → 幂等	是
`make stage3`	三存储对账 + 20 题召回 + CLI YAML	是
`make stage4`	10 题基线 + 生成层测试	否
`make stage5`	策略 + 36 条注入语料 + 零误杀	否
`make stage5-live`	只读账号开通 / 验证 + 两屏对比	仅 MySQL
`make stage6`	16 错误码 + 6 类失败 + 校正 + 取消	否
`make stage6-ui`	六类失败各一个浏览器用例	否（要浏览器）
`make stage7`	生成物新鲜度 + SSE 夹具 + 前端契约	否
`make stage7-ui`	13 条 F 缺陷各一个浏览器用例	否（要浏览器）
`make baseline`	单独打印 10 题基线表	否

表 E-2 · Docker 与诊断

命令	作用
`make docker-env`	生成 `.env`（已存在则跳过）

命令	作用
`make docker-up`	构建并启动五服务 + 一次性 init
`make docker-down`	停止并删除容器与网络，保留卷
`make docker-logs`	跟踪全部服务日志
`make docker-smoke`	八步完整验收
`make docker-reset`	输入 `yes` 后 `down -v`，删三个卷
`docker compose ps -a`	看全部服务状态，含已退出的 init
`docker compose logs -f backend`	只看后端日志
`docker compose port frontend 80`	查前端实际发布的宿主机端口
`docker compose exec -T frontend nginx -T`	取 nginx 完整生效配置
`docker compose run --rm --no-deps init`	重跑一次性初始化
`source scripts/export-docker-env.sh`	把 `.env` 导出为宿主 CLI 的 `REBUILD_*`
`docker compose config --quiet`	校验 Compose 文件（不打印密码）

两条纪律：`docker compose config` 不加 `--quiet` 会把 `.env` 里的真实密码打印出来；`docker compose down -v` 不要手动执行，走 `make docker-reset`。

典型组合有三条。第一次上机：`make install` → `make stage0` → `make stage1`，先确认不依赖存储的部分是绿的。准备构建：`make docker-env` → `docker compose up -d mysql elasticsearch qdrant` → `source scripts/export-docker-env.sh` → `make stage2` → `make stage3`。全量交付：`make docker-up` → `curl -fsS http://127.0.0.1:8080/api/health` → `make docker-smoke`。

排障时优先用「只读」命令：`docker compose ps -a`、`logs`、`port`、`exec -T ... nginx -T`。不要一上来就 `down` / `rm` / `reset`；先确定问题在哪一层，再决定要不要动容器。`docker compose run --rm --no-deps init` 会重跑初始化的全部步骤，属于「有副作用但可重复」的操作，跑之前先确认三存储已经在跑。

F. 术语表

表 F-1 · **术语**

术语	含义
契约（contract）	前后端与构建/查询共用的唯一事实源，放在 `contracts/`；改契约不改代码是缺陷
构建时（build time）	S2 一次性 CLI 运行期间；结束后进程退出，查询时不再运行
查询时（query time）	用户发起问数到 SSE 收尾的链路；只读三存储，不写
召回（recall）	从 ES / Qdrant / Meta 取回列、取值、指标与表信息
值域（value domain）	一列允许的取值集合；enum 写 ES，range 不写
manifest	S2 构建结束时写的对账清单，记录计数与目标，供 `--verify` 使用
SSE	Server-Sent Events，后端以 `data: <json>` 逐条推送 progress / result / error
幂等（idempotent）	同一输入重复构建不产生新写入；S2 用 `--assert-idempotent` 验证
只读执行器	`ReadOnlyExecutor`，只允许单条 SELECT，带 `max_execution_time`
校正闭环	可重试错误进入 `correct_sql`，最多一次，校正后重新过 guard 与 policy
阶段门禁	每个 S 阶段的硬验收命令，必须能红也能绿
证据等级	✓ 已验证 / ! 推断 / ? 未验证，禁止把 ! 写成 ✓
一次性任务	`init` 容器，成功退出 0 后不再运行；`ps` 要加 `-a` 才看得到
假阳性	验收脚本报 PASS，但它验的东西根本没验；例如用 `assert` 写的检查
两屏对比	S5 实战里旧执行器接受、新策略拒绝的并排演示

表 F-2 · **工程与测试术语**

术语	含义
契约测试	只验契约与代码是否一致、不碰外部依赖的测试，如 `test_s0_contract.py`
黄金集	20 道问题与期望召回结果的固定集合，S3 用它防召回回归
场景应用	S6 的验收用 FastAPI 工厂，替换模型与数据库，其余全真
回放夹具	S7 录下的 SSE 响应，浏览器验收时按场景 id 回放
事实源	同一件事只允许有一个可编辑来源；契约就是计数与形状的事实源
阶段（S0–S8）	每阶段消灭一个不确定性，并有独立门禁；S8 尚未实现

G. 文件地图

表 G-1 · 什么时候读哪个文件

路径	作用	什么时候读
`compose.yaml`	六服务、卷、端口、健康检查	部署 / 排障
`Makefile`	阶段门禁与 docker 命令的入口	跑验收
`contracts/schema.contract.yaml`	5 表 / 24 列 / 6 指标的事实源	改数据模型 / 查口径
`contracts/steps.yaml`	12 个步骤	改流水线 / 查步骤
`contracts/errors.yaml`	16 个错误码与校正策略	改失败通道
`contracts/sse.schema.json`	3 种 SSE 事件	改前端协议
`contracts/sql.policy.yaml`	只读、单语句、白名单等策略	改安全边界
`backend/app/main.py`	FastAPI 入口、五个端点	查 API
`backend/app/agent/orchestrator.py`	查询时编排	查步骤顺序
`backend/app/build_meta_knowledge.py`	S2 CLI 入口	构建 / 对账
`backend/app/security/policy.py`	AST 策略判定	排查拒绝原因
`backend/app/security/executor.py`	只读执行器	排查执行错误
`scripts/docker-smoke.sh`	八步验收	部署后验收
`scripts/smoke_checks.py`	smoke 的具体判据	改验收 / 查假阳性
`scripts/export-docker-env.sh`	导出宿主 CLI 凭据	跑 S2/S3/S5-live 前
`docs/DOCKER-DEPLOY.md`	部署手册	首次部署 / 轮换密码
`docs/SETUP-FROM-SCRATCH.md`	从零搭建与故障表	新机器 / 查症状
`fixtures/failures/scenarios.yaml`	6 类失败剧本	改失败通道 / 跑 S6
`design/inventory/facts.md`	共享事实底稿	引用任何计数前

文件地图的使用方式是「从问题倒推」：想知道一个端口从哪来，查 `compose.yaml`；想知道一个错误码为什么出现，查 `contracts/errors.yaml` 与 `backend/app/agent/errors.py`；想知道一次查询走了哪几步，查 `contracts/steps.yaml` 与 `backend/app/agent/orchestrator.py`。不要从目录树第一行开始读，目录树不告诉你优先级。

`docs/` 与 `design/` 的分工也要分清：`docs/` 是随代码维护的操作文档，`design/` 是这次重建的设计产物与书稿。两者冲突时，以 `contracts/` 与 `compose.yaml` 的当前内容为准，`design/inventory/facts.md` 是仲裁记录。

H. 证据台账

表 H-1 · ✓ 已验证

事实	证据
拉取基线 `eea5e75`	`git pull --ff-only` 成功
`--dry-run` 计数 5 / 24 / 6 / 9 / 76 / 98 / 22	2026-10-27 本机输出
旧系统计数未变	2026-10-25/26 本机记录
`make stage0` – `stage7` 曾在 Windows / Linux 跑绿	`docs/SETUP-FROM-SCRATCH.md` §7
Docker smoke 八步通过	`docs/DOCKER-HANDOFF.md` §6.1（那次环境）
后端 782 passed / 31 skipped	`docs/DOCKER-HANDOFF.md` §5.1（那次环境）
端口 / 服务名 / 卷名	`compose.yaml` 逐字

表 H-2 · ! 推断

事实	推导链
写入总数 241	44 Meta 行 + 76 ES + 98 + 22 Qdrant + 1 manifest，按代码公式
本工作树的 782 测试	README 与交接文档声称，本工作树未跑
阶段测试数会随代码增长	文档快照来自不同时间点
`/api/health` 能证明进程活着	只证明 FastAPI 起来了，不证明数据面

表 H-3 · ? 未验证

事实	缺什么
Docker 全栈从零启动	本机无 Docker socket 权限
`--fresh` 真实写三存储	同上；计数来自 dry-run
`make stage5-live` 只读账号	无运行中的 MySQL
真实 LLM 生成 / 校正	无端点、无 Key

事实	缺什么
浏览器验收 stage0-ui / 6-ui / 7-ui	本机未装 Chrome/Edge
真实召回接入 `/api/query`	S8 才收口，当前上下文离线裁剪

使用这份台账的规则很简单：写任何结论前，先在表 H-1 找到它；找不到就降到 ⚠，连推导链都写不清就标 ❓。❓ 不是羞耻，把 ❓ 写成 ✅ 才是。

台账还要定期复查。代码一改，昨天的 ✅ 可能今天就变成 ⚠：例如 README 里的测试数会随新增用例变化，Docker 交接文档里的「线上栈在跑」也会随环境消失。复查时不要只改数字，要连带改证据行；没有证据行的数字，一律按 ⚠ 处理。

这份台账只记录「会写进书里的结论」，不记录过程日志。过程日志留在 `design/inventory/` 的交接文件里；书里的证据行要能让人顺着一条命令或一个文件行号复核。三条状态决定读者该不该相信下一句话。